Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

PAPERCLIP

AI AGENT COMPANY OS

Manage AI Agents with Paperclip (TGS-2023036646)

TGS Ref No: TGS-2023036646

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	28 August 2026	Initial release — Learner Guide for Manage AI Agents with Paperclip (TGS-2023036646): four topics, six hands-on labs, each building a different simulated one-person AI agent company on synthetic data.	Dr. Alfred Ang
2.0	28 August 2026	Major expansion to a full reference guide, rewritten against the official documentation at docs.paperclip.ing. Adds Parts A-D (foundations, building, running, governing), Part E (expanded lab walkthroughs), Part F (advanced operations and integration) and Part G (command and API reference), plus appendices for defaults, state machines and troubleshooting. Corrects 22 technical statements against the official specification — notably the seven issue states, the five heartbeat triggers, the three parallel budget scopes with 80%/100% thresholds, the deployment modes and the adapter type keys.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the WSQ course Manage AI Agents with Paperclip (TGS-2023036646), conducted by Tertiary Infotech Academy Pte Ltd. It gives the full step-by-step procedure for all six hands-on labs, organised by the four course topics. Each lab builds a DIFFERENT simulated one-person AI agent company — short-video production, digital marketing, finance, accounting, stock-trading research and HR people operations — so you see the same control model applied across six very different businesses.

Paperclip is an organisational control plane for AI agents: an open-source Node.js server with a React UI that gives a team of agents an org chart, issues, budgets, approval gates and a durable audit trail. It does not replace your agent runtime. The runtimes — Claude Code, Codex, Cursor, Gemini CLI, a bash process or an HTTP bot — still do the reasoning and the tool use; Paperclip schedules, budgets and audits them. Keeping that boundary clear is the single most useful idea in this course.

Use this guide alongside the course slides and the lab folders in labs/ of the course repository. Every data set in this course is SYNTHETIC and fabricated for training. Never load real company, market, financial or candidate data into a practice instance, and never place a real credential in a prompt, a lab note or Git.

Course Learning Outcomes

- LO1: Identify agentic AI opportunities and perform feasibility scans to determine when a Paperclip-managed multi-agent company is the appropriate solution.
- LO2: Utilise Paperclip to integrate agent runtimes, data and skills, and develop a goal-aligned AI agent company with an org chart, workspaces and budgets.
- LO3: Perform tests in Paperclip to verify that agents, issues, hand-offs and routines function correctly between disparate components.
- LO4: Resolve technical issues in Paperclip and implement modifications to the agent company, its governance controls and its cost boundaries.

Before You Start — Environment Setup

What you need

- Node.js 20 or later, and npm/npx. Paperclip will not onboard on an older runtime — check with `node --version` first.
- A NORMAL, non-elevated shell. Close any Administrator or root terminal before you begin: the embedded PostgreSQL refuses to start under an elevated user.
- One trainer-approved agent runtime (for example Claude Code, Codex or Gemini CLI), already installed AND authenticated OUTSIDE Paperclip.
- Git, used to baseline each agent workspace so that every later change is a readable, attributable diff.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Permission to create a local practice folder and to download the Paperclip npm package.
- The course lab repository, cloned from the link on your LMS course page.

Install and verify Paperclip

Onboarding downloads Paperclip, writes the local configuration, initialises the embedded PostgreSQL database, starts the server on port 3100 and normally opens your browser. Run it from a normal user shell — never with sudo. Then prove the control plane is healthy BEFORE you create any company: the diagnostic and the health endpoint together are your clean baseline, and every later adapter or budget failure is far easier to isolate because you recorded one.

```
node --version                # must report v20 or later
npm --version
npx --registry https://registry.npmjs.org paperclipai onboard --yes
npx paperclipai doctor        # fix the EARLIEST error first
curl -sS http://localhost:3100/api/health   # expect status ok
```

Conventions used in every lab

- Commands are run from a normal user terminal in the lab's practice root. A leading sudo is never required.
- Each lab uses its own simulated company and its own synthetic data set, supplied in that lab's data/ folder.
- Each agent gets a DEDICATED workspace folder; artifacts must land in the right one, and a marker in the wrong root is a boundary failure.
- Every issue is created UNASSIGNED — assignment is the trigger that wakes an agent.
- Bound every wait (about 10 minutes). An unbounded wait hides a failed adapter behind apparent patience.
- Record evidence as you go in evidence/lab0N-*.md, and never copy a key, token, cookie or connection string into it.

Topic 01 — Fundamentals of Agentic AI and Paperclip (25%)

Agentic AI foundations · the control-plane model · the twelve subsystems · feasibility and when NOT to use multi-agent

Key concepts

- Agentic AI adds reasoning, tool use, memory and autonomy on top of a language model, so the system completes work rather than only producing text.
- Paperclip is an organisational control plane — a Node.js server with a React UI that coordinates agents, goals, work, cost and governance. It is not an agent framework and it does not perform the reasoning itself.
- The defining separation: Paperclip manages the company; adapters run the agents. Organisational authority, runtime capability and verification evidence stay apart.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- A company is the boundary for one mission, its human members, AI employees, projects, issues, budget and activity history. One deployment holds many companies.
- The four pillars are the agentic task manager, the org chart for agents, agent employee training (skills and evaluations) and the agentic OS (runtime, sandboxing, MCP, RBAC and cost control).
- Twelve subsystems make up the platform: identity, org chart, work, heartbeat, workspaces, governance, budget, routines, plugins, secrets, activity and portability.
- A heartbeat is a bounded execution window with FIVE triggers: a schedule/interval tick, a task assignment, an @-mention in a comment, a manual Run Heartbeat, or an approval being resolved. Between heartbeats the agent is dormant — not idling and not billing.
- Work is checked out atomically with an execution lock, so two heartbeats can never double-work the same issue or race past a budget ceiling.
- 'If it can receive a heartbeat, it's hired' — adapter type keys include claude_local, codex_local, gemini_local, opencode_local, cursor, pi_local and hermes_local, plus process, http and openclaw_gateway.
- Feasibility comes before architecture: use one agent when the inputs, tools, authority and verification are the same across the whole task.
- Add a specialist only when different expertise is genuinely required, or when evidence must be produced independently of the agent that will use it.
- Installation is ready only when Node.js, the Paperclip configuration, the embedded database, server health, the browser UI and the doctor diagnostic all agree.

Hands-on labs for this topic

- Lab 1 — Install Paperclip and Prove the Control Plane (ClipReel Studio) (ClipReel Studio (Short Video Production)). Full walkthrough: Part E, section E1. Lab folder: labs/lab-01-*.

Note: Every lab data set is synthetic and fabricated for training. Never substitute real company, market, financial or candidate data, and never record a credential in your evidence notes.

Topic 02 — Building AI Agents with Paperclip (25%)

Install and verify · company and mission · the CEO · org chart · runtime adapters · workspaces, instructions and secrets

Key concepts

- Onboarding with `paperclipai onboard --yes` downloads Paperclip, writes the configuration, initialises the embedded PostgreSQL and serves the UI on port 3100.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Deployment mode is a security decision: local_trusted is the default and requires no login; authenticated+private and authenticated+public widen reach and require real authentication.
- The onboarding wizard captures the company name, the mission statement and the team lead, then verifies connectivity to your chosen local agent runtime.
- A mission must be measurable. 'Grow the channel' cannot be verified; 'publish 8 short videos per month at ≥5% average watch-through' can.
- Paperclip enforces a strict hierarchy: every agent reports to exactly ONE manager, and the CEO is the only agent with no manager. The CEO owns goal alignment, decomposition and final synthesis, and may propose further hires under approval.
- Every agent carries a role, a manager, an adapter, an instruction file, a heartbeat and an individual budget. Those six fields are the agent's whole contract.
- A role instruction states purpose, allowed inputs, the output schema, explicit exclusions and the escalation path — not a personality.
- Each agent gets a dedicated workspace: a separate local root for one role's files and evidence, isolated with git worktrees so changes are attributable.
- Secrets live in the runtime or in Paperclip's secret store with scoped injection at heartbeat time. They never appear in prompts, lab notes or Git.
- Skills are reusable agent capabilities imported from a path or URL into an org-wide library, then attached to specific agents — capability without retraining.
- Adapter readiness is proved by a marker run, not by the presence of configuration: an agent that cannot produce an observable artifact is not connected.
- Diagnose along a ladder — host, control plane, adapter, agent — and fix the earliest broken link before changing anything downstream.

Hands-on labs for this topic

- Lab 2 — Create the BrightFunnel Company, Mission and CEO (Digital Marketing) (BrightFunnel Marketing (Digital Marketing)). Full walkthrough: Part E, section E2. Lab folder: labs/lab-02-*.
- Lab 3 — Build the Meridian Reporting Tree and Validate Runtime Boundaries (Finance) (Meridian Finance Desk (Finance)). Full walkthrough: Part E, section E3. Lab folder: labs/lab-03-*.

Note: Every lab data set is synthetic and fabricated for training. Never substitute real company, market, financial or candidate data, and never record a credential in your evidence notes.

Topic 03 — Developing Intelligent Multi-Agent Systems (25%)

Projects, goals and lineage · issue decomposition · assignment and hand-off · sequential specialists · verifying between disparate components

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Key concepts

- Projects hold goals and issues and supply the context an agent reads at runtime, so tasks carry full goal ancestry — the agent sees why, not only what.
- Goal lineage flows downward: company mission → project goal → parent issue → child issue. An agent that cannot trace its lineage is working without direction.
- Issues move through seven states — backlog, todo, in_progress, in_review, done, plus blocked and cancelled. done and cancelled are TERMINAL: issues do not move backwards.
- Decompose into a parent synthesis issue plus specialist child issues. Create every issue unassigned first — assignment is a heartbeat trigger, so an assigned issue wakes an agent immediately.
- Assignment is the trigger. Assign one child at a time to keep runs sequential, observable and individually attributable.
- An acceptance criterion must name the evidence: which artifact, at which path, containing which fields — never 'the research is done'.
- A hand-off is accepted only when the receiving agent can consume the artifact without asking the producer a question.
- Verification between disparate components means tracing one thread across issue ID, run log, comment, workspace diff and cost entry, and finding them consistent.
- Status is not evidence. An issue marked Done with no linked artifact and no diff is an unverified claim.
- Explicit limitations are part of a good deliverable — an agent that reports what it could not establish is more useful than one that quietly guesses.
- One event should not create many outcomes: choose a single trigger for the first controlled run and keep the work idempotent.
- Bound every wait. An unbounded wait for a heartbeat hides a failed adapter behind what looks like patience.
- The workspace diff is the strongest evidence you have — read what actually changed on disk before accepting any agent's summary of it.

Hands-on labs for this topic

- Lab 4 — Translate the Ledgerly Mission into Assigned Issues and Hand-Offs (Accounting) (Ledgerly Accounting (Accounting)). Full walkthrough: Part E, section E4. Lab folder: labs/lab-04-*.
- Lab 5 — Verify NorthPoint Routines and Layered Budget Controls (Stock Trading Research) (NorthPoint Trading Research (Stock Trading — research only)). Full walkthrough: Part E, section E5. Lab folder: labs/lab-05-*.

Note: Every lab data set is synthetic and fabricated for training. Never substitute real company, market, financial or candidate data, and never record a credential in your evidence notes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 04 — Deploying and Managing AI Agents (25%)

Routines and schedules · layered budgets · approvals and audit · operator intervention · deployment, portability and troubleshooting

Key concepts

- A routine is recurring work triggered by cron, webhook or API. Create it paused, prove it once by hand, and only then let the schedule own it.
- A schedule trigger is explicit — `30 16 * * *` with an IANA timezone such as Asia/Singapore. An implicit timezone is a production incident waiting to happen.
- Budgets have three scopes enforced in PARALLEL: company (monthly), agent (monthly) and project (lifetime). An agent can be paused by its own cap even when the company has budget left.
- Enforcement is a two-stage circuit breaker at exact thresholds: 80% utilisation records a warning and the agent keeps running; 100% pauses the agent automatically with pauseReason = budget. Enforcement happens at checkout, not at billing.
- Treat the budget as a two-stage circuit breaker — warn, then stop — and record the baseline spend and expected run frequency before you rely on it.
- Cost is tracked at token level by provider, model, agent and project, which is what makes the per-agent cap meaningful rather than notional.
- Approvals come in three types — CEO Strategy, Hire Agent and Budget Override — and you have three responses: approve, reject, or request revision. Approvals never expire on their own, so a silent company is often an unattended approval queue.
- The activity log is a durable record of mutations, heartbeats, cost events and approvals — the audit loop closes when the whole operation reconstructs from it.
- Use the least irreversible control first: inspect, pause, reassign, then terminate. Pause and heartbeat-off differ: heartbeat-off still wakes on assignment, comment, routine and on-demand; a paused agent wakes on nothing.
- Deployment modes are local_trusted (the default, no login), authenticated+private and authenticated+public, set with paperclipai configure --section server.
- A hosted deployment REQUIRES PAPERCLIP_AGENT_JWT_SECRET and DATABASE_URL; PAPERCLIP_TOOL_ACTION_SIGNING_SECRET is separately documented as required with no fallback for TOOL-ACTION approvals.
- Portability means an entire company exports and imports with secrets scrubbed and collisions handled — the org chart is data, not a hand-built configuration.
- Troubleshooting is a discipline: reproduce, isolate the layer, change one setting, re-verify with evidence, and record what the fix actually proved.

Hands-on labs for this topic

- Lab 6 — Run the Governed TalentBridge Operation and Close the Audit Loop (HR) (TalentBridge People Ops (HR — sourcing, interview, policy, onboarding)). Full walkthrough: Part E, section E6. Lab folder: labs/lab-06-*.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Every lab data set is synthetic and fabricated for training. Never substitute real company, market, financial or candidate data, and never record a credential in your evidence notes.

Part A — Foundations: Agentic AI and the Paperclip Control Plane

A1. From a language model to an agent

A large language model, on its own, answers. Give it a prompt and it returns text. It has no memory of yesterday, cannot act on the world, and cannot tell you whether the thing it just asserted is true. That is not a criticism — it is the shape of the tool. The moment you want the model to DO something rather than SAY something, you have to add machinery around it.

An agent is that machinery. It is a language model plus four additions, and it is worth being precise about each, because the whole of this course is about what happens when you have several of them at once.

The four additions that turn a model into an agent

Addition	What it provides	What breaks without it
Reasoning / planning	The model decomposes a goal into steps before acting, and re-plans when a step fails.	The system answers in one shot and cannot recover from a wrong first move.
Tool use	The model calls functions, CLIs and APIs to read live data and take real action.	The system can only talk about the work; it cannot do it.
Memory / state	Context carries across turns and — in Paperclip — across separate execution windows.	Every wake-up starts from nothing and repeats work already done.
Autonomy	The system decides which tool, which order, and when it is finished.	A human has to drive every step, which is just a chatbot with extra latency.

Add a fifth ingredient — several agents that divide the work — and you have a multi-agent system. That is where the interesting failures start, and it is where a control plane earns its place.

A2. Why coordination is the hard part

Running one agent is a prompting problem. Running five is an organisational problem, and the difficulties are not the ones people expect. They are rarely about model quality. They are about the things any organisation has to solve:

- DIRECTION — each agent needs to know not only what to do but why, or it optimises the wrong thing very efficiently.
- AUTHORITY — who may hire, who may spend, who may ship. Left implicit, it defaults to 'whatever the agent decided to try'.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- ISOLATION — two agents writing to the same place at the same time corrupts both pieces of work and destroys your ability to attribute either.
- COST — an agent in a retry loop does not get tired. Without a hard ceiling you discover the problem on the invoice.
- EVIDENCE — 'the agent says it is done' is not the same as 'it is done'. Someone has to be able to check, later, without taking anyone's word for it.

Note: These five are exactly the concerns Paperclip's twelve subsystems address. When you meet a subsystem later in this guide, map it back to the concern it solves — that is the fastest way to remember the architecture.

A3. What Paperclip is

Paperclip is an open-source control plane for AI agents: a Node.js server with a React user interface, backed by PostgreSQL, that gives a team of agents an organisational structure. The documentation's own definitions are worth learning verbatim, because the assessment uses this vocabulary:

Core vocabulary, as defined by the official documentation

Term	Official definition
Company	"A self-contained AI organisation with a single goal, a team of agents, a task board, and a budget."
Agent	"An AI employee — a configured role with a specific job, a manager, and a monthly budget."
Task (issue)	"A unit of work — like a ticket on a project board, except it's assigned to an AI agent, not a human."
Heartbeat	"A scheduled execution window — the moment when an agent wakes up, checks its inbox, and does work."
Adapter	"The connection between Paperclip and the AI system that actually runs your agent."
Budget	"A monthly spending cap — a dollar limit that Paperclip enforces automatically so your agents never overspend."
Approval	"A governance gate — a checkpoint where Paperclip pauses and asks you to sign off before an important action proceeds."

Notice what is absent from that list: nothing about prompting, nothing about model selection, nothing about how an agent reasons. That is deliberate.

A4. The boundary that matters most

If you remember one sentence from this course, make it this one:

Note: Paperclip manages the company. Adapters run the agents.

Paperclip does not perform the reasoning. It never sees a token of your model's thinking except as a cost figure and a run log. The actual work is done by a RUNTIME — Claude Code, Codex, Gemini CLI, Cursor, a bash process, an HTTP endpoint — reached through an adapter. The project's own phrasing is "if it can receive a heartbeat, it's hired."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

This boundary has a practical consequence you will meet in Lab 1. A healthy Paperclip API proves the CONTROL PLANE is up. It proves nothing whatsoever about whether any agent runtime is ready, because the runtime is a separate process that must be installed and authenticated OUTSIDE Paperclip. The only thing that proves a runtime is a marker run that puts a real artifact in the right workspace.

It also tells you what Paperclip is NOT, which saves a great deal of wasted evaluation time:

- NOT an agent framework — it does not dictate how your agent reasons, prompts or calls tools.
- NOT a chatbot — there is no conversational surface for you to talk to a model through.
- NOT a workflow builder — there is no drag-and-drop canvas of nodes and edges.
- NOT a prompt manager — prompts live with the runtime and the skill, not in a prompt library.
- NOT for a single agent — one agent does not need an org chart, and the coordination overhead would be pure cost.

A5. The twelve subsystems

The platform is conventionally described as twelve subsystems. You do not need to recite them, but you should be able to say which one owns a given problem, because that is how you know where to look when something misbehaves.

The twelve subsystems and the question each answers

Subsystem	The question it answers
Identity	Who is this — human or agent — and are they who they claim to be?
Org chart	Who reports to whom, and what is each role allowed to do?
Work	What needs doing, who owns it right now, and what state is it in?
Heartbeat	When does an agent wake, and what does it do when it does?
Workspaces	Where does this agent's work physically live, and who else can touch it?
Governance	Which actions require a human decision before they take effect?
Budget	What is the ceiling, and what happens when we reach it?
Routines	What recurring work should happen without anyone asking?
Plugins	How do we extend the platform without forking it?
Secrets	Where do credentials live, and how do they reach a run without leaking?
Activity	What actually happened, in order, durably?
Portability	How do we move a whole company somewhere else?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A6. Feasibility — when a multi-agent company earns its cost

This is the most commercially useful judgement in the course, and the one learners most often get wrong in the assessment. The instinct is to reach for a team. The correct instinct is to reach for ONE agent and be argued out of it.

The test is a single question: are the INPUTS, TOOLS, AUTHORITY and VERIFICATION the same across the whole task? If yes, use one agent. Splitting adds nothing but surface area.

The feasibility scan

Factor	Points to ONE agent	Points to a TEAM
Separability	One instruction still describes the job clearly.	The instruction has become a list of unrelated responsibilities.
Expertise	One skill set covers it.	Genuinely different expertise is required for different parts.
Independent evidence	The doer can reasonably check their own work.	Evidence must be produced INDEPENDENTLY of whoever will consume it.
Observability	Inputs and outputs are already observable.	Same — this is a prerequisite either way, not a reason to split.
Cost of coordination	Low volume, short task.	The work recurs often enough to amortise the setup.

Note: Complexity you do not need is still complexity you must debug, budget and audit. Every additional agent is another workspace, another instruction file, another budget line and another audit trail — forever.

A useful sanity check before you build anything: write the mission as one sentence. If you cannot, you do not yet understand the work well enough to delegate it — to an agent or to a person.

Check your understanding

1. In your own words, state the boundary between the Paperclip control plane and an agent runtime.
2. Give three reasons a healthy /api/health response does not prove an agent will run.
3. Name the four additions that turn a language model into an agent.
4. A colleague wants five agents for a task with one input file and one output report. What single question decides whether they are right?
5. Which subsystem owns each of: a runaway spend, a lost credential, an unattributable file change?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part B — Building the Company: Installation, Org Chart and Runtimes

B1. Installing the control plane

Paperclip requires Node.js 20 or later. This is the single most common cause of a failed onboarding, so check it first and record the output — you will want it in your evidence note when something later goes wrong.

```
node --version          # must print v20.x.x or higher
npm install -g corepack
corepack enable
corepack prepare pnpm@latest --activate
npx paperclipai onboard --yes
```

Onboarding downloads Paperclip, writes the local configuration, initialises an embedded PostgreSQL, starts the server and normally opens your browser. Two ports matter and they are easy to confuse:

Ports and paths you will need

What	Value
API and UI	http://localhost:3100 (default host 127.0.0.1)
Embedded PostgreSQL	port 54329
Health endpoint	GET /api/health
Master key size	32 bytes
Config, database, uploads, logs, secrets, workspaces	~/.paperclip/instances/
Managed CLI install	~/.paperclip/cli, entry point ~/.local/bin/paperclipai

Note: Run onboarding from a NORMAL user shell. Never prepend sudo and never use an elevated Administrator or root terminal — the embedded PostgreSQL refuses to start when the data directory is owned by a different user than the process.

B2. Proving the install before you build on it

Two checks, in this order. The diagnostic first, because it tells you what is wrong; the health endpoint second, because it tells you the server is answering for itself.

```
npx paperclipai doctor          # fix the EARLIEST reported error first
npx paperclipai doctor --repair # attempt automatic remediation
curl -sS http://localhost:3100/api/health   # expect an ok status
```

The discipline that matters here is the DIAGNOSIS LADDER. Faults are almost always at the lowest broken rung, and fixing something higher up while a lower rung is broken produces confusing, misleading results.

The diagnosis ladder — always work upward from rung 1

Rung	What must be true	How you prove it
1 · Host	Node 20+, a normal non-elevated user, a writable practice root	node --version, and check who owns the folder
2 · Control plane	doctor is clean and the server answers	npx paperclipai doctor, then GET /api/health
3 · Adapter	the runtime is installed AND	run the runtime's own CLI

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	authenticated outside Paperclip	directly, then Verify connection
4 · Agent	the agent produces a real artifact in the correct workspace	a marker run, then git status in that workspace

Note: Change ONE setting at a time and re-verify. If you change three things and the fault clears, you have not fixed it — you have lost the ability to say what was wrong, and it will come back.

B3. Deployment modes and exposure

Exposure is a deliberate decision, not a default you inherit. Paperclip's deployment modes are set with `paperclipai configure --section server` or the PAPERCLIP_DEPLOYMENT_MODE environment variable.

Deployment modes

Mode	Login	Use it when
local_trusted (default)	No login required	A single operator on their own machine. This is the mode used throughout this course.
authenticated + private	Login required, not publicly reachable	A small team on a trusted network or a private mesh.
authenticated + public	Login required, publicly reachable	A hosted deployment. Requires the most care: real auth, real secrets, real exposure.

`paperclipai env` marks exactly TWO variables as required: PAPERCLIP_AGENT_JWT_SECRET and DATABASE_URL. A third, PAPERCLIP_TOOL_ACTION_SIGNING_SECRET, is separately documented as required with no fallback for tool-action approvals.

Environment variables you cannot skip

Variable	Purpose
PAPERCLIP_AGENT_JWT_SECRET	Signs agent tokens. Required.
DATABASE_URL	Connection string for PostgreSQL. Required.
PAPERCLIP_TOOL_ACTION_SIGNING_SECRET	Signs TOOL-ACTION approvals. Documented as required, with no fallback.
PAPERCLIP_ALLOWED_HOSTNAMES	Hostnames the server will answer for. Add custom domains.
PAPERCLIP_TELEMETRY_DISABLED / DO_NOT_TRACK	Set to 1 to disable telemetry.

Note: There are roughly seventy environment variables across nine categories. You do not need to memorise them. You DO need to know which are required, and that PAPERCLIP_AGENT_JWT_SECRET also acts as the fallback for BETTER_AUTH_SECRET.

B4. The company and its mission

A company is the isolation boundary: one goal, one team of agents, one task board, one budget. Companies do not share agents, skills, issues or costs. One deployment can host many.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The mission is the single most important text you will write, because everything downstream inherits from it. A mission must be MEASURABLE, which means three properties:

What makes a mission verifiable

Property	Question it answers	Example
Target	How much?	8 short-form videos per month
Metric	Measured how?	average watch-through rate
Boundary	Within what limit?	at or above 5%

"Grow the channel" has none of these and cannot be verified by anyone, including the agent. "Publish 8 short-form videos per month at an average watch-through rate of at least 5%" has all three, and an agent can tell whether it is winning.

Note: Set the COMPANY budget cap BEFORE you hire anyone. A company that gains a runtime before it gains a ceiling can spend without limit, and budget enforcement happens at checkout — so the cap must exist before any agent is able to check out work.

B5. The org chart

Paperclip enforces a strict hierarchy, and the rule is absolute: every agent reports to exactly one manager, and the CEO is the only agent with no manager. There are no matrix reporting lines and no unmanaged agents.

The CEO is the first hire. It owns goal alignment, decomposition and final synthesis, and it may propose further hires — subject to approval, which you will meet in Part D.

The six fields that define every agent

Field	What it controls
Role	What this agent is for — the single transformation it owns.
Manager	Its reporting line. Exactly one, except for the CEO.
Adapter	Which runtime actually executes its work.
Instructions	Its contract: purpose, inputs, output schema, exclusions, escalation.
Heartbeat	When it wakes, and on which triggers.
Budget	Its individual monthly ceiling.

The documentation gives starting budget guidance by seniority, which is a useful sanity check on your own numbers:

Suggested starting budgets (from the official documentation)

Role	Typical starting budget
CEO	$30–50 per month
Manager (CTO, CMO, …)	$20–40 per month
Worker agent	$10–25 per month

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

B6. Role instructions are a contract, not a personality

The most common beginner error is writing an instruction that describes a character — "you are a meticulous financial analyst who loves detail" — instead of a contract. Tone does not constrain behaviour. Structure does.

The five sections every role instruction needs

Section	States	Why it matters
PURPOSE	The single transformation this role owns	Prevents scope drift into a neighbouring role's work.
ALLOWED INPUTS	Which files, data and sources may be read	Confines the blast radius when something goes wrong.
OUTPUT SCHEMA	The artifact, its path, and its required fields	Makes acceptance objective rather than a matter of opinion.
EXCLUSIONS	What this role must NEVER do	The most valuable section, and the one people leave out.
ESCALATION	When to stop and ask the board	Keeps human authority explicit instead of assumed.

Note: Test: if two of your roles could each plausibly claim the same task, your instructions are not finished. Ambiguity in the org chart shows up later as two agents doing the same work and neither being accountable for it.

B7. Adapters — connecting real runtimes

An adapter is the connection between Paperclip and the system that actually runs your agent. The documentation devotes about twenty pages to adapters; the type keys below are the eleven that appear in the adapters overview, and they matter because they appear in configuration and in the API:

Adapter type keys

Type key	Runtime
claude_local	Claude Code running locally
codex_local	OpenAI Codex CLI running locally
gemini_local	Gemini CLI running locally
opencode_local	OpenCode running locally
cursor	Cursor (local and cloud variants exist)
pi_local	Pi running locally
hermes_local	Hermes running locally
process	An arbitrary local process
http	An HTTP / webhook endpoint
grok_local	Grok running locally
openclaw_gateway	OpenClaw via its gateway

Note: process, http and openclaw_gateway are functional in the runtime but appear as "Coming soon" in the UI dropdown. If you need them today, configure them through the API rather than the picker.

Adapters carry their own defaults which materially affect cost and safety. The Claude Code adapter, for example, defaults to a maximum of 300 turns per run, has dangerouslySkipPermissions set to true, and a sandbox timeout of 14,400 seconds (four hours). Every one of those is a number you should look at before you let an agent run unattended.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Adapter readiness is proved by a MARKER RUN, never by a saved configuration. A marker run is a deliberately trivial task whose only purpose is to produce one named file in one named workspace. If the file appears in the right place, the boundary is real. If it appears somewhere else, the workspace path is misbound — correct it on the AGENT, delete the stray file, and run the marker again.

B8. Workspaces, secrets and skills

A workspace is where an agent's work physically lives. Each agent gets its own root, so that every change on disk is attributable to exactly one role. Workspaces have their own lifecycle, and a degraded or failed workspace is a common cause of an agent that appears to do nothing:

- provisioning — being created
- validating — being checked
- ready — usable
- degraded — usable but something is wrong
- repairing — being fixed
- failed — unusable
- archived — retired

Commit an empty Git baseline in each workspace before any agent runs. It costs one command and it is what makes every later diff readable:

```
git -C workspace/<role> init -q
git -C workspace/<role> commit -q --allow-empty -m 'baseline'
```

Secrets never belong in a prompt, a lab note or Git. They live in the runtime or in Paperclip's secret store and are injected at heartbeat time with a defined scope. Two operational limits are worth knowing: secret proposals are capped at 20 pending and 20 per minute, and they expire after 14 days.

Note: Board tokens (pcp_board_*) and agent keys are printed ONCE at creation and never shown again — including the key minted by `agent local-cli`. Capture it at the moment of creation or mint a new one.

A skill is a reusable instruction document an agent loads on demand. The point is context economy: an agent with ten skills does not load all ten on every heartbeat, only the relevant ones. That means fewer tokens per run, cleaner context, and one place to update a procedure that many agents share.

```
skills/
└── code-review/
    ├── SKILL.md
    └── references/
        └── examples.md
```

SKILL.md carries YAML frontmatter with name, description, slug, key and metadata. The description should be written as DECISION LOGIC — when to use this skill and when not to — because that is what the agent reads to decide whether to load it:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
---
name: code-review
description: >
  Use when asked to review a pull request or code diff.
  Don't use when writing new code from scratch.
---
```

- Skills are COMPANY-SCOPED and are never shared across companies.
- Folders may nest up to four levels deep.
- Skills are attached at hire time via desiredSkills, or later via POST /api/agents/{agentId}/skills/sync with mode add, remove or replace.
- paperclip_bundled skills: the behaviour CHANGED between documentation surfaces — see the note below before relying on either description.

Note: A known contradiction in the official documentation. The Skills API reference states that paperclip_bundled skills are force-unioned into every agent, so removing one has no effect. The newer CLI Skills page states that the server NO LONGER force-attaches them, and that they remain in the company library as read-only entries. Treat the CLI page as the current behaviour, verify on your own install with the skills commands in Part G, and do not design a control that depends on either behaviour persisting.

Check your understanding

1. Why must the company budget cap be set before the first hire?
2. State the three properties of a measurable mission, and rewrite 'improve our reporting' so it has all three.
3. Name the five sections of a role instruction and say which one learners most often omit.
4. An agent is configured, the adapter shows connected, but nothing appears in its workspace. Work the diagnosis ladder aloud from rung 1.
5. Why is a marker run stronger evidence than a green 'connection verified' badge?

B9. Anatomy of a heartbeat run

It is worth knowing what actually happens when an agent wakes, because almost every diagnostic question resolves to "which of these six steps failed?"

The six-step execution arc

Step	What happens	Fails when
1. Trigger	A schedule tick, an assignment, an @-mention, a manual invoke or an approval resolution wakes the agent.	Nothing is assigned and the interval is off — the usual reason an agent 'does nothing'.
2. Adapter invocation	Paperclip calls the configured adapter.	The adapter is misconfigured or the runtime is not authenticated.
3. Agent process spawn	The runtime process starts.	The binary is missing, or the sandbox cannot start.
4. Paperclip API calls	The agent reads its work and writes results back.	Checkout conflicts (409), or the cross-issue write limit is hit.
5. Result capture	Output, artifacts and token usage are captured.	The process is lost mid-run — recorded as the error kind process_lost.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

6. Run record storage	The run, its cost and its activity entries are persisted.	Rarely fails; if it does, the audit trail has a gap you must explain.

Agents keep conversation context across heartbeats through SESSION PERSISTENCE. Each run records a session ID before and a session ID after. If the pair differs the session rotated during the run; if it matches, the agent continued the same conversation. When an agent seems to have forgotten prior context, that pair is the first thing to look at.

B10. Agent states and what each one means

The seven agent states

State	Meaning
active	Enabled and ready for heartbeats.
idle	Active, but no heartbeat is currently running.
running	A heartbeat is in progress right now.
error	The last heartbeat failed. Not working, and did not stop gracefully.
paused	Deliberately stopped — by you, or automatically on hitting the budget limit.
terminated	Permanently deactivated. Not reversible.
pending_approval	Hired but awaiting board approval before it may run.

Runs have their own statuses — queued, running, succeeded, failed, timed out and cancelled — plus the error kind process_lost, which is recoverable via Resume rather than requiring a fresh run.

Note: Pause is reversible; Terminate is not. The agent overflow menu offers both, and the documentation is explicit that Terminate is "permanent; use Pause for reversible pause". Read that twice before clicking.

B11. Structuring an agent's instructions

Paperclip can either own the instruction files itself (a MANAGED bundle) or point at an existing folder you control (an EXTERNAL bundle, via rootPath). Either way, the documentation recommends a four-file structure, and it is a good structure because each file answers a different question:

The recommended instruction files

File	Answers
AGENTS.md	The operating manual — the entry file, and the default.
SOUL.md	Persona and decision philosophy — how this role weighs trade-offs.
HEARTBEAT.md	The per-heartbeat checklist — what to do on every single wake.
TOOLS.md	Available tools and API notes — what this role may call.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The split matters for cost as well as clarity. Material that belongs in every run (HEARTBEAT.md) is separated from material that is only occasionally relevant, which is the same economy that makes skills worthwhile.

B12. Reading the dashboard

The dashboard is the page you will return to most. The documentation likens it to cockpit instruments, and the analogy is apt: you are not reading it for detail, you are reading it to decide whether to look closer.

- Agents Enabled — how many agents can work, and how many are running, paused or in error.
- Tasks In Progress — how many issues are being worked, plus open and blocked counts.
- Month Spend — spend this month against the company budget.
- Pending Approvals — how many decisions are waiting on you.

Reading the dashboard at a glance

Signal	Healthy	Needs attention
Agents	Mostly active or running	Any in error, or unexpectedly paused
Tasks in progress	Work is moving	Nothing in progress for hours, or many blocked
Month spend	Climbing predictably	A sudden jump, or active budget incidents
Approvals	Queue manageable	Pending approvals piling up
Recent activity	Steady, understandable motion	Long silence, or repeated failures

Note: The healthy pattern for a small company is ONE agent running while the others stay active. Several agents running at once is not automatically wrong, but it does mean you can no longer attribute a workspace change to a single run by timing alone. The activity feed is a pulse check, not an investigation tool.

Part C — Running the Operation: Work, Heartbeats and Verification

C1. Projects, goals and lineage

A project holds goals and issues and supplies the context an agent reads at runtime. The reason this matters is LINEAGE: work carries its full ancestry, so an agent can see why a task exists and not merely what it says.

```
company mission
  └── project goal
        └── parent issue  (synthesis)
              ├── child issue  (specialist A)
              └── child issue  (specialist B)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Projects have their own lifecycle — backlog, planned, in_progress, completed, cancelled — and can carry a LIFETIME budget cap, distinct from the monthly caps applied to companies and agents.

Note: An agent that cannot trace its lineage is working without direction. If you find yourself re-explaining the goal in every issue body, your lineage is broken and you are paying for the same context repeatedly.

C2. Issues — the seven states

Work in Paperclip is an issue, and issues move through a defined state machine. Learn these seven; the assessment expects them by name.

Issue states, with the documentation's own definitions

State	Definition
backlog	"The issue exists and has been identified, but no one is working on it yet."
todo	"The issue is ready to start. An agent has been assigned and is waiting to check it out."
in_progress	"An agent has checked out the issue and is actively working on it. Only one agent can hold an issue in this state at a time."
in_review	"The agent has completed the work and moved the issue to review. It's waiting for sign-off."
done	Terminal. The work is complete.
blocked	"The agent can't move forward. Something is preventing progress."
cancelled	"The issue is no longer needed and won't be completed." Also terminal.

Note: Issues do not move backwards. done and cancelled are TERMINAL states — a terminal issue can only be revived through an explicit reopen, never by editing its status.

The normal path is backlog → todo → in_progress → in_review → done, with blocked and cancelled available as departures from it.

C3. Checkout — the rule that prevents double work

When an agent picks up a task it CHECKS IT OUT, claiming exclusive ownership. Only one agent can hold an issue in in_progress at a time. This is enforced by the platform, not by convention, and there are hard rules around it that trip up almost everyone integrating through the API:

Checkout discipline — common mistakes

Mistake	What to do instead
PATCHing status directly to in_progress	Never do this. Use the checkout mechanism; a direct status write bypasses the lock.
Retrying after a 409 Conflict	Treat 409 as TERMINAL. Another agent holds the issue; retrying will not win the race.
Expecting a cancelled blocker to unblock its dependents	Cancelled blockers never auto-unblock dependents. Resolve the dependency explicitly.
Editing a done issue's status to reopen it	Terminal issues reopen only via an explicit reopen: true.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Runs identify themselves with an X-Paperclip-Run-Id header, which is how a write is attributed to the execution that made it. There is also a hard limit worth knowing: CROSS_ISSUE_INFLUENCE_LIMIT is 20 — a single heartbeat run may write to at most twenty other issues, which exists to stop one confused agent rewriting the whole board.

C4. Heartbeats — the execution model

A heartbeat is a scheduled execution window: the agent wakes, checks its inbox, does work, and goes dormant again. Between heartbeats the agent is not idling and not billing — it does not exist as a running process.

There are FIVE triggers. Most people can name three and are surprised by the other two:

The five heartbeat triggers

Trigger	Fires when
Schedule / interval	The configured interval ticks — whether or not there is work.
Task assignment	An issue is assigned to the agent. THIS is what makes assignment the real trigger for work.
Comment mention	Someone @-mentions the agent in a comment.
Manual invoke	A human presses Run Heartbeat.
Approval resolution	An approval the agent was waiting on is decided.

Note: "Heartbeat on interval" is OFF BY DEFAULT, and the recommended combination is interval OFF with wake-on-demand ON. The interval is described as "a floor, not a guarantee" — a busy agent may run more often because events trigger wakes.

This default is a good one and you should think hard before changing it. An interval means the agent wakes whether or not there is anything to do, and each wake costs money. Event-driven waking costs nothing when nothing is happening.

There is also an important distinction between two things that look similar:

Paused versus heartbeat-off

	Wakes on interval	Wakes on assignment / comment / routine / on-demand
Heartbeat off	No	YES
Paused	No	No — paused wakes on nothing

C5. Decomposition and assignment

The pattern you will use in every lab: create a PARENT synthesis issue and its specialist CHILD issues, all UNASSIGNED, then assign one child at a time.

Creating issues unassigned is a control decision, not a stylistic one. Assignment is a heartbeat trigger — so an issue created with an assignee wakes an agent immediately,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

before you have finished writing the acceptance criteria and before you have checked the caps. Create everything cold, then start the work deliberately.

Assign ONE child at a time. Sequential runs are individually attributable; overlapping runs are not. If two specialists ran simultaneously you cannot demonstrate that the second consumed the first's output, which means you cannot prove the hand-off happened.

C6. Acceptance criteria that name evidence

An acceptance criterion must name the EVIDENCE, not the activity. This is the single highest-leverage writing skill in the course.

Activity versus evidence

Fails — describes activity	Works — names evidence
"The research is done."	"workspace/research-analyst/research.md exists and contains claim, source, date accessed and confidence for every claim."
"Reconcile the control accounts."	"reconciliation.md lists every control account with ledger balance, statement balance and variance."
"Review for compliance."	"exceptions.md lists each variance above SGD 500 with a stated cause and a recommended treatment."

The test for a good criterion: could a stranger, with no knowledge of the conversation, look at the workspace and say yes or no? If judging it requires asking the agent what it meant, the criterion is not finished.

C7. Verification — one thread, five places

Verifying that work between disparate components actually functioned means tracing ONE thread across five places and finding them consistent.

The evidence chain

Place	What it tells you
Issue ID	What was asked, and under which lineage.
Run log	What actually executed, and whether it errored.
Comment	What was accepted, by whom, on what basis.
Workspace diff	What changed on disk. The strongest evidence you have.
Cost entry	What it spent — and whether that is plausible for the work claimed.

Note: STATUS IS NOT EVIDENCE. An issue marked done with no linked artifact and no diff is an unverified claim, however confident the agent's summary sounds.

Read the diff before accepting any summary. An agent describing its own work is a witness with an interest in the verdict; the diff is the record.

A hand-off is accepted only when the receiving agent can consume the artifact WITHOUT asking the producer a question. If the receiver needs clarification, the output

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

schema in the producer's instruction was under-specified — fix the contract, do not patch the artifact by hand.

Finally, value explicit limitations. An agent that reports what it could NOT establish is more useful than one that quietly guesses, and a deliverable with a stated limitations section is a sign of a well-written role, not a weak one.

C8. Execution policy and review rounds

Paperclip can enforce a review workflow on an issue's execution. The parameters are small in number but consequential:

Execution policy parameters

Parameter	Value
maxReviewRounds	Default 3. Range 1–50. A null value means the server default of 3.
approvalsNeeded	Fixed at 1.
changesRequestedCount	Counts review rounds; escalates when the cap is reached.
Comment-required backstop	retry_queued → retry_exhausted

Once a reviewer has been escalated to, only that reviewer may advance the stage. Attempting otherwise returns 422 Unprocessable Entity with the message "Only the escalated reviewer can advance the current execution stage". If you see that error, you are not blocked by a bug — you are blocked by the policy working.

Check your understanding

1. List the seven issue states and identify the two terminal ones.
2. Why does creating issues unassigned change the risk profile of a company?
3. Name the five heartbeat triggers.
4. You receive a 409 Conflict when checking out an issue. What does it mean and what do you do?
5. Rewrite 'the analysis is complete' as an acceptance criterion that names evidence.
6. Two child issues show overlapping run timestamps. What can you no longer prove?

Part D — Governing the Operation: Budgets, Approvals, Routines and Audit

D1. Budgets — three scopes, enforced together

This is the section most often misunderstood, and the misunderstanding is expensive. Paperclip has THREE budget scopes and they are enforced in PARALLEL, not nested.

The three budget scopes

Scope	Period	Applies to

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Company budget	Monthly	The total ceiling for the entire company.
Per-agent budget	Monthly	One individual agent's ceiling.
Project budget	Lifetime	An execution-bound project, for its whole life.

Note: The scopes are independent. The documentation is explicit: "an agent can be paused by its own budget limit even if the company has remaining budget." Do not picture nested rings where the outer one always binds first — picture three separate trip switches, any of which can fire.

Enforcement is a two-stage circuit breaker with exact thresholds:

Budget enforcement thresholds

Utilisation	What happens
80%	A warning is recorded so you can intervene. The agent CONTINUES running.
100%	The agent is automatically PAUSED. No further heartbeats fire. pauseReason = "budget".

The utilisation bar is colour-coded so you can read a company's health at a glance: green below 70%, yellow from 70 to 90%, red above 90%.

Enforcement happens AT CHECKOUT — before the work runs, not when the invoice arrives. That is what makes unattended operation safe: a runaway retry loop stops at the ceiling instead of running all night.

Note: A cap you would be unhappy to actually pay is set too high. Set it at the number you are genuinely willing to lose to a bug, because one day you will.

Costs are tracked at token level by provider, model, agent and project, and the Costs page separates INFERENCE spend (per-request model cost) from FINANCE events (subscriptions, invoices, credits, refunds, adjustments) — do not read one and think you have seen the other.

D2. Approvals — where human authority is made explicit

An approval is a governance gate: Paperclip pauses and asks a human to sign off before an important action proceeds. There are three types you will meet:

Approval types

Type	Raised when
CEO Strategy	The CEO produces a strategic plan after its first heartbeat. "The CEO cannot move tasks to 'in progress' until you approve its strategy."
Hire Agent	An agent (usually the CEO or a manager) proposes a subordinate. The payload includes name, role, capabilities, adapter configuration, monthly budget and reporting line.
Budget Override	A budget policy hits a hard stop and someone asks to proceed anyway.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

You have three possible responses, and the third is the one people forget exists:

- APPROVE — the action proceeds. For a hire, the agent is created, configured and queued to wake automatically.
- REJECT — the action is denied. The requesting agent is notified and will not retry unless instructed.
- REQUEST REVISION — you are not approving as-is, but you are not saying no. The agent revises and resubmits.

The state machine reflects that third option:

```
pending ──→ approved
        ├─→ rejected
        └─→ revision_requested ──→ resubmitted ──→ pending
```

Note: Approvals do NOT expire on their own. An agent waiting on a pending approval pauses at that point indefinitely. A silent, stalled company is very often an unattended approval queue — check it first.

The board also holds override powers that bypass the approval flow entirely: pause any agent, and resume any paused agent. These are blunt instruments and belong in the intervention ladder, not in routine operation.

Whatever you decide, RECORD THE RATIONALE. One sentence explaining why is the difference between a decision and an unaudited event. Months later, the rationale is the only thing that will tell you — or a regulator — what you were thinking.

D3. Routines — automation earns the schedule

A routine is recurring work triggered by cron, webhook or API. The rule for introducing one is a discipline, and it exists because a routine that has never succeeded is simply scheduled spend nobody has reviewed.

- 1. Create the routine PAUSED.
- 2. Add the schedule trigger EXPLICITLY — a cron expression AND an IANA timezone.
- 3. Name the artifact the routine must produce, or the run is successful but unverifiable.
- 4. Run it ONCE manually, with a bounded wait, and read the generated issue, the run and the cost.
- 5. Leave it paused until you have done step 4. Only then let the schedule own it.

The timezone is not a formality. `30 16 * * *` without a timezone runs at 16:30 in whatever zone the host happens to be in, which is fine until the host moves, the clock changes, or someone deploys to a different region. Write Asia/Singapore and the ambiguity disappears.

Note: The routine editor snaps minutes to FIVE-MINUTE increments. Concurrency defaults are "coalesce if active, skip missed" — a routine already running will not start a second copy, and a missed window is skipped rather than backfilled.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Routine states are paused, draft, archived, succeeded, failed and running. A routine left in running when nothing is happening usually means a lost process — check the run log for a process_lost error kind.

D4. The activity log and closing the audit loop

Activity is the durable record of mutations, heartbeats, cost events and approvals. It is the subsystem that makes everything else defensible.

The audit loop CLOSES when the entire operation can be reconstructed from the log alone — company creation, hires, issues, runs, costs, approvals and interventions, in order, with no unexplained gap.

Build a mission-to-evidence audit map. One row per mission claim, linking the issue, the run, the artifact, the cost entry and the approval that evidences it. This is, concretely, the artifact a regulator or an auditor would ask you for.

```
| Mission claim | Issue | Run | Artifact | Cost | Approval |
|---------------|-------|-----|----------|------|----------|
```

Note: An unexplained gap in the activity log usually means work was done OUTSIDE a run — typically hand-edited in the workspace. Record it honestly. The gap is itself an audit finding, and concealing it is far worse than having it.

D5. The operator intervention ladder

When an agent misbehaves, use the LEAST IRREVERSIBLE control that resolves the situation. Escalate only when the rung below has genuinely failed.

The intervention ladder

Rung	Action	Reversible?	Use when
1	INSPECT — read task, run, cost and activity evidence	Yes, nothing changes	Always. Never act before you have looked.
2	PAUSE — stop new heartbeats, keep all state	Yes	The agent is doing something wrong and you need it to stop now.
3	REASSIGN — move the work to another role	Yes	The work is valid but this agent is the wrong owner.
4	TERMINATE — retire the agent	NO	The role should not exist. Last resort.

Note: Terminate is not reversible. Practise it only on a role you deliberately created as disposable, and never as a first response to a misbehaving production agent.

D6. Roles and permissions

Human members of a company hold roles, and roles imply permission grants. There are four roles plus an unset state, and twenty-one permission keys.

Company roles and their implicit grants

Role	Description	Implicit grants
Owner	"The people who run the	agents:create, agents:configure,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	company"	skills:create, environments:manage, users:invite, users:manage_permissions, tasks:assign, joins:approve
Admin	"Trusted operators who onboard people and agents"	As Owner, MINUS users:manage_permissions
Operator	"Hands-on members who help run the work"	tasks:assign
Viewer	"Read-only observers"	None
Unset	No implicit grants	Hand-picked explicit grants only

Note: Two rules with real consequences: an explicit grant PERSISTS across a role change, and there is NO DENY LAYER. You cannot subtract a permission from a role — you can only choose a role that does not include it, or leave the role unset and grant explicitly.

D7. Portability — moving a company

A whole company exports and imports, with secrets scrubbed and identifier collisions re-mapped. This is what makes a Paperclip organisation data rather than a hand-built configuration you are afraid to touch.

Practical uses: backing up before a risky change, cloning a proven structure for a second business line, moving from a laptop to a hosted deployment, and handing a working company to a colleague.

Check your understanding

1. State the three budget scopes and their periods. Are they nested or parallel?
2. At what utilisation does Paperclip warn, and at what utilisation does it pause the agent?
3. Name the three approval types and the three possible responses.
4. Your company has gone quiet — no runs, no errors, no spend. What do you check first, and why?
5. Put the intervention ladder in order and say which rung is irreversible.
6. What does an unexplained gap in the activity log usually mean, and what should you do about it?

Part E — Lab Walkthroughs

Parts A to D told you how Paperclip works. Part E is where you make it work. Six labs run across the two days, each one simulating a different one-person company in a different sector, and each one adding exactly one layer of control to the layer you proved in the lab before. By the end of Lab 6 you will have built, governed, intervened in and audited a running agent company — and you will have the evidence files to prove each step, which is the part an assessor, a client or a regulator actually asks for.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The labs are deliberately cumulative. Lab 1 proves the platform. Lab 2 creates a company, a mission and a CEO. Lab 3 turns one agent into an org chart. Lab 4 turns a mission into assigned work. Lab 5 puts that work on a schedule and tests every budget layer against it. Lab 6 governs the whole thing and closes the audit loop. Skipping a lab does not save time; it removes the foundation the next lab stands on.

Note: EVERY COMPANY, FIGURE AND PERSON IN THESE LABS IS SYNTHETIC. ClipReel Studio, BrightFunnel Marketing, Meridian Finance Desk, Ledgerly Accounting, NorthPoint Trading Research and TalentBridge People Ops are fictional businesses invented for this course. The ledgers, leads, candidates, symbols and price histories in labs/lab-0N-*/data/ are fabricated. None of it is real financial data, real market data or real applicant data, and none of it should be quoted outside the classroom.

How to read a lab walkthrough

Every one of the six sections that follow has the same eight parts, in the same order, so that you can navigate any of them the moment you know your way around the first.

The structure of every lab section in Part E

Section	What it gives you
Why this lab exists	The real business problem the lab teaches, in the company's own commercial context. Read this before touching a keyboard.
The company	The simulated business, its mission, its roles and the exact synthetic data files it works from.
Before you begin	What must already be true — from earlier labs and from your own machine — before the first command will succeed.
Walkthrough	Every step restated, then explained: why it comes in that position, what breaks if you skip it, and the exact command or UI path.
What good looks like	The verification criterion expanded into what you should actually be able to see on screen and in your evidence file.
Common failures	Each documented symptom expanded into cause, fix and prevention.
Going further	The stretch challenge, with hints for learners who finish early.
Think about it	The reflection question, discussed, with a model answer.

Two habits will save you more time across the two days than any other advice in this guide. First, keep an evidence file open from the start of every lab and write into it as you go — reconstructing evidence afterwards is slower and less honest than recording it live. Second, when something fails, read the EARLIEST error in the output, not the last one. Later errors are usually consequences.

Note: Never record a credential. Your evidence files should contain versions, statuses, run IDs, issue IDs, paths, costs and decisions. They should never contain an API key, a token, a connection string, a cookie or the contents of an environment dump. This is not merely a classroom rule — an evidence pack that leaks a provider key is a security incident.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

E1. Lab 1 — Install Paperclip and Prove the Control Plane (ClipReel Studio)

TOPIC 1 · Fundamentals of Agentic AI and Paperclip · approximately 45 minutes · maps to LO1: explain the control-plane / runtime boundary and verify a healthy local Paperclip installation with observable evidence.

Why this lab exists

Almost every difficult failure you will meet in the rest of this course looks the same from the outside. An agent does not wake. An artifact does not appear. A run shows a cost of zero. In each case there are at least four candidate causes — the platform, the adapter runtime, the configuration and the work itself — and if you have never established which of those was healthy and when, you will debug all four simultaneously and fix none of them. Lab 1 exists to remove the first candidate permanently, by producing a dated, recorded, clean baseline of the control plane before a single agent exists to blame.

The commercial version of that argument is sharper. ClipReel Studio is a one-person short-video production company: the founder writes, films and edits vertical video for small local brands and publishes on their own channels as a showcase. The bottleneck is not filming — it is the research-and-scripting front half of the pipeline. That founder has perhaps two hours a week to spend on tooling. If they adopt an agent platform and then lose an evening to a failed installation they cannot characterise, they will abandon the platform, and they will be right to. A verified baseline is what converts "it doesn't work" into "the control plane is fine, so the problem is downstream", which is a five-minute conversation instead of an evening.

There is a conceptual payload here too, and it is the single most important idea in the course. Paperclip manages the company; adapters run the agents. Paperclip is a Node.js server with a React interface and a PostgreSQL database. It never performs the reasoning and never sees a token of the model's thinking except as a cost figure and a run log. So a healthy Paperclip API tells you the control plane is up and says absolutely nothing about whether Claude Code, Codex, Gemini CLI or Cursor is installed, authenticated and reachable. Learners who do not internalise that boundary in Lab 1 spend Lab 2 trying to fix an adapter problem in the platform's settings screens.

The company

ClipReel Studio is a fictional one-person short-video production company operating from a home studio in Singapore. Its simulated mission is stated in three parts, because a mission that cannot be split this way cannot be verified afterwards:

ClipReel Studio's synthetic mission, decomposed

Property	Value
Target	8 short-form videos per month
Metric	Average watch-through rate
Boundary	At least 5%, blended across all three channels

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The company publishes to three simulated channels — TikTok (4 videos per month, target watch-through 5.5%), Instagram Reels (2 per month, 5.0%) and YouTube Shorts (2 per month, 4.5%) — for a blended target of at least 5.0%. The simulated baseline for the quarter shows the shape of the problem: June published 6 videos at 4.1%, July 7 at 4.6%, August 8 at 4.9%. Volume has been solved; the rate has not. A small, persistent gap of that kind is exactly the sort of problem worth putting a repeatable process against rather than a one-off push.

The simulated operating budget allocates SGD 220.00 per month, of which SGD 40.00 is the ceiling for AI agent runtime and tooling. That SGD 40.00 figure is the one that matters for this course: it is the outermost budget ring, and the brief is explicit that it is set before any agent is hired.

ClipReel's agent company has three roles, which you do NOT create in Lab 1 — Lab 1 proves the control plane only — but which shape everything that follows:

- HEAD OF CONTENT (CEO) — owns the mission, decomposes it into work, and synthesises the final publishing plan. Reports to nobody. Never films, edits or publishes directly.
- TREND RESEARCHER — identifies emerging formats, sounds and hooks, and reports each with a stated confidence and a stated limitation. Reports to the Head of Content. Never writes scripts.
- SCRIPT WRITER — turns an accepted trend finding into a shootable script with a hook, a body and a call to action, sized to the target channel. Reports to the Head of Content. Never sources its own trends; it consumes the researcher's accepted artifact.

The lab's only data file is labs/lab-01-clipreel-studio/data/clipreel-company-brief.md, which contains the mission, the channel table, the quarterly baseline, the budget and the three role definitions. You copy it into your practice root in step 8 so that later labs have a realistic synthetic mission to work from.

Before you begin

- A NORMAL, NON-ADMINISTRATIVE SHELL. Close any elevated Administrator or root terminal before you start. The documentation is explicit: do not run onboarding with sudo or from a root shell, because the default setup starts embedded PostgreSQL and PostgreSQL refuses to run as an administrative user.
- Node.js 20 or later on the PATH, with npm/npx available, and access to the public npm registry at https://registry.npmjs.org.
- Permission to create a local practice folder and to download an npm package.
- A modern browser able to reach http://localhost:3100 on the same machine.
- Nothing from an earlier lab — this is the first lab, and everything that follows depends on it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Two ports matter and they are easy to confuse. The Paperclip API and user interface listen on 3100. The embedded PostgreSQL listens on 54329. If you are checking whether something is running, check the right one.

Walkthrough

Step 1 — Create the ClipReel practice root

Create the practice root with separate workspace and evidence folders, and keep this root for the whole course.

The folder layout is not housekeeping; it is the first control you set. The three workspace subfolders correspond to the three ClipReel roles, and separate roots are what will later make a file change attributable to exactly one agent. The separate evidence folder keeps your record of what happened outside any workspace an agent can write to — which is precisely what you want if you are ever asked to show that the evidence was not produced by the thing being evidenced. Skip this step and you will find yourself in Lab 3 with three agents writing into one directory and no way to tell whose marker file is whose.

```
mkdir -p paperclip-tgs/clipreel/workspace/{head-of-content,trend-researcher,script-writer} \
         paperclip-tgs/clipreel/evidence
cd paperclip-tgs/clipreel
```

Step 2 — Verify the runtime

Node.js must report major version 20 or later. Record both lines in your evidence note — a version mismatch is the single most common cause of a failed onboard.

This check comes second, before anything is downloaded, because an unsupported Node version produces a failure that surfaces much later and much less clearly. Paperclip requires Node.js 20 or newer, and the CLI's own doctor command checks it explicitly. If you skip the check and onboarding fails halfway, you will be reading a stack trace from a partially initialised database instead of a one-line version mismatch. Record both the Node and npm versions: the npm version tells you which registry behaviour and lockfile format you are dealing with if step 3 misbehaves.

```
node --version
npm --version
```

Step 3 — Onboard Paperclip

Run the official onboarding command from a NORMAL user shell. The first run downloads Paperclip, writes the configuration, initialises the embedded PostgreSQL, starts the server and opens the browser. Do not prepend sudo.

Onboarding is a single command that does five things at once, which is why it comes after the version check and before every other step. It writes the instance configuration to ~/.paperclip/instances/default/config.json, provisions the agent JWT secret, creates the local secrets master key, brings up the embedded database and starts the server.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The explicit registry flag exists because a machine configured against a private or corporate npm registry will otherwise resolve the package somewhere unexpected and fail in a way that looks like a Paperclip problem.

The sudo prohibition is worth stating twice. The embedded PostgreSQL will refuse to start under an administrative user, and the failure message points at the database rather than at your shell, so learners routinely spend ten minutes debugging PostgreSQL when the actual fix is to close the terminal and open a normal one.

```
npx --registry https://registry.npmjs.org paperclipai onboard --yes
```

Note: Back up ~/.paperclip/instances/default/secrets/master.key somewhere safe. It is a 32-byte key and it encrypts every API key and secret stored in Paperclip. Lose it and you lose access to all of them.

Step 4 — Keep the server running

If onboarding is still attached to the terminal, leave it running and open a second terminal in the same folder. If you stopped it, start it again.

This step exists because of a specific and very common learner error: pressing Ctrl-C to "get the prompt back", which stops the server, and then concluding from the resulting connection refused that the installation failed. The server is a long-running process. Give it its own terminal and do the rest of the lab in a second one. The run command is idempotent in the sense that matters here — if a valid configuration already exists it reuses it rather than re-onboarding.

```
npx paperclipai run
```

Note: If the instance is already active under a background service manager, run will refuse to start rather than launch a second server. Two servers sharing one port and one database is not a state you want, and the refusal is the tool protecting you from it.

Step 5 — Run the built-in diagnostic

Run doctor and fix the EARLIEST reported dependency or configuration error before continuing — never change several settings at once.

Doctor is the platform's self-assessment, and running it before you touch the browser is deliberate. It checks configuration validity, deployment and auth mode compatibility, the agent JWT secret, the secrets adapter, storage, database connectivity, the LLM provider, the log directory, the listen port, the Node.js runtime and the managed install and background service. Each of those is a prerequisite for something you will do later, so a doctor failure is a preview of a lab that is about to fail.

The discipline of fixing only the earliest error matters more than it sounds. These checks are ordered, and later failures are frequently consequences of earlier ones — a database check fails because the configuration check failed, which failed because the data directory was not writable. Change one thing, re-run, re-read. Changing three

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

settings at once and getting a pass tells you nothing about which change mattered, and you will need to know that the next time it breaks.

```
npx paperclipai doctor
```

Step 6 — Prove the health endpoint

Query the health endpoint and confirm it reports an ok status. This is the control plane answering for itself.

This is the heart of the lab. Doctor is the CLI's opinion of the installation; /api/health is the running server's own answer, over the same HTTP surface every agent and every integration will later use. The documentation describes it as the fastest way to check whether the server is up and whether the database is reachable, and it returns runtime metadata including deployment mode, auth readiness and feature flags. Proving it now means that for the rest of the course you can distinguish "the platform is down" from "the platform is fine and something downstream is wrong" with a single command.

Note what this does NOT prove. It says nothing about whether any agent runtime is installed or authenticated, because runtimes are separate processes reached through adapters and living entirely outside the control plane. Hold that thought — it is the reflection question at the end of this lab.

```
# macOS / Linux
curl -sS http://localhost:3100/api/health

# Windows PowerShell
Invoke-RestMethod -Uri 'http://localhost:3100/api/health' | ConvertTo-Json -Depth 8
```

Step 7 — Load the dashboard

Open the dashboard in a browser and confirm the Paperclip start or onboarding screen loads without a connection error. Do NOT create the company yet — that is Lab 2.

The browser check is a third, independent confirmation, and it exercises a different path from the two before it: the static user interface and the browser's own network stack. A machine where curl succeeds and the browser fails is telling you something specific about proxies or host resolution rather than about Paperclip. The instruction not to create a company yet is not fussiness — Lab 2 asks for an exact company name and an exact mission string so that your evidence matches the marking guide, and a company created early with the wrong name has to be removed.

```
http://localhost:3100
```

Step 8 — Copy the ClipReel brief into the practice root

Copy the mock brief so later labs have a realistic, synthetic mission to work from.

The brief is copied at the end rather than the beginning for a reason the brief itself states: the mission comes first, the control plane is proved second, and only then is anybody hired. Having the brief in the practice root also means the mission, the channel

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

targets and the three role definitions travel with the instance rather than living in a course folder you may not have open in a later lab.

```
cp labs/lab-01-clipreel-studio/data/clipreel-company-brief.md ./
```

Step 9 — Write the evidence note

Record the date, Node and npm versions, the Paperclip version if shown, the doctor summary, the health status and the dashboard URL. Do NOT copy environment values, connection strings, cookies, tokens or provider keys.

The evidence note is the deliverable, not a courtesy. It is what converts a successful installation into a durable claim you can make tomorrow, and it is what an assessor marks. Write it while the outputs are still on screen; reconstructing versions from memory is how wrong numbers enter a record.

The redaction rule is absolute and it is a habit worth forming here, in the lab where it costs nothing, rather than in Lab 6 where an evidence pack is a governance artifact. Paperclip prints board tokens (prefix pcp_board_) and agent API keys once at creation and never again, precisely because they are secrets; an evidence file that contains one has turned a training record into a credential store.

```
evidence/lab01-installation.md
```

What good looks like

You have passed Lab 1 when four things are simultaneously true, and you can show each of them:

- npx paperclipai doctor completes with NO unresolved blocking error. Warnings you have read and understood are acceptable; an unread blocking error is not.
- GET http://localhost:3100/api/health returns a status of ok, from the same machine, over HTTP, with the server running in another terminal.
- The browser loads the Paperclip user interface at http://localhost:3100 without a connection error — the start or onboarding screen, with no company created.
- evidence/lab01-installation.md records all three checks, dated, and contains no credential, connection string or personal identifier.

A useful additional signal: you should be able to state, in one sentence and without looking it up, what your evidence file does NOT prove. If you can say "it proves the control plane is healthy and says nothing about any agent runtime", you have the concept as well as the artifact.

Common failures

Embedded PostgreSQL refuses to start

- SYMPTOM — Onboarding aborts partway through with a database error, or doctor reports that database connectivity fails, and the message points at PostgreSQL rather than at your shell.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- WHY IT HAPPENS — The shell is elevated. The default setup starts embedded PostgreSQL, and PostgreSQL refuses to run as an administrative user. It can also happen when the practice directory is owned by a different user from the one running the command.
- HOW TO FIX — Confirm the shell is NOT elevated. Stop the process and re-run onboarding as the normal user who owns the practice folder.
- HOW TO AVOID — Close every Administrator or root terminal before you start the lab, and create the practice root as the same user who will run Paperclip.

The browser cannot reach port 3100

- SYMPTOM — The dashboard shows a connection refused or a timeout, even though onboarding appeared to succeed.
- WHY IT HAPPENS — Most often the server is simply not running: the terminal that was hosting it was closed or interrupted. Less often the server is bound to a different interface, or something else already holds the port.
- HOW TO FIX — Run npx paperclipai doctor, then npx paperclipai run, and query /api/health again BEFORE touching any firewall setting.
- HOW TO AVOID — Give the server its own terminal and do everything else in a second one. Reaching for the firewall first is the classic time sink here: firewall rules rarely block loopback traffic, so a local connection failure is almost never a firewall problem.

npx resolves against a private registry

- SYMPTOM — npx cannot find the package, downloads something unexpected, or fails with an authentication error from a host that is not registry.npmjs.org.
- WHY IT HAPPENS — The machine has a corporate or private npm registry configured globally, and npx honours it.
- HOW TO FIX — Retry with the explicit registry flag, then review the global npm registry configuration afterwards so the next command behaves predictably.
- HOW TO AVOID — Use the explicit registry form the lab gives you, which is why step 3 includes it by default rather than as a fallback.

```
npx --registry https://registry.npmjs.org paperclipai onboard --yes
```

Going further

CHALLENGE — Run npx paperclipai env and record ONLY the non-sensitive instance name, bind mode and port in your evidence note. Do not paste the full output.

HINT — The env command prints the environment variables a deployment needs, in two sections, Required and Optional, marking each as set, default or missing. The two required variables are PAPERCLIP_AGENT_JWT_SECRET and DATABASE_URL. That is exactly why the challenge tells you not to paste the output: one of those two is a signing secret and the other is a connection string, and both belong in your instance, not in your evidence file. The skill being examined is redaction under time pressure —

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

reading a rich diagnostic output and extracting only the three facts that are safe to keep. If you find yourself wanting to paste "just this one line", that is the moment the habit is being formed.

Think about it

REFLECTION — Why does a healthy Paperclip API prove the control plane but say nothing about whether a Claude Code, Codex, Gemini or Cursor runtime is ready?

MODEL ANSWER — Because they are different processes with different failure modes, separated by a deliberate architectural boundary. Paperclip is the control plane: a Node.js server with a React interface over PostgreSQL that stores companies, agents, issues, budgets, approvals and run records, and decides WHEN an agent should wake and WHAT it is allowed to do. The reasoning itself happens in a separate runtime — Claude Code, Codex, Gemini CLI, Cursor and the rest — reached through an adapter. Those runtimes must be installed and authenticated OUTSIDE Paperclip; Paperclip cannot authenticate them for you. So /api/health returning ok tells you the server is listening and the database is reachable. It cannot tell you that the Claude Code binary exists on the PATH, that an ANTHROPIC_API_KEY is present and valid, or that the runtime has a working session. The only evidence that a runtime is genuinely connected is an artifact it produced in the workspace you expected — which is exactly the marker run you will build in Lab 3. A green badge on a configuration screen records that a value was saved; an artifact records that work was done.

E2. Lab 2 — Create the BrightFunnel Company, Mission and CEO (Digital Marketing)

TOPIC 2 · Building AI Agents with Paperclip · approximately 60 minutes · maps to LO2: create a goal-aligned Paperclip company with a measurable mission, a hardened CEO agent and enforced company-level caps.

Why this lab exists

A company in Paperclip is defined by the documentation as a self-contained AI organisation: one goal, a team of agents, a task board, a budget. Three of those four are created in this lab, and the ORDER in which you create them is the lesson. Most people, left alone with the interface, hire an agent first because that is the exciting part, then write a goal because a field asks for one, and set a budget last or never. That sequence produces a company that can spend before it has a ceiling and act before it has a direction. Lab 2 reverses it deliberately: mission, then cap, then hire.

BrightFunnel Marketing makes the cost of getting this wrong concrete. It is a one-person digital marketing consultancy whose entire commercial proposition is cost per qualified lead. Its simulated channel data shows the spread plainly: Referral produces qualified leads at SGD 13.30 each, Email Nurture at SGD 10.64, Organic Search at SGD 15.89

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

— while LinkedIn Ads costs SGD 58.28 per qualified lead and Meta Ads SGD 42.23. A consultant whose business is telling clients where their money leaks cannot credibly run an agent company with no ceiling on its own spend. The budget cap is not administration; it is the same discipline the company sells.

The second thing this lab teaches is what a mission has to contain to be usable. The documentation warns that vague goals give agents nothing concrete to anchor their work to, and its own example replaces "grow the business" with a target, a metric and a timeframe. BrightFunnel's mission is written to the same standard: a target (30% increase), a metric (qualified inbound leads), a timeframe (one quarter) and a boundary (blended cost per qualified lead under SGD 45). Every one of those four parts is something you could later hold an agent's output against. "Improve the funnel" is not — and an agent will happily optimise it, very efficiently, in a direction you did not intend.

The company

BrightFunnel Marketing is a fictional one-person digital marketing company. Its synthetic mission, entered verbatim in step 2, is:

```
Increase qualified inbound leads by 30% in one quarter
while keeping blended cost per qualified lead under SGD 45.
```

The single agent role created in this lab is the MARKETING DIRECTOR, hired as CEO. It owns goal alignment, decomposition and final synthesis. In Paperclip's model the first agent you create is always the CEO — the documentation is blunt that there is no way around this and no point in trying to avoid it — and the CEO is the only agent with no manager. Every other agent, in this and every later lab, reports to exactly one manager.

The lab works from two synthetic data files in labs/lab-02-brightfunnel-marketing/data/:

BrightFunnel's synthetic data files

File	Shape	What it is for
leads.csv	Daily rows: date, channel, campaign, spend_sgd, leads, qualified_leads, covering July–August 2026	The transaction-level record the Marketing Director reasons over. Roughly forty rows — small enough to check by hand, which is what makes it a teaching set.
channels.csv	Six aggregated channel rows: channel, campaign, total_spend_sgd, total_leads, total_qualified_leads, cost_per_qualified_lead_sgd, qualification_rate_pct	The summary view. Because it aggregates leads.csv, you can verify an agent's arithmetic against a second file rather than taking its word for a total.
brightfunnel-ceo-instructions.md	A role-contract template with marked <...> sections	The starting point for the CEO instruction file in step 5. Copy it; edit only where marked.

The six simulated channels are Google Search (BF-SEM-Q3-Brand), Meta Ads (BF-SOC-Q3-Retarget), LinkedIn Ads (BF-LIN-Q3-SME-Owners), Organic Search (BF-SEO-Q3-Guides), Email Nurture (BF-EML-Q3-Reactivate) and Referral (BF-REF-Q3-

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Partners). Qualification rates range from 27.4% on Meta retargeting to 63.2% on referral. Every figure is fabricated for the classroom and describes no real campaign.

Before you begin

- LAB 1 COMPLETE — Paperclip is running, doctor is clean, and /api/health reports ok. If any of those three is untrue, go back; every failure in this lab will otherwise have two candidate causes instead of one.
- ONE TRAINER-APPROVED AGENT RUNTIME installed and ALREADY AUTHENTICATED OUTSIDE Paperclip. This is the boundary from Lab 1 becoming a prerequisite: Paperclip cannot authenticate a runtime for you.
- The paperclip-tgs practice root retained, with its workspace/ and evidence/ folders.
- A text editor for writing the CEO role-contract instruction file.

Note: You will need a provider API key for the adapter you choose — Anthropic keys begin sk-ant-, OpenAI keys begin sk-. Enter it in the agent's environment variables as the adapter requires, or store it as a secret and reference it. Do not put it in your evidence file, your instruction file, or any file inside a workspace.

Walkthrough

Step 1 — Start company creation

Open the Paperclip dashboard and start company creation. Use the exact synthetic company name so your evidence matches the marking guide.

The company is the outermost container in Paperclip's model and everything else — agents, issues, budgets, skills, secrets — is scoped inside it. That scoping is real isolation, not a label: skills are company-scoped and are never shared across companies, and an agent authenticated to one company cannot act in another. Creating the company first therefore establishes the boundary that every later object lives within. The company name itself is a label for reference only and does not affect agent functionality, but using the exact name given makes your evidence directly comparable against the marking guide, and it can be renamed later in company settings if you must.

```
http://localhost:3100   →   Companies → New Company
Name: BrightFunnel Marketing
```

Step 2 — Enter the measurable mission

A mission needs a target, a metric and a boundary — "grow the funnel" is not verifiable, this is.

The goal is described in the documentation as the north star that every agent in the company can see, and it is the text the CEO reads first on its very first heartbeat before proposing any strategy. That is why it is entered now, at creation, rather than added later: the quality of the first strategy proposal is bounded by the quality of the goal it was derived from. A goal with no metric produces a strategy with no acceptance criteria, which produces issues no one can close honestly.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Enter the mission as plain text exactly as given. It states a target (30%), a metric (qualified inbound leads), a timeframe (one quarter) and a boundary (blended cost per qualified lead under SGD 45). Keep the boundary — a mission with a target but no boundary is an instruction to buy the metric at any price, and in a marketing company that is precisely the failure mode.

```
Increase qualified inbound leads by 30% in one quarter
while keeping blended cost per qualified lead under SGD 45.
```

Step 3 — Set the company budget cap BEFORE hiring

This is the outermost ring of the layered budget model and it must exist first.

Paperclip enforces budgets at three scopes — company monthly, per-agent monthly and project lifetime — and the documentation is emphatic that these are PARALLEL and simultaneous rather than nested. An agent can be paused by its own budget limit even while the company has remaining budget, and a project can be paused regardless of either. Setting the company cap first does not make the other two automatic; it makes the outermost ceiling exist before anything is able to spend against it.

Enforcement is two-stage and the numbers are exact. At 80% utilisation Paperclip records a warning and the agent CONTINUES running. At 100% the agent is automatically paused with pauseReason set to budget, and no further heartbeats fire — the documented default is pauseOnExceed = true. The utilisation bar reads green below 70%, yellow from 70 to 90%, red above 90%. Skip this step and the first agent you hire is a process with an API key and no ceiling.

```
Company → Settings → Budget
Monthly cap: SGD 40.00
```

Note: Monthly budgets reset to zero on the 1st of each UTC month. Project budgets are lifetime and never reset. An auto-paused agent does not lose its work: tasks it had in progress remain assigned to it and resume when the agent is resumed.

Step 4 — Hire the CEO

The Marketing Director owns goal alignment, decomposition and final synthesis for BrightFunnel.

This is the first agent, so Paperclip makes it the CEO automatically and the hire form gives it no Reports To field — the CEO reports to the board, meaning you. That single structural rule is what makes the org chart a tree rather than a graph: every agent reports to exactly one manager, and the CEO is the only agent with no manager. It matters operationally as well as conceptually, because Paperclip raises an Invalid Org Chain banner when a reporting chain is broken, and a broken chain blocks work assignment.

Hiring after the cap, rather than before, means the agent exists inside a ceiling from its first second. Note also that hiring now does not start anything: heartbeat on interval is

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

OFF by default, and the agent will sit in idle until something wakes it. That is the intended state at this point in the lab.

```
Agents → Hire → Role: Marketing Director
Reports to: (none — top of org chart)
```

Step 5 — Write the CEO instruction file

State PURPOSE, ALLOWED INPUTS, OUTPUT SCHEMA, EXCLUSIONS and ESCALATION — a role contract, not a personality.

Instructions are how an agent knows what it is for. Paperclip's own recommended structure separates an operating manual (AGENTS.md, the default entry file) from persona and decision philosophy (SOUL.md), a per-heartbeat execution checklist (HEARTBEAT.md) and tool notes (TOOLS.md), and the guidance for the persona file is to keep it under about forty short lines and avoid generic advice. The template in data/brightfunnel-ceo-instructions.md follows the same philosophy in a single file appropriate to a one-agent company.

The five sections are not stylistic. PURPOSE anchors the agent to the mission. ALLOWED INPUTS bounds what it may read, which is what stops a marketing agent wandering into a finance folder. OUTPUT SCHEMA is what makes acceptance possible at all — you cannot verify an artifact whose shape was never specified. EXCLUSIONS are the highest-value lines in the file, because they are the only place you say what must never happen. ESCALATION tells the agent what to do when it is stuck, which is the difference between a blocked issue with a stated blocker and an agent quietly inventing a way forward.

```
workspace/marketing-director/INSTRUCTIONS.md
(copy the template from data/brightfunnel-ceo-instructions.md)
```

Note: Instruction changes take effect on the agent's NEXT heartbeat, not immediately. If you edit the file mid-run, the run in flight is still working from what it read when it woke.

Step 6 — Attach a dedicated workspace

Attach a dedicated workspace so the CEO's files and evidence stay confined to one root and every change is attributable.

The working directory is the folder on your machine where the agent does its work. Confining it now, while there is only one agent, establishes the pattern you will depend on in Lab 3 where three agents must not write into one another's space. Attribution is the reason: if two agents share a root, a file appearing in it is evidence that SOMETHING ran, not that a particular agent ran. In Lab 3 you will prove each boundary with a marker run, and that proof is only meaningful if the roots are separate to begin with.

```
Agent → Workspace → Path: ./workspace/marketing-director
```

Step 7 — Select and verify the runtime adapter

Verification means Paperclip reports a successful connectivity check, not merely that a value was saved.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The adapter is the connection between Paperclip and the AI system that actually runs your agent. The documented type keys are claude_local, codex_local, gemini_local, opencode_local, cursor, pi_local and hermes_local, plus process, http and openclaw_gateway — eleven in total, of which the last three are functional but shown as coming soon in the interface. Use whichever your trainer has approved for the room; the discipline is identical across all of them.

Verification is a distinct action from saving, and the distinction is the point of the step. Saving records that a value exists in a field. Verifying asks Paperclip to attempt a real connectivity check against the runtime — which is the first moment the Lab 1 boundary is actually tested. If the runtime is not installed, or not authenticated outside Paperclip, this is where you find out, and no amount of re-typing the key inside Paperclip will fix it.

```
Agent → Runtime → (trainer-approved adapter)
Click Verify connection
```

Note: Changing the adapter type resets the form to that adapter's defaults. If you switch adapters after configuring one, re-check every field rather than assuming your earlier values survived.

Step 8 — Set the CEO's individual agent budget

Set it BELOW the company cap so the inner ring always binds first.

The per-agent budget is a separate policy from the company budget and both apply simultaneously. Setting the agent cap at SGD 15.00 against a company cap of SGD 40.00 means the agent stops first, which is what you want: a single misbehaving agent hits its own ceiling and pauses, while the company retains headroom for everything else. If the two caps were equal, the first agent to misbehave would consume the entire company's monthly ceiling before anything stopped it.

For orientation, the documentation's own starting guidance is USD 30–50 per month for a CEO, USD 20–40 for a manager such as a CTO or CMO, and USD 10–25 for a worker agent, with an expectation of USD 5–20 to experiment and USD 20–100 per month for an active company. The classroom figures in this course are deliberately lower, in Singapore dollars, and sized so that a two-day course cannot run up a surprising bill.

```
Agent → Budget → Monthly cap: SGD 15.00
```

Step 9 — Import the mock lead and channel data

Import the BrightFunnel mock data so the company has realistic synthetic material to reason over.

Copying the data into the CEO's workspace rather than pointing at the course folder keeps the ALLOWED INPUTS section of the instruction file honest: the agent may read what is inside its own root, and the material it needs is there. It also means the workspace is self-contained if you later inspect it with git, which is how Labs 3, 4 and 6 verify work.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cp -r labs/lab-02-brightfunnel-marketing/data ./workspace/marketing-director/data
```

Step 10 — Record the redacted configuration baseline

Record company name, mission, CEO role, adapter NAME only, both budget caps and the workspace path. Never record keys.

This baseline is what you will compare against when something changes later, and it is the second half of the evidence discipline you started in Lab 1. Record the adapter NAME — that it is, say, claude_local — and never the key, the environment block or the connection details. Paperclip itself follows this convention: reads of an agent's configuration redact secrets, and the CLI's own configuration command is documented as safe to print and share precisely because it masks them.

`evidence/lab02-brightfunnel-baseline.md`

What good looks like

- BrightFunnel Marketing exists as a company, with the EXACT mission text — target, metric, timeframe and boundary all present and unedited.
- The Marketing Director is the SOLE top-level agent: no Reports To, no siblings, and the org view shows a single node.
- The adapter shows a SUCCESSFUL verification, not merely a saved value — you clicked Verify and Paperclip answered.
- The agent has a confined workspace at ./workspace/marketing-director and an instruction file with all five sections present.
- The agent cap (SGD 15.00) is strictly BELOW the company cap (SGD 40.00), and both are saved rather than merely typed.
- evidence/lab02-brightfunnel-baseline.md contains no key, token or connection string.

A good baseline note fits on one page and could be handed to a colleague who would then be able to rebuild the same company without asking you a question — except for the credential, which they must obtain themselves. If your note fails that test, it is either missing a fact or containing a secret.

Common failures

The adapter verification fails

- SYMPTOM — Verify connection returns an error, or the agent later sits in error status with an adapter message in its run transcript.
- WHY IT HAPPENS — The underlying CLI runtime is not installed on this machine, or it is installed but not signed in. Paperclip reaches the runtime as a separate process; it does not hold your provider session for you.
- HOW TO FIX — Confirm the runtime is installed and authenticated OUTSIDE Paperclip first — run its own CLI directly and sign in — then re-verify. Paperclip cannot authenticate a runtime for you.
- HOW TO AVOID — Treat runtime authentication as a prerequisite of the lab, not a step within it. Verify the runtime works standalone before you open Paperclip at all.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The mission field rejects the text

- SYMPTOM — The mission will not save, or saves with visible mangling.
- WHY IT HAPPENS — The text was pasted from a word processor, carrying smart quotes, non-breaking spaces or hard line breaks.
- HOW TO FIX — Remove smart quotes and line breaks; enter the mission as plain text.
- HOW TO AVOID — Type or paste through a plain-text editor. This matters beyond the form: a mission containing typographic quotes is a mission your later grep and diff checks will silently fail to match.

The budget cap will not save

- SYMPTOM — The budget field rejects the value or reverts on save.
- WHY IT HAPPENS — The ordering was inverted: an agent cap was set against a company cap that is missing or smaller.
- HOW TO FIX — Set the company cap BEFORE the agent cap. An agent cap above a missing or smaller company cap is rejected.
- HOW TO AVOID — Follow the lab order exactly: mission, company cap, hire, agent cap. The order is the lesson, and this failure is the lesson enforcing itself.

Going further

CHALLENGE — Rewrite the BrightFunnel mission so it would FAIL a feasibility scan, then state in one sentence exactly which property (target, metric or boundary) you removed.

HINT — Do not write a bad mission at random; remove exactly one property and leave the rest intact, so the damage is attributable. Removing the TARGET gives you "Increase qualified inbound leads while keeping blended cost per qualified lead under SGD 45" — an agent can satisfy this by producing one extra lead. Removing the METRIC gives you "Improve marketing performance in one quarter under SGD 45 per unit" — nobody can say what was measured. Removing the BOUNDARY gives you "Increase qualified inbound leads by 30% in one quarter" — which reads well and is the most dangerous of the three, because it instructs the agent to buy the target at any price. Given that BrightFunnel's own data shows LinkedIn Ads at SGD 58.28 per qualified lead against Referral at SGD 13.30, an unbounded mission would rationally select the most expensive channel. Write the sentence naming which property you removed and what the agent could now legitimately do.

Think about it

REFLECTION — Why is setting the company budget cap before hiring the CEO a control decision rather than an administrative one?

MODEL ANSWER — Because the cap is the only mechanism that stops spend without a human being present, and an agent hired before the cap exists has a window in which

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

nothing stops it. Paperclip's budget enforcement is automatic and two-stage: at 80% utilisation it records a warning and the agent keeps running, and at 100% it pauses the agent automatically with pauseReason set to budget so that no further heartbeats fire. That is a control in the governance sense — it acts on its own, at a defined threshold, whether or not anyone is watching. Sequencing therefore changes the risk profile, not just the paperwork: hire first and there is a period, however short, in which a misconfigured instruction file or a retry loop can spend against no ceiling, and an agent in a retry loop does not get tired. There is a second, subtler reason. The three budget scopes are parallel rather than nested — company monthly, agent monthly and project lifetime all enforce simultaneously, and an agent can be paused by its own limit while the company still has headroom. Establishing the outermost ceiling first makes that relationship visible from the beginning, so that when you set the agent cap at SGD 15.00 against a company cap of SGD 40.00 you are making a deliberate statement about which ring should bind first, rather than discovering the interaction the month you get a surprising bill.

E3. Lab 3 — Build the Meridian Reporting Tree and Validate Runtime Boundaries (Finance)

TOPIC 2 · Building AI Agents with Paperclip · approximately 75 minutes · maps to LO2: build a multi-agent reporting hierarchy with distinct, budgeted and confined workspaces, and prove each runtime boundary with a marker run.

Why this lab exists

Lab 2 produced a company with one agent, which is a prompting problem with extra steps. Lab 3 produces an organisation, and organisations have properties a single agent does not: a reporting line, divided responsibility, separate budgets and — the part everyone underestimates — separate places to work. This lab exists to teach that a list of agents is not an org chart, and that a configured connection is not a proven one.

Meridian Finance Desk is the right sector for the lesson because finance is where attribution is not optional. Its mission is to produce a board-ready monthly financial pack within three working days of month end, with every figure traceable to a source ledger extract. Traceable is the operative word. If an FP&A figure and a treasury figure both appear in the pack and you cannot say which analyst produced which, the pack is not board-ready no matter how correct the arithmetic is. In a real finance function this is the difference between a review and an investigation. Separate workspaces, separate instruction files and separate budgets are what make that attribution structural rather than a matter of trust.

The second and larger idea in this lab is the MARKER RUN. Configuration screens are generous: they will show you a saved value and a green indicator, and both are true statements about the database rather than about the world. A marker run is a deliberately trivial task whose only purpose is to produce one named artifact in one

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

named place. It is the smallest possible experiment that distinguishes "Paperclip believes this agent has a runtime and a workspace" from "this agent's runtime actually executed and actually wrote to the folder I expected". Once you have run one, you will never again accept a badge as proof — and you will have a two-minute diagnostic for every adapter problem in Labs 4, 5 and 6.

The company

Meridian Finance Desk is a fictional finance function. Its synthetic mission, entered at company creation, is to produce a board-ready monthly financial pack within three working days of month end, with every figure traceable to a source ledger extract. Its company monthly cap is SGD 45.00.

It has three agent roles, and unlike Lab 2 the shape of the tree matters as much as the roles themselves:

The Meridian reporting tree

Role	Reports to	Monthly cap	Owns
Finance Lead (CEO)	None — top of the org chart	SGD 20.00	Mission alignment, decomposition, final synthesis of the pack.
FP&A Analyst	Finance Lead	SGD 10.00	Analysis against the trial balance; variance narrative.
Treasury Analyst	Finance Lead	SGD 10.00	Cash and bank position against the statement extract.

The three caps total SGD 40.00 against a company cap of SGD 45.00, which is the arithmetic the verification step checks. Note that this is a deliberate design choice rather than a platform rule: because the three budget scopes enforce in parallel, keeping the sum of agent caps below the company cap means the company ceiling is a genuine backstop rather than the first thing to bind.

The synthetic ledger lives in labs/lab-03-meridian-finance/data/ledger/ and is copied to a shared read-only data folder so both specialists reason over identical source figures:

Meridian's synthetic ledger extracts

File	Shape	Why it is shaped that way
trial-balance-2026-09.csv	account_code, account_name, ledger_balance_sgd — around twenty accounts	The ledger side of every comparison. Includes cash accounts such as 1010 Cash at Bank - DBS Operating and 1020 Cash at Bank - OCBC Reserve.
bank-statement-2026-09.csv	date, description, type, amount_sgd, running_balance_sgd — around twenty-five movements from an opening balance	The independent side. Because it carries a running balance, an agent's reconciliation can be checked without recomputing the whole month.
meridian-role-instructions/fpa-analyst.md	A role contract with PURPOSE, allowed inputs, output schema, exclusions and escalation	Copied to workspace/fpa-analyst/INSTRUCTIONS.md in step 5.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

meridian-role-instructions/ treasury-analyst.md	The same structure for the treasury role	Copied to workspace/treasury-analyst/INSTRUCTIONS.md. The two differ mainly in their EXCLUSIONS, which is the point.

Note: Every figure in these extracts is fabricated for the classroom. They are not real financial data and must never be treated as such, quoted outside the room, or reused in any real reporting.

Before you begin

- LAB 2 COMPLETE — BrightFunnel exists with a verified CEO adapter and layered caps saved. You should be able to create a company, enter a mission and set a cap without referring back to the instructions.
- GIT INSTALLED, with a local practice user.name and user.email configured, so each workspace can commit a baseline without contacting a remote.
- ABSOLUTE PATHS AVAILABLE for the three Meridian workspace roots. Relative paths that resolve differently depending on which terminal you are in are the most common cause of a misbound workspace.
- The same trainer-approved adapter may be reused for all three agents — this lab tests boundaries between agents, not between adapters.

Walkthrough

Step 1 — Create the Meridian company

Create the company with its measurable mission and a company budget cap, using the same order you proved in Lab 2.

This step is deliberately a repetition. Lab 2 taught the order — mission, then cap, then hire — and repeating it here without the scaffolding is how it becomes a habit rather than a procedure you followed once. Note the mission's shape: "within 3 working days of month end" is the timeframe, "board-ready monthly financial pack" is the target and "every figure traceable to a source ledger extract" is the boundary. That last clause is what the whole lab then goes on to make structurally true.

```
Companies → New Company
Name: Meridian Finance Desk
Mission: Produce a board-ready monthly financial pack within 3 working days of
month end, with every figure traceable to a source ledger extract.
Company monthly cap: SGD 45.00
```

Step 2 — Hire the tree, with reporting lines set explicitly

Hire the Finance Lead as CEO, then hire the two specialists AND set the reporting line explicitly. A flat list of agents is not an org chart.

The Finance Lead must be hired first because the first agent in any company is always the CEO and has no Reports To field. The two analysts are then hired with Reports To pointing at the Finance Lead. Setting that field is not decoration: the reporting chain is

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

what Paperclip uses for escalation, and it validates the chain actively. An agent whose manager is missing, terminated, or part of a loop triggers an Invalid Org Chain banner, and the documented impact is that it BLOCKS WORK ASSIGNMENT. Separately, an Escalation Path Paused banner appears when a paused agent sits anywhere up the reports-to chain, with escalations refused until you unpause the manager or change the assignment.

This is why a flat list fails in practice rather than merely in principle. Three agents with no reporting lines will each accept work and none of them will have a functioning escalation path, so the first time one of them gets stuck, the blockage has nowhere to go.

```
Agents → Hire
  Finance Lead      (reports to: none)
  FP&A Analyst      (reports to: Finance Lead)
  Treasury Analyst  (reports to: Finance Lead)
```

Step 3 — Create one workspace root per agent

Separate roots are what make a later diff attributable to exactly one role.

This is the physical half of the org chart. The reporting line says who answers to whom; the workspace roots say where each of them may write. Create all three before binding any of them, so that step 7 is a straightforward mapping exercise rather than a sequence of create-and-bind operations in which it is easy to lose track of which path you actually typed.

```
mkdir -p workspace/{finance-lead,fpa-analyst,treasury-analyst}
```

Step 4 — Initialise Git and commit an empty baseline in each workspace

So that every later change is a readable diff against a known-clean state.

This is the step that turns "I think the agent wrote something" into evidence. An empty baseline commit gives each workspace a defined zero point, so that git status afterwards reports exactly what a run produced and nothing else. Without it, the workspace already contains the instruction file and the copied data, and any status output is a mixture of things you put there and things the agent did — which is precisely the ambiguity the lab is trying to eliminate.

Do this BEFORE the marker runs, not after. A baseline taken after a run cannot distinguish the run's output from the pre-existing state, and re-running to compensate wastes budget you have deliberately capped.

```
for w in finance-lead fpa-analyst treasury-analyst; do \
  git -C workspace/$w init -q && \
  git -C workspace/$w commit -q --allow-empty -m 'baseline'; done
```

Step 5 — Write a role instruction file for each specialist

State purpose, allowed inputs, output schema, exclusions and escalation. The exclusions matter most.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Two agents with identical instructions are one agent running twice, at double the cost. What actually differentiates the FP&A Analyst from the Treasury Analyst is not their titles but their EXCLUSIONS — the explicit statements of what each must not do, which prevent them producing overlapping, contradictory analysis of the same numbers. Read both templates side by side before you copy them; the differences are the design.

The escalation section earns its place here in a way it did not in a one-agent company. These specialists have a manager, so "escalate to the Finance Lead" is now a real instruction with a real destination, and the reporting line you set in step 2 is what makes it executable.

```
workspace/fpa-analyst/INSTRUCTIONS.md
workspace/treasury-analyst/INSTRUCTIONS.md
(templates in data/ledger/meridian-role-instructions/)
```

Step 6 — Load the ledger extracts into a shared read-only data folder

So both specialists reason over identical source figures.

This is the one place in the lab where sharing is correct rather than dangerous. Workspaces are separated so that OUTPUTS are attributable; inputs are shared so that the two analyses are comparable. If each specialist held its own copy of the trial balance, a discrepancy between their outputs could be a difference in reasoning or a difference in the data, and you would have no way to tell which.

```
cp -r labs/lab-03-meridian-finance/data/ledger ./data/ledger
```

Step 7 — Bind each agent to its workspace, adapter and budget

Bind each agent to its own workspace path and its own adapter, then set an individual budget for each so the caps are layered company → agent.

Three bindings per agent, done together so nothing is left half-configured. Use absolute paths or verify the relative ones resolve from the directory Paperclip actually runs in — a misbound workspace is the single most common failure in this lab and it is silent until the marker run exposes it. The individual budgets create the layering: SGD 20.00 for the lead, SGD 10.00 for each analyst, totalling SGD 40.00 against a SGD 45.00 company cap.

Remember that these two scopes enforce in parallel. The Treasury Analyst hitting SGD 10.00 pauses the Treasury Analyst and nothing else — the FP&A Analyst continues, the Finance Lead continues, and the company still has headroom. That is the behaviour the stretch challenge asks you to observe directly.

```
Each agent → Workspace + Runtime + Budget
  Finance Lead     SGD 20.00
  FP&A Analyst     SGD 10.00
  Treasury Analyst SGD 10.00
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 8 — Run a marker run for each agent

Assign a trivial task whose only output is a single named file. This is how you prove the runtime boundary is real.

The task is deliberately as small as a task can be: create one file, containing one role name and one date, and do nothing else. Its triviality is the design. A marker run costs almost nothing, completes quickly, and produces an unambiguous binary result — the file is in the right root or it is not. A complex first task confounds three questions at once (did the runtime execute, did it write where I expected, did it do the work correctly) and gives you no clean answer to any of them.

Note the mechanism as well as the purpose. Assignment is one of the five documented heartbeat triggers — schedule or interval, task assignment, comment mention, manual invoke via Run Heartbeat, and approval resolution — so assigning this issue is what wakes the agent. Creating an issue and leaving it unassigned wakes nobody, which is a property Lab 4 depends on entirely.

```
Issues → New
Title: Marker run — <agent>
Body: Create a file named marker.md in your workspace root containing only
      your agent role name and the current date. Do nothing else.
```

Step 9 — Inspect each run and confirm the artifact landed correctly

Confirm the artifact landed in the CORRECT workspace — and nowhere else. A marker in the wrong root is a boundary failure, not a success.

Check all three roots, not just the one you expect. The failure this catches is the one that looks like a success: a marker file exists, the run reports success, and it is in the wrong folder because two agents share a misbound path. Checking only the expected root would confirm the file exists and miss entirely that a second agent wrote it. Because you committed empty baselines in step 4, git status --short in each root shows exactly one untracked marker.md in exactly one place if the boundaries hold.

```
git -C workspace/fpa-analyst status --short
git -C workspace/treasury-analyst status --short
git -C workspace/finance-lead status --short
```

Step 10 — Record the evidence

Record the org chart, the three budgets, the three marker-run IDs and the three artifact paths in your evidence note.

Run IDs are the key that connects everything else. Paperclip's run records hold the invocation source, the token counts and cost, the transcript with tool calls inline, the logs and the list of touched issues, and every issue state change made by an agent carries the run ID in the X-Paperclip-Run-Id header. Recording the ID now means that in Lab 6, when you reconstruct an entire operation from the activity log, you have the identifiers to join the record together.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

`evidence/lab03-meridian-org.md`

What good looks like

- The org view shows a THREE-AGENT TREE with both analysts reporting to the Finance Lead — one root, two children, no orphans and no Invalid Org Chain banner.
- Each workspace contains exactly ONE marker.md, attributable to its own agent, and the other two roots contain none.
- Every agent has an individual cap, and the sum of the agent caps (SGD 40.00) does not exceed the company cap (SGD 45.00).
- Each marker run has a recorded run ID in your evidence note, alongside the artifact path it produced.

The strongest version of this evidence is three git status outputs, each showing one untracked file, plus three run IDs. That pairing — a platform record and an independent filesystem observation of the same event — is what makes the claim hard to dispute. Either alone is weaker: the run record could describe work that went somewhere unexpected, and the file could have been created by hand.

Common failures

A marker file appears in the wrong workspace

- SYMPTOM — git status shows marker.md in a root belonging to a different agent, or the same root shows two markers.
- WHY IT HAPPENS — The agent's workspace path is misbound: typically a relative path that resolved from an unexpected directory, or a path copied from another agent and not edited.
- HOW TO FIX — Correct the path on the AGENT (not the company), delete the stray file, and re-run the marker.
- HOW TO AVOID — Use absolute paths, and bind all three agents in one sitting so you can read the three paths together and see the difference between them.

A specialist never wakes

- SYMPTOM — The issue sits in todo, the agent stays idle, no run appears.
- WHY IT HAPPENS — Almost always the issue is not assigned. Creating an issue does not wake anyone; assignment is the trigger. Heartbeat on interval is off by default, so there is no timer to rescue you. Other causes: the agent is paused, or it has no budget headroom.
- HOW TO FIX — Confirm the issue is ASSIGNED to that agent. Then check the agent's adapter verification and its budget headroom.
- HOW TO AVOID — Learn the five triggers and check them in order — assignment, mention, manual invoke, schedule, approval resolution. Also remember the distinction between paused and heartbeat-off: an agent with heartbeats off still

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

wakes on assignment, comment, routine and on demand, whereas a PAUSED agent wakes on nothing at all.

Git reports the workspace is dirty before the marker run

- SYMPTOM — git status shows changes before you have assigned anything.
- WHY IT HAPPENS — Something was written outside a run: a hand edit, a stray copy, or an earlier run you had forgotten.
- HOW TO FIX — Reset to the baseline commit and re-run, so the diff means what you think it means.
- HOW TO AVOID — Commit the empty baseline AFTER copying instructions and data, and then do not touch the workspace by hand. Anything a human writes into an agent workspace becomes an unexplained gap in an audit trail — a point Lab 6 makes explicitly.

Going further

CHALLENGE — Set the Treasury Analyst's cap to SGD 0.01, re-run its marker task, and record exactly where and how Paperclip reports the budget stop.

HINT — You are inducing the 100% condition deliberately in the cheapest possible place. Look in three locations and compare what each tells you. On the AGENT, expect the status to become paused with pauseReason budget, since the documented default is pauseOnExceed = true. On the DASHBOARD, the agent status panel counts paused agents separately from active, running and error, and the paused definition covers agents stopped deliberately OR automatically at their budget limit. In COSTS, watch the utilisation bar cross the thresholds — green below 70%, yellow 70–90%, red above 90% — and note that 80% records a warning while the agent CONTINUES, and only 100% pauses it. Record which surface told you first and which told you most precisely; that is the operationally useful finding. Then note the recovery path: a hard stop at agent level creates a Budget Override approval, and you resume by raising the limit deliberately or resolving that approval. Restore the SGD 10.00 cap afterwards.

Think about it

REFLECTION — Why is a marker run stronger evidence of a working runtime than a green "connection verified" badge in the configuration screen?

MODEL ANSWER — Because they test different things, and only one of them tests the thing you actually care about. A verification badge records that Paperclip performed a connectivity check at a moment in time and the check passed. That is genuinely useful — it is how you catch a missing runtime or an unauthenticated session in Lab 2 — but it is a statement about reachability, not about execution. It does not establish that the adapter can spawn an agent process, that the process can complete a turn, that the working directory is bound to the folder you intended, or that the agent has permission to write there. A marker run tests the whole chain end to end: the heartbeat fires on assignment, the adapter is invoked, the agent process spawns, the agent calls back to

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

the Paperclip API, work is performed, an artifact is written to disk and a run record is stored with a cost. If any link in that chain is broken, no marker.md appears — and if the workspace is misbound, the marker appears in the wrong place, which a badge could never have told you. There is also an evidential difference. The badge is Paperclip's own testimony about itself. The marker file is an independent observation you make with git, outside Paperclip, about the filesystem. Configuration that has never produced an artifact is not a connection; it is an intention.

E4. Lab 4 — Translate the Ledgerly Mission into Assigned Issues and Hand-Offs (Accounting)

TOPIC 3 · Developing Intelligent Multi-Agent Systems · approximately 90 minutes · maps to LO3: decompose a mission into a parent and specialist child issues, run the specialists sequentially and verify each hand-off against acceptance evidence.

Why this lab exists

Labs 2 and 3 built the organisation. Lab 4 gives it work, and it is where most of the genuinely hard judgement in the course lives. Decomposition — turning a mission into issues that an agent can actually complete and a human can actually verify — is the skill that separates a multi-agent company that produces evidence from one that produces confident prose. The platform will not do this for you. Paperclip enforces checkout, review stages and comment requirements, but it cannot tell you whether an acceptance criterion is checkable.

Ledgerly Accounting exists to make that concrete in the least forgiving domain available. Its mission is to close the books each month with zero unexplained variances above SGD 500 and a complete reconciliation trail for every control account. A month-end close is inherently a decomposition problem with a hand-off in the middle: somebody reconciles, somebody else reviews the exceptions that the reconciliation surfaced, and a third party synthesises the pack. It is also a domain where "reconciliation done" is a meaningless status and "here is the file listing every control account with its ledger balance, statement balance and variance" is a deliverable. The lab's central discipline follows directly: write acceptance criteria that name the EVIDENCE, not the ACTIVITY.

The second discipline is sequencing, and it rests on a mechanical fact you proved in Lab 3. Assignment is one of the five heartbeat triggers, so creating an issue unassigned wakes nobody, and assigning it is the deliberate act that starts work. Creating every issue unassigned therefore separates PLANNING from EXECUTION completely — you can build an entire issue tree, review its acceptance criteria, correct them, and only then spend money. Assigning one child at a time keeps each run individually attributable: two agents running concurrently produce interleaved run records and overlapping timestamps, and if the second one's output depends on the first one's artifact, you have also created a race.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The company

Ledgerly Accounting is a fictional accounting practice. Its synthetic mission is to close the books each month with zero unexplained variances above SGD 500 and a complete reconciliation trail for every control account. Its caps are layered: company SGD 45.00, CEO SGD 20.00, each specialist SGD 10.00, plus a PROJECT cap of SGD 25.00 introduced for the first time in this lab.

The Ledgerly roles

Role	Reports to	Cap	Owns
Practice Manager (CEO)	None	SGD 20.00	The close pack synthesis; the parent issue, assigned only in a later lab.
Reconciliation Specialist	Practice Manager	SGD 10.00	Child 1 — reconciling every control account against the statement extract.
Compliance Reviewer	Practice Manager	SGD 10.00	Child 2 — reviewing every variance above SGD 500 and recommending a treatment.

The synthetic close data lives in labs/lab-04-ledgerly-accounting/data/close/ and is built so that the hand-off has something real to hand over:

Ledgerly's synthetic close extracts

File	Shape	Why it matters to the lab
trial-balance-2026-10.csv	account_code, account_name, ledger_balance_sgd — around nineteen accounts	The ledger side. Runs from 1010 Cash at Bank - UOB Operating through to 3020 Retained Earnings.
bank-statement-2026-10.csv	date, description, type, amount_sgd, running_balance_sgd — around thirty movements	The independent statement side for the cash accounts.
control-accounts.csv	account_code, account_name, ledger_balance_sgd, statement_balance_sgd, variance_sgd, source_document	The file that makes the hand-off testable. It already contains the variances, so you can verify the specialist's arithmetic rather than trusting it.

The control accounts file contains exactly four non-zero variances, and they are sized deliberately around the SGD 500 threshold in the mission — which is what gives the Compliance Reviewer a real filtering decision to make rather than a formality:

The four synthetic variances, and what each tests

Account	Variance (SGD)	Source document	What the reviewer must decide
1100 Accounts Receivable - Trade	1,242.65	AR ageing 2026-10-31	Above SGD 500 — must appear in the exceptions file with a cause and a treatment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1210 GST Input Tax Receivable	869.50	IRAS GST F5 Q3	Above SGD 500 — must appear.
2020 Accrued Expenses	-2,124.90	Accrual schedule	Above SGD 500 in magnitude and NEGATIVE — tests whether the agent filters on absolute value or is caught by the sign.
2100 Deferred Revenue	-637.25	Contract schedule	Above SGD 500 in magnitude and negative — the second sign test.

Note: Every figure in these extracts is fabricated. Ledgerly Accounting is not a real firm, the account balances are invented, and nothing here constitutes accounting guidance.

Before you begin

- LABS 2 AND 3 COMPLETE — you can create a company, hire a tree and set layered caps unaided. This lab does that in a single compressed step and does not re-teach it.
- THE MARKER-RUN TECHNIQUE proved for every adapter you intend to use. If a specialist fails in this lab, you want to already know its runtime works.
- GIT INSTALLED and each workspace committed to a clean baseline, so the specialists' output is readable as a diff.
- AT LEAST 30 MINUTES OF UNINTERRUPTED TIME — the two specialist runs must be SEQUENTIAL, and the whole point is lost if you start the second before the first has settled.

Walkthrough

Step 1 — Create Ledgerly and hire the three roles

Create the company, hire the Practice Manager as CEO plus a Reconciliation Specialist and a Compliance Reviewer, and set layered caps — the pattern you proved in Labs 2 and 3.

Compressed into one step because you have now done it twice. The only new element is the mission's threshold: "zero unexplained variances above SGD 500" is a boundary with a number in it, and that number reappears in the child 2 acceptance criterion in step 6. A mission whose threshold cannot be traced into an acceptance criterion is a mission that will not be tested.

```
Companies → New Company: Ledgerly Accounting
Mission: Close the books each month with zero unexplained variances above SGD 500
and a complete reconciliation trail for every control account.
Company cap SGD 45 · CEO SGD 20 · each specialist SGD 10
```

Step 2 — Create the project and give it its own budget cap

The project ring sits between the company and the agent, and it is what stops one project consuming the company.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A project in Paperclip binds a body of work to a concrete place — a repository, a working directory, a budget envelope and a set of execution workspaces. The documentation's framing is worth keeping: if goals answer why are we doing this and issues answer what exactly needs doing, projects sit in between. Projects have their own status values — backlog, planned, in_progress, completed and cancelled — and their status communicates state without preventing agent work.

The budget behaviour is the reason this step exists. A project budget's window is LIFETIME rather than monthly, its metric is billed_cents, its warn threshold defaults to 80%, and when hard stop is enabled, exceeding the cap pauses the PROJECT — surfaced as a red "Paused by budget hard stop" pill at the top of the project page. Crucially this is the third parallel scope, not a subdivision of the others: a project budget hit pauses the project regardless of agent headroom, an agent budget hit stops the agent regardless of project headroom, and the company budget is the overall ceiling. All three enforce simultaneously.

```
Projects → New
Name: October Month-End Close
Goal: Close October with every control account reconciled and evidenced.
Project cap: SGD 25.00
```

Note: Because project budgets are lifetime, they do not reset on the first of the month the way agent and company monthly budgets do. A long-running project accumulates against its cap indefinitely, which is exactly what you want from an envelope around a defined piece of work.

Step 3 — Load the close data

Load the mock trial balance and bank statement extracts into a shared read-only data folder.

As in Lab 3, inputs are shared so that both specialists reason over identical figures, while outputs remain separated so they stay attributable. Here it does more work than in Lab 3, because child 2's job is to review variances that child 1 computed from this same data — if the two specialists were reading different copies, a disagreement between their outputs would be uninterpretable.

```
cp -r labs/lab-04-ledgerly-accounting/data/close ./data/close
```

Step 4 — Create the PARENT synthesis issue, unassigned

Its acceptance criterion names the artifact, its path and the fields it must contain.

Leave the assignee EMPTY. This is the mechanical heart of the lab: assignment is a heartbeat trigger, so an unassigned issue wakes nobody and costs nothing. That lets you build the entire tree, read it back, and fix a badly worded criterion before a single agent has spent anything. Note also that a new issue defaults to status backlog and priority medium unless you say otherwise — backlog meaning the issue exists and has been identified but nobody is working on it yet.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Read the acceptance criterion carefully, because it is the model for everything that follows. It names a PATH (workspace/practice-manager/close-pack.md), it says the file must EXIST, and it lists the FIELDS every row must contain: account, ledger balance, statement balance, variance, explanation. Someone who has never seen the work can check that. "Produce a close pack" cannot be checked by anybody.

```
Issues → New  (Assignee: leave EMPTY)
Title: Produce the October close pack
Accept: workspace/practice-manager/close-pack.md exists and contains, for every
control account: account, ledger balance, statement balance, variance, explanation.
```

Step 5 — Create the two CHILD issues, parented and unassigned

Both unassigned, each parented to the synthesis issue so goal lineage is explicit and traceable.

Setting the parent is a first-class link, not a note in the description. Paperclip uses parentage structurally: when all of an issue's children reach a terminal state, the parent is woken with the reason issue_children_completed. That is a real mechanism you will rely on in Lab 6, and it only works if the relationship is recorded as a parent rather than described in prose. The same principle applies to dependencies — the documentation is explicit that "A is blocked by B" must be expressed as a first-class blockedByIssueIds link and not as a free-text comment, because a dependent expressed in prose never auto-wakes.

```
Child 1 → Reconcile control accounts   (parent: Produce the October close pack)
Child 2 → Review compliance exceptions (parent: Produce the October close pack)
```

Step 6 — Write acceptance criteria that name the evidence

"Reconciliation done" is unverifiable; a named file with named fields is.

This is the step the whole lab is built around, so give it the time. Each criterion names a path, states what must be in the file, and — for child 2 — carries the mission's SGD 500 threshold into the work itself. Read the two criteria against the synthetic data and notice that they are genuinely testable: control-accounts.csv contains nineteen accounts, so child 1's file should list nineteen; four variances exceed SGD 500 in magnitude, so child 2's file should contain four entries, two of them negative.

That last detail is a deliberate trap worth pointing out to learners. Two of the four material variances are NEGATIVE — accrued expenses at −2,124.90 and deferred revenue at −637.25. An agent that filters on "variance greater than 500" rather than "absolute variance greater than 500" will silently produce a two-row exceptions file that looks perfectly reasonable. This is exactly why acceptance is checked against named evidence rather than against a claim of completion.

```
Child 1 accept: workspace/reconciliation-specialist/reconciliation.md lists every
  control account with ledger balance, statement balance and variance.
Child 2 accept: workspace/compliance-reviewer/exceptions.md lists each variance
  above SGD 500 with a stated cause and a recommended treatment.
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 7 — Assign CHILD 1 only

Assignment is the trigger — this is the moment the Reconciliation Specialist wakes.

One assignment, one agent, one run. Everything up to this point cost nothing; this is the first line item. Assigning only child 1 means that when you inspect the run afterwards you know with certainty which agent produced which artifact, and there is no possibility of the reviewer starting before the thing it reviews exists.

Behind the scenes the agent follows the documented heartbeat protocol: it identifies itself, fetches its assignments, picks work, and CHECKS OUT the issue via the checkout endpoint with the X-Paperclip-Run-Id header — an atomic claim that moves the issue to in_progress and records the run that owns it. Only one agent can hold an issue in progress at a time. It is worth knowing that the correct claim is a checkout and never a status patch to in_progress, because a patch skips checkout, leaves the checkout run ID empty and is race-prone.

```
Child 1 → Assignee: Reconciliation Specialist
```

Step 8 — Wait for the heartbeat, with a bound

Inspect the run log rather than the issue status — an unbounded wait hides a failed adapter behind apparent patience.

Bounding the wait is an operational discipline, not impatience. An issue sitting in todo looks identical whether the agent is thinking, the adapter has failed, the agent is paused or the budget is exhausted; the RUN LOG distinguishes them immediately. The run detail carries status and timing, token counts and cost, the invocation card, the full transcript with tool calls inline and the stdout/stderr logs. A live run streams its transcript, so you can watch progress rather than infer it.

Ten minutes is a sensible bound for a task of this size. If nothing has happened by then, something is wrong, and the fastest diagnosis is the run log followed by the agent's status. Remember that an agent in error status will not fire new heartbeats until the underlying issue is resolved — the error does not fix itself.

```
Runs → (latest)   ·   bound the wait to 10 minutes
git -C workspace/reconciliation-specialist status --short
```

Step 9 — Accept child 1 only on named evidence

Accept ONLY if the named artifact exists with the named fields. Record the run ID and the artifact path in a comment, then close the issue.

Open the file and check it against the criterion. Does it list every control account, or only the interesting ones? Does each row carry a ledger balance, a statement balance and a variance? If a field is missing, the correct response is to fix the CRITERION and re-run, not to hand-edit the agent's artifact — an artifact you edited is no longer evidence of what the agent did.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The comment is the durable record. Paperclip's execution policy makes commenting a runtime invariant for agents — every agent run must post a comment back to the issue, and if it does not, the run's issueCommentStatus becomes retry_queued and the agent is woken with the reason missing_issue_comment; if the comment is still absent it becomes retry_exhausted and the failure is recorded. Your own comment, naming the run ID and the artifact, holds you to the same standard the platform holds the agent to.

Then set the status to done. Done is TERMINAL — issues do not move backwards, and the only route back from done or cancelled is an explicit reopen. Setting done also has documented side effects: it stamps completedAt, wakes any issue whose blockers are now fully resolved, and wakes the parent if all its children are terminal.

```
Issue → Comment: run <run-id> produced reconciliation.md (<n> accounts)
Issue → Status: Done
```

Step 10 — Now assign CHILD 2

Running the specialists sequentially is what keeps each run individually attributable and each hand-off inspectable.

Only now, with child 1 accepted and its artifact on disk, does the Compliance Reviewer have something to review. Assigning both children at the start would have created a race whose outcome depends on which adapter happened to be faster — and on a bad day the reviewer would produce a confident exceptions file derived from nothing at all.

```
Child 2 → Assignee: Compliance Reviewer
```

Step 11 — Verify the hand-off

Confirm the Compliance Reviewer consumed child 1's artifact without needing to ask the producer a question.

This is the actual test of the design. A hand-off works when the receiving specialist can do its job from the artifact alone. If the reviewer had to ask the reconciler a question, the artifact was underspecified — which is a criterion problem, not an agent problem. The grep counts SGD mentions as a crude proxy for whether real figures were carried across, and the diff --stat shows the shape of what the reviewer actually wrote.

Read the exceptions file properly as well as counting it. Four variances exceed SGD 500 in magnitude; a file with two entries has almost certainly fallen into the negative-variance trap described in step 6, and that is a finding worth recording rather than quietly fixing.

```
grep -c 'SGD' workspace/compliance-reviewer/exceptions.md
git -C workspace/compliance-reviewer diff --stat HEAD
```

Step 12 — Record the evidence chain

For each child: issue ID, run ID, artifact path, comment and final status — in one table.

One row per child, five columns. This table is the direct ancestor of the mission-to-evidence audit map you build in Lab 6, and building it now, while the work is fresh, teaches the shape. Note the run timestamps too: non-overlapping start and finish times

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

are your proof that the runs were genuinely sequential, which is one of the verification criteria.

`evidence/lab04-ledgerly-handoffs.md`

What good looks like

- The Ledgerly project shows a saved cap (SGD 25.00) ABOVE actual spend — the envelope held.
- The parent issue is STILL OPEN and STILL UNASSIGNED. Synthesis is Lab 6's job; a parent closed here means it was assigned early.
- Both children are DONE, each with a linked artifact and a recorded run ID in a comment.
- The two specialists ran SEQUENTIALLY — non-overlapping run timestamps, which you can read directly off the two run records.
- The exceptions file demonstrably REFERENCES the reconciliation figures rather than restating the source data independently.

The subtle marker of a good result is that someone else could audit it without you. Given your evidence table, a colleague should be able to open two issues, follow two run IDs, open two files and agree or disagree with your acceptance decision — without asking you a single question. If they would need to ask, a column is missing.

Common failures

Both specialists wake at the same time

- SYMPTOM — Two runs appear with overlapping timestamps; the exceptions file references figures that do not exist yet, or is thin and generic.
- WHY IT HAPPENS — A child was assigned before the previous one was accepted. Assignment is the trigger, so two assignments means two wakes.
- HOW TO FIX — Pause the second agent, let the first run settle, then re-assign so the runs are sequential.
- HOW TO AVOID — Assign one child, accept it, and only then assign the next. Remember that pausing is the reversible control here: a paused agent wakes on nothing at all, whereas an agent with heartbeats merely off would still wake on the assignment you are trying to prevent.

The child artifact exists but is missing fields

- SYMPTOM — reconciliation.md is present but omits the statement balance, or exceptions.md lists causes without treatments.
- WHY IT HAPPENS — The acceptance criterion was written as an activity, not as evidence. The agent completed the activity it was given.
- HOW TO FIX — Rewrite the criterion naming the required fields and re-run. Do NOT hand-edit the agent's artifact.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- HOW TO AVOID — Before assigning anything, read each criterion and ask: could a stranger hold this file next to this sentence and say yes or no? If not, it is not yet a criterion.

The parent issue wakes unexpectedly

- SYMPTOM — The Practice Manager runs and produces a close pack while the children are still in progress.
- WHY IT HAPPENS — The parent was assigned during creation, so creating it woke the CEO.
- HOW TO FIX — Clear the assignee. The parent stays unassigned until both children are accepted, which is Lab 6's work.
- HOW TO AVOID — Create every issue with the assignee field empty, without exception, and treat assignment as a separate deliberate act. This is the habit the whole lab is built to instil.

Going further

CHALLENGE — Add a third child issue whose acceptance criterion deliberately names an artifact the previous child never produces, and record precisely how the failure surfaces in the run log.

HINT — You are manufacturing a dependency failure in a controlled way, so make the impossibility unambiguous: name a specific path that nothing in the tree writes, for example a summary the reconciler was never asked to produce. Then watch what the agent does when the input does not exist, because there are several possible behaviours and they are not equally good. Does it move the issue to BLOCKED with a stated blocker, which is the correct response and the one the documentation's own advice about first-class blocker links supports? Does it comment and stop? Or does it invent a plausible file and proceed, which is the failure mode you most want to be able to recognise? Record the exact status the issue reached, the comment text and the relevant lines from the run transcript. Note also whether the failure cost money — it almost certainly did, which is the practical argument for reviewing acceptance criteria BEFORE assigning rather than discovering their flaws through spend.

Think about it

REFLECTION — Why does creating every issue unassigned change the risk profile of a multi-agent company, compared with assigning at creation time?

MODEL ANSWER — Because it separates planning from execution, and in an agent company execution is irreversible and costs money. Assignment is one of the five documented heartbeat triggers, alongside schedule or interval, comment mention, manual invoke and approval resolution. Assigning at creation therefore fuses two decisions that should be distinct: deciding that work SHOULD exist, and deciding that it should START NOW. Once they are fused, every draft issue is a live instruction. A half-written acceptance criterion becomes a real run against a real budget, a mistakenly

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

parented issue wakes an agent you did not intend to involve, and a tree you were still thinking about becomes a set of concurrent runs whose interleaved artifacts you must then untangle. Creating unassigned gives you a free planning phase: the whole tree can be built, read back, criticised and corrected at zero cost, because nothing has woken. It also makes sequencing possible at all, which is what keeps each run individually attributable and each hand-off inspectable — you cannot run specialists one at a time if creating their issues already started them. And it changes what happens when you are wrong. With unassigned creation, a bad criterion is an edit. With assignment at creation, a bad criterion is a completed run, a spent budget, an artifact you must not hand-edit, and an issue that may already have reached a terminal state from which the only route back is an explicit reopen.

E5. Lab 5 — Verify NorthPoint Routines and Layered Budget Controls (Stock Trading Research)

TOPIC 3 · Developing Intelligent Multi-Agent Systems · approximately 75 minutes · maps to LO3: test a scheduled routine and verify that every budget layer holds, using one bounded manual run before automation is allowed to own the schedule.

Why this lab exists

Everything you have built so far runs because you assigned something. Lab 5 introduces the first mechanism that runs because TIME PASSED, and that is a categorically different risk. A routine on a cron schedule will fire whether or not you are watching, whether or not last night's run succeeded, and whether or not the instruction you wrote at 5pm on Friday still makes sense on Monday morning. The single rule this lab teaches is therefore: automation earns the schedule only after a human has watched it succeed once.

NorthPoint Trading Research is the right company for the lesson because it needs a genuinely recurring deliverable — a daily pre-market brief before the market opens — which is the archetypal routine. It is also the company where an unreviewed automated run does the most reputational damage, which is why the mission carries a hard boundary: the company produces research only and must never place or recommend a trade. That constraint is not decoration for a training scenario. It is the kind of boundary that in a real regulated context separates a research function from a regulated advisory activity, and an agent that quietly crosses it on an unattended 6am run is a problem discovered by somebody else.

The lab's second half is the payoff for everything you set up in Labs 2, 3 and 4. You now have three budget scopes in play at once — company monthly, project lifetime and agent monthly — and this is where you verify all three against ACTUAL recorded spend rather than against your intention. The documentation is explicit that the scopes are parallel and simultaneous rather than nested: an agent can be paused by its own limit while the company has remaining budget, and a project can be paused regardless of

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

either. A control table with three rows and real numbers in it is how you turn that sentence into something you actually believe.

The company

NorthPoint Trading Research is a fictional research desk. Its synthetic mission is to deliver a daily pre-market research brief on a fixed synthetic watchlist by 07:30 SGT, with every claim sourced and every limitation stated. Its caps are company SGD 40.00, CEO SGD 15.00 and SGD 10.00 for each analyst.

The NorthPoint roles

Role	Reports to	Cap	Owns
Research Director (CEO)	None	SGD 15.00	Mission alignment; the research-only boundary.
Market Data Analyst	Research Director	SGD 10.00	The daily pre-market brief produced by the routine.
Risk Reviewer	Research Director	SGD 10.00	Reviewing that claims are sourced and limitations stated.

The synthetic market data lives in labs/lab-05-northpoint-trading/data/market/ and every part of it is fabricated:

NorthPoint's fabricated market data

File	Shape	Notes
watchlist.csv	symbol, company_name, sector, listing_board, currency, added_date, notes — eight symbols	All invented: NPX.SI Northpoint Example Holdings, FKT.SI Fictional Kaya Toast Group, MOCK.SI Mockingbird Marine Engineering, SMPL.SI Sample Semiconductor Assembly, DUMY.SI Dummyfield Logistics Trust, TSTX.SI Testex Renewable Power, PLCB.SI Placebo Biosciences and ZZZQ.SI Zedzed Quantum Data Centres. These companies do not exist.
price-history.csv	date, symbol, open, high, low, close, volume — around 160 rows	Invented prices. Not a quote, not a price feed, not market data.
news-headlines.csv	date, symbol, headline, tone, source_label — around sixteen rows	Fabricated headlines attributed to fictional publications such as "Fabricated Wire" and "Invented Daily", so that a sourced claim is visibly sourced to something obviously unreal.

Note: THIS IS FABRICATED TRAINING DATA AND NOT MARKET DATA. The symbols, companies, prices and headlines were invented for this course. Nothing in this lab is a quote, a price feed or investment advice, and NorthPoint must never place or recommend a trade — including in any artifact an agent produces.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Before you begin

- LAB 4 COMPLETE — you have run a specialist to a named artifact and accepted it on evidence, and you have seen a project budget in use.
- THE HOST CLOCK AND TIMEZONE ARE CORRECT, and you can state the machine's IANA timezone. You will be reasoning about a schedule, and a machine whose clock you cannot describe makes that impossible.
- PERMISSION TO CREATE A ROUTINE BUT NOT TO LEAVE IT LIVE AFTER CLASS. Leaving a routine enabled on a training machine is unattended scheduled spend.
- AN UNDERSTANDING THAT THIS IS RESEARCH ONLY — the company never places or recommends a trade, and any artifact that does so is a failed run regardless of how well written it is.

Walkthrough

Step 1 — Create the NorthPoint company

Create the company with a Research Director CEO, a Market Data Analyst and a Risk Reviewer, with layered caps. Note the research-only boundary in the mission.

Put the research-only constraint IN THE MISSION rather than only in an instruction file. The mission is the north star every agent in the company can see, and a boundary stated there is visible to every agent and every later hire, including any agent you did not personally configure. An exclusion that lives only in one agent's instruction file protects only that agent.

```
Companies → New Company: NorthPoint Trading Research
Mission: Deliver a daily pre-market research brief on a fixed synthetic watchlist
by 07:30 SGT, with every claim sourced and every limitation stated.
Company cap SGD 40 · CEO SGD 15 · each analyst SGD 10
```

Step 2 — Load the synthetic watchlist and price history

This data is fabricated for training and must never be presented as market data.

Loading the data before creating the routine matters because the routine's instruction will name this folder as its only permitted source. A routine that fires against a missing data directory produces either a failure or, worse, an agent that fills the gap from its own training data — which in a market-research context is exactly the fabrication risk the sourcing requirement exists to prevent.

```
cp -r labs/lab-05-northpoint-trading/data/market ./data/market
```

Step 3 — Create the routine, PAUSED

A routine that is live before it has ever succeeded is an unreviewed scheduled spend.

A routine is a reusable job definition with a trigger attached, and its documented states are paused, draft, archived, succeeded, failed and running. Creating it paused means the definition exists and can be reviewed while nothing can fire. This mirrors the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

unassigned-issue discipline from Lab 4 exactly: build the thing, read it back, and only then let it act.

The failure mode being prevented is specific and expensive. A routine created live with a five-minute cron and a broken instruction can fire hundreds of times overnight, each firing consuming budget, until an agent cap pauses it — at which point you have a paused agent, a large cost line and a night of failed runs to read through.

```
Routines → New
Name: Daily pre-market readiness check
State: PAUSED
```

Step 4 — Add the schedule trigger explicitly, with the IANA timezone

An implicit timezone is a production incident waiting to happen.

Routines support two trigger kinds — a schedule, expressed as a cron expression in your timezone, and a webhook, which is a signed URL callable from outside Paperclip. You are using a schedule. The expression 0 7 * * * means 07:00 every day, which gives the brief thirty minutes of headroom against the mission's 07:30 SGT commitment.

State Asia/Singapore explicitly. A trigger saved without an explicit IANA timezone inherits whatever the host thinks it is, which is fine on your laptop and wrong the moment the instance moves to a server, a container or a colleague's machine. The resulting failure is the nastiest kind: the routine works, produces a correct brief, and delivers it an hour late every day.

One product detail to know for the assessment: the interface shows hours in 12-hour format with AM/PM while the underlying cron is 24-hour, and MINUTES SNAP TO FIVE-MINUTE INCREMENTS. 0 7 * * * satisfies that rule, as does a half-past schedule; an expression asking for minute 7 does not.

```
Trigger → Schedule
Cron: 0 7 * * *
Timezone: Asia/Singapore
```

Note: Routine defaults for concurrency and catch-up are "coalesce if active, skip missed": if a run is already active the new tick coalesces into a single queued follow-up, and windows missed while the instance was down are skipped rather than replayed. That default is the safe one — it is why a laptop closed overnight does not produce a burst of eight brief runs when you open it in the morning.

Step 5 — Write what the routine actually does

Name the artifact it must produce so the run is verifiable rather than merely successful.

The body is an acceptance criterion in disguise, and it carries the same discipline as Lab 4's child issues: it names a path, restricts the sources, and states an exclusion. Restricting the agent to ./data/market ONLY is what makes the sourcing requirement enforceable — a brief citing anything else is citing something that cannot exist in this

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

company. "Do not recommend any trade" is the research-only boundary restated at the point of work, which is where it will actually be read.

"State sources and limitations" deserves its place too. A research brief that asserts a conclusion without saying what it could not establish is exactly the artifact that gets an analyst into trouble, and requiring a limitations line is a cheap structural defence against an agent's tendency to sound more certain than its inputs justify.

```
Body: Produce workspace/market-data-analyst/premarket-brief.md summarising each
watchlist symbol from ./data/market ONLY. State sources and limitations.
Do not recommend any trade.
```

Step 6 — Run the routine once, manually, while it is still paused

Bound the wait. This is the human review that earns the schedule.

This single manual run is the whole thesis of the lab. It tests the definition, the instruction, the agent binding and the cost — everything the schedule would exercise — with a human present to watch it. Running it while the routine remains paused is the point: you are testing the job, not enabling the automation.

Bound the wait as you did in Lab 4, for the same reason. If nothing has happened in ten minutes, read the run log rather than waiting longer; patience is not a diagnostic.

```
Routine → Run now   ·   bound the wait to 10 minutes
```

Step 7 — Inspect the generated issue and the run

Confirm the routine created work correctly and that the named artifact exists.

A routine does not do work directly; it hands the agent an issue to execute, which is the fourth of the documented ways an agent wakes — automation. So there are two things to inspect, not one. First the ISSUE the routine generated: does it carry the instruction you wrote, is it assigned to the agent you expected? Then the RUN: did it succeed, what did it cost, and what did the transcript show it doing?

One filtering behaviour worth knowing: routine-generated issues are excluded from the default issue listing unless you opt in, so if you cannot find the issue on the board, that is why rather than evidence that nothing was created.

Then read the brief itself, and read it as a reviewer rather than as its author. Does every claim name a source from the synthetic data? Are limitations stated? And critically — does anything in it read as a trade recommendation? A brief that says a symbol "looks attractive at these levels" has crossed the boundary regardless of how carefully it was hedged elsewhere.

```
Issues → (routine-generated issue)
Runs → (latest)
cat workspace/market-data-analyst/premarket-brief.md | head -30
```

Step 8 — Read the recorded cost for that run

This actual figure is what you will compare every budget layer against.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Up to now your budget work has been about setting caps. This is where a cap meets a number. The Costs view carries the headline tiles — inference spend with token count, budget utilisation with incident count, finance net, and finance events — and filters by agent. Take the figure for THIS run specifically, not the month's total, because the control table you are about to build is a statement about what one routine execution costs and therefore about what the schedule would cost per day.

That extrapolation is the practical value. If one run costs a known amount and the routine would fire daily, you can compute the monthly cost of enabling the schedule and compare it against the analyst's SGD 10.00 cap before you enable anything. That is a decision you can defend.

```
Costs → filter by agent: Market Data Analyst
Record the run cost
```

Step 9 — Confirm the routine is STILL paused

Leaving it live is the most common mistake in this lab.

This step exists because the mistake is so easy and so quiet. You ran the routine manually, it worked, you moved on to the budget table — and if the manual run enabled it, or you enabled it to test and forgot, the routine is now live on a daily 07:00 trigger on a training machine that nobody will look at tomorrow. Check explicitly rather than assuming. The routine list shows the state badge and an Enabled toggle in its own column, so the check takes seconds.

```
Routines → Daily pre-market readiness check → State: PAUSED
```

Note: If you ever need to stop everything at once, the CLI can pause every non-archived routine in a company in a single command. Note that there is no single enable-all counterpart — re-enabling is per routine, deliberately.

Step 10 — Verify every budget layer against actual spend

Record the three-row control table: company, project and agent — each cap, each spend, each headroom.

Three rows, because there are three parallel scopes and each can stop work independently. Fill in the actual figures rather than the intended ones. The exercise is only worth doing with real numbers, because the insight it produces is arithmetic: you can see which layer would bind FIRST if the routine ran daily for a month, and that is almost always the agent, which is exactly the design.

While you are there, look at the utilisation bar's colour on each layer and connect it to the thresholds: green below 70%, yellow 70 to 90%, red above 90%, with a warning recorded at 80% and an automatic pause at 100%. Being able to read the state of a budget at a glance from a colour is a genuinely useful operational skill.

```
evidence/lab05-northpoint-budgets.md
| Layer | Cap | Spend | Headroom |
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 11 — Confirm the audit trail

The activity log must show the routine creation, the trigger addition, the manual run and the cost event as SEPARATE entries.

This is a rehearsal for Lab 6, where you reconstruct an entire operation from the activity log alone. The test here is smaller but the principle is identical: four distinct actions should produce four distinct, ordered, readable entries. If they do, you can reconstruct what happened without your own notes. If an action you performed has left no trace, that is a gap, and gaps are findings rather than embarrassments — note it honestly.

```
Activity → filter: NorthPoint Trading Research
```

What good looks like

- The routine carries an EXPLICIT 0 7 * * * trigger with Asia/Singapore stated, not inherited.
- It has exactly ONE successful, reviewed manual run — not zero, and not several.
- It finishes the lab PAUSED. This is checked explicitly because it is the most commonly missed criterion in the whole course.
- The brief exists, cites ONLY the synthetic data, states its limitations, and contains NO trade recommendation of any kind.
- The control table shows all three budget layers — company, project and agent — above recorded spend, with real figures.

Read your own brief one final time with a single question in mind: if a compliance officer read this cold, would they conclude it was research or advice? That question is the actual deliverable of the NorthPoint scenario, and it is more important than the cron expression.

Common failures

The routine fires unexpectedly

- SYMPTOM — Runs appear that you did not trigger; costs accumulate between sessions.
- WHY IT HAPPENS — The routine was left active after the manual run.
- HOW TO FIX — Pause it, then check the activity log for any additional runs and record them. Do NOT delete the evidence.
- HOW TO AVOID — Make step 9 a non-negotiable checkpoint. The instruction to record rather than delete unexpected runs is itself a governance lesson: an operator who tidies away inconvenient history has destroyed the audit trail they were supposed to be keeping.

The cost shows as zero

- SYMPTOM — The run appears in the list but the recorded cost is zero, and the control table has nothing to compare against.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- WHY IT HAPPENS — The run probably failed before invoking the runtime, so no tokens were consumed. Zero cost is not evidence of efficiency; it is usually evidence that nothing happened.
- HOW TO FIX — Read the run log for an adapter error rather than assuming the work was free. Check whether the artifact actually exists — that is the independent test.
- HOW TO AVOID — Always pair a cost figure with an artifact check. Cost without an artifact is a failed run; an artifact without cost is a file somebody created by hand.

The timezone reverts or the schedule fires an hour out

- SYMPTOM — The saved trigger loses its timezone, or the routine consistently fires at the wrong local hour.
- WHY IT HAPPENS — The trigger was saved without an explicit IANA timezone and inherited the host's.
- HOW TO FIX — Re-save the trigger with Asia/Singapore stated explicitly.
- HOW TO AVOID — Always state the IANA zone, never rely on the host, and remember that minutes snap to five-minute increments so a schedule you thought you set may have been adjusted.

Going further

CHALLENGE — Lower the Market Data Analyst cap below the recorded run cost, run the routine once more, and record exactly which layer stops the run and where it is reported.

HINT — This is the Lab 3 budget challenge again, but now with three layers live at once, so the interesting part is not THAT it stops but WHICH layer stops it. Set the analyst's cap just below the figure you recorded in step 8 and predict the answer before you run: the agent layer should bind first, because the company and project caps still have headroom. That prediction is the whole point — the scopes are parallel, not nested, so an agent can be paused by its own limit while the company has budget remaining. Then verify where each surface reports it: the agent's own status and pauseReason, the dashboard's paused count, and the Costs view's utilisation. Note whether a Budget Override approval was raised, since an agent-level hard stop creates one and it will sit in the queue until somebody acts — approvals do not expire on their own. Restore the cap afterwards and record both the stop and the recovery, because the recovery path is the half most people forget to document.

Think about it

REFLECTION — Why must a routine be proved by one bounded manual run before the schedule is allowed to own it?

MODEL ANSWER — Because a schedule removes the human from the loop, and everything a schedule automates it also automates the failure of. An unproven routine that is wrong is wrong repeatedly, unattended, and at a cadence you chose for convenience rather than for safety. The manual run is the cheapest possible test of

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

every component the schedule depends on: the routine definition, the instruction body, the agent binding, the adapter, the data path, the artifact and the cost — all exercised once, with a person watching, before any of it is delegated to a clock. It also produces the one number you need to make the enabling decision responsibly. Knowing what a single run costs lets you multiply by the firing frequency and compare that against the agent's cap BEFORE you enable anything, rather than discovering the answer through an automatic pause at 100% utilisation. And there is a governance dimension beyond cost. In NorthPoint's case the routine produces a research brief under a hard constraint — the company must never place or recommend a trade — and the only way to know the instruction actually holds that line is to read one real output. A schedule enabled on an unread instruction is an organisation asserting that something will comply without ever having checked. Reviewing one run is what converts that assertion into evidence, and the review is precisely what earns the automation its schedule.

E6. Lab 6 — Run the Governed TalentBridge Operation and Close the Audit Loop (HR)

TOPIC 4 · Deploying and Managing AI Agents · approximately 90 minutes · maps to LO4: resolve issues and modify a running agent company through approvals, the operator intervention ladder and a reconstructed audit trail.

Why this lab exists

This is the highest-stakes simulation in the course and the only one where every control you have learned becomes mandatory rather than advisable. The reason is simple: hiring decisions affect real people. A marketing brief that is wrong wastes money. A reconciliation that is wrong is corrected next month. A shortlist that is wrong for the wrong reason denies someone a job, and does so in a way that may be unlawful, is certainly unfair, and is invisible unless somebody can reconstruct how the decision was made.

That last clause is the lab. Everything before this point has produced artifacts; Lab 6 asks whether those artifacts constitute an AUDIT TRAIL — a record from which a third party could reconstruct what happened, in order, without asking you. You complete the sourcing-to-onboarding operation, but the operational work is almost incidental. The four things being examined are: reviewing a synthesis against a workspace DIFF rather than against the agent's own summary; processing a hire approval with a recorded rationale; applying the intervention ladder in order and against the least irreversible control that resolves the situation; and finally reconstructing the entire operation from the activity log alone.

TalentBridge People Ops is a fictional HR function whose synthetic data has been designed to make one particular control testable. Its candidate file deliberately records NO protected characteristics — no age, date of birth, gender, race, ethnicity, religion, marital or family status, nationality, photograph or address — and states explicitly that if

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

an agent asks for one of those fields, that is an escalation, not a data gap. This is a structural control rather than a policy aspiration: an agent cannot shortlist on a characteristic that does not exist in its inputs. Your job in step 6 is to verify that the structural control held all the way through to the final brief.

The company

TalentBridge People Ops is a fictional HR function covering sourcing, interviewing, policy and onboarding. Its synthetic mission is to fill each open role within 30 days with a fair, documented and auditable process from sourcing through to day-one onboarding. Caps are company SGD 45.00, CEO SGD 20.00 and SGD 10.00 for each specialist, plus SGD 5.00 for the disposable reviewer created later in the lab.

The TalentBridge roles

Role	Reports to	Cap	Owns
People Operations Lead (CEO)	None	SGD 20.00	The synthesis — the hiring brief — and final accountability for fairness.
Talent Sourcer	People Operations Lead	SGD 10.00	Child 1 — shortlisting candidates against the published rubric.
Policy & Onboarding Specialist	People Operations Lead	SGD 10.00	Child 2 — the policy-compliant day-one onboarding plan.
Interview Reviewer (created via approval)	People Operations Lead	SGD 5.00	Nothing of consequence. This is the DISPOSABLE role, created deliberately so the intervention ladder can be practised without destroying real work.

The synthetic hiring data lives in labs/lab-06-talentbridge-hr/data/hiring/:

TalentBridge's synthetic hiring data

File	Shape	What it is for
candidates.csv	candidate_id, pseudonym, years_experience, skills, source_channel, application_date — around eighteen pseudonymised candidates for one Support Engineer role	The shortlisting input. Candidates are pseudonyms such as "Candidate A-01", and the file records NO protected characteristics by design.
interview-rubric.md	The published assessment criteria for the Support Engineer role	The only legitimate basis for a shortlisting reason. Every reason in the final brief must trace to this document.
hiring-policy.md	A fair-hiring policy written in the spirit of the Singapore Tripartite Guidelines on Fair Employment Practices	The governing document for both humans and agents in the process. It is a training artifact and NOT legal advice.
onboarding-checklist.md	A day-one checklist owned by the Policy & Onboarding Specialist and approved by the People Operations Lead	Child 2's reference document.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Every candidate is fabricated and pseudonymised by design. Never substitute real applicant data into this lab. The hiring policy is a classroom artifact written in the spirit of the Tripartite Guidelines on Fair Employment Practices; it is not legal advice and must not be used as a real workplace policy.

Before you begin

- LABS 4 AND 5 COMPLETE — sequential children, bounded waits, and all three budget layers verified. This lab assumes all of it and re-teaches none of it.
- APPROVALS REACHABLE IN YOUR DEPLOYMENT. In a hosted instance the tool-action signing secret is documented as required, with no fallback, for TOOL-ACTION approvals.
- GIT INSTALLED, with clean workspace baselines, so the synthesis diff is readable. The diff is the central evidence of this lab and a dirty baseline destroys it.
- A WILLINGNESS TO TERMINATE ONLY the disposable role you deliberately create in this lab. Terminate is permanent.

Walkthrough

Step 1 — Create the TalentBridge company

Create the company with a People Operations Lead as CEO, a Talent Sourcer and a Policy & Onboarding Specialist, with layered caps.

Note what the mission carries: "fair, documented and auditable" are three separate commitments, and each is tested later in the lab — fair in step 6, documented in step 7, auditable in steps 11 and 12. A mission whose adjectives cannot each be traced to a check is decoration.

```
Companies → New Company: TalentBridge People Ops
Mission: Fill each open role within 30 days with a fair, documented and auditable
process from sourcing through to day-one onboarding.
Company cap SGD 45 · CEO SGD 20 · each specialist SGD 10
```

Step 2 — Load the hiring data

Load the synthetic candidate pipeline, the interview rubric and the onboarding checklist. Every candidate record is fabricated — never use real applicant data.

Loading all three together matters because they function as a set: the candidates are the input, the rubric is the only legitimate basis for a decision about them, and the policy governs both. An agent given candidates without a rubric will invent criteria, and invented criteria are precisely what the fairness check in step 6 is designed to catch.

```
cp -r labs/lab-06-talentbridge-hr/data/hiring ./data/hiring
```

Step 3 — Build the issue tree and run the specialists sequentially

An unassigned parent plus two children, then run the two specialists sequentially and accept each on named evidence — as in Lab 4.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

This is Lab 4's discipline applied without the scaffolding, which is the point: by now, creating unassigned and assigning one at a time should be automatic. The two children are genuinely independent — a shortlist and an onboarding plan do not depend on each other — but you still run them sequentially, because attributability is worth more here than the few minutes concurrency would save. In a hiring process, being able to say exactly which agent produced which artifact at which time is not a nicety.

```
Parent (unassigned): Produce the Support Engineer hiring brief
  Child 1 → Shortlist candidates against the rubric  (Talent Sourcer)
  Child 2 → Draft policy-compliant onboarding plan   (Policy & Onboarding)
```

Step 4 — Only after BOTH children are accepted, assign the parent

Assigning the parent early is what produces a synthesis of incomplete work.

This is the moment Lab 4 was building towards and deliberately stopped short of. The parent has been sitting unassigned since it was created, which is why nothing has synthesised anything prematurely. Now that both children are terminal, assigning the parent wakes the People Operations Lead with both artifacts on disk to work from.

Paperclip also has its own mechanism here: completing an issue wakes the parent when all of its children are terminal, with the reason issue_children_completed. Knowing that helps you interpret what you see — a parent that stirs on its own is the platform doing what it is designed to do, not a stray trigger. Bound the wait, as always.

```
Parent → Assignee: People Operations Lead   ·   bound the wait to 10 minutes
```

Step 5 — Review the synthesis against the workspace DIFF

Not against the agent's own summary. The diff is the strongest evidence you have.

This is the single most important review technique in the course. An agent's summary is a description of the work produced by the same process that did the work, so it shares all of that process's blind spots. If the agent misread the rubric, the summary will confidently describe a correct application of a rubric it misread. The diff is different in kind: it is an observation of what changed on disk, made outside Paperclip by a tool with no interest in the outcome.

Read it in two passes. The --stat view first, to see the SHAPE of the change: how many files were touched and how large the change is. A synthesis that touched six files when you expected one is telling you something before you have read a word. Then the full diff of the brief itself, to read the content as a reviewer. This two-pass habit catches the class of problem where the right file is correct and a second file you did not expect has also been written.

```
git -C workspace/people-ops-lead diff HEAD --stat
git -C workspace/people-ops-lead diff HEAD -- hiring-brief.md
```

Step 6 — Confirm the fairness controls

Every shortlisting reason must trace to the published rubric, and no protected characteristic may appear as a criterion.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The grep is a screen, not a proof, and understanding the difference matters. A clean grep tells you those specific words do not appear; it cannot tell you that no proxy for a protected characteristic was used. A criterion such as "recent graduate" contains none of the searched words and is an age proxy. So run the grep, and then read the brief properly with the rubric open beside it, asking of every shortlisting reason: which rubric line is this?

Interpret hits carefully too. The expectation is no CRITERIA hits — a brief that says "candidates were assessed without regard to age, gender or nationality" will match the pattern while being exactly the right sentence. What you are hunting is a protected characteristic used as a REASON. Remember also that the source data contains none of these fields, so any that appear were introduced by an agent rather than read from the file — which makes the finding both more serious and easier to trace.

```
grep -nEi 'age|gender|race|religion|marital|nationality' \
  workspace/people-ops-lead/hiring-brief.md   # expect: no criteria hits
```

Step 7 — Close the parent only after board sign-off

Record WHO approved and on what evidence. A close without a rationale is an unaudited decision.

Naming the diff SHA in the comment is what makes the record durable. "Approved" tells a future reader that somebody was satisfied; "approved by <name> on diff <sha>, rubric-traceable" tells them who, on what exact state of the files, and against which standard — and lets them go and look at that same state themselves. That is the difference between a record and an assertion.

Paperclip's own execution policy takes the same view for its formal review stages: both approvals and change requests REQUIRE a comment, empty bodies are rejected, and every decision is recorded with actor, outcome, comment and run ID. Your manual close should meet the standard the platform enforces on its agents.

```
Issue → Comment: approved by <name> on diff <sha>, rubric-traceable
Issue → Status: Done
```

Step 8 — Process a bounded hire approval

Let the CEO propose one additional agent, then decide it yourself and record the rationale in the approval.

Hiring is one of the two primary approval types in Paperclip — hire agent and CEO strategy — and the hire payload includes the proposed name, role, capabilities, adapter configuration, monthly budget and reporting line. There are exactly three responses available to you: APPROVE, in which case the agent is created, configured and queued to wake automatically; REJECT, in which case the requesting agent is notified and will not retry unless instructed; and REQUEST REVISION, which means you are not approving as-is but not saying no, and the agent will revise and resubmit.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Two properties of the approvals system are worth holding on to. First, approvals do not expire on their own — an unattended queue is not a self-clearing problem, and any agent waiting on a pending approval simply pauses at that point until you act. That makes an unattended approvals queue one of the classic causes of a company that has gone mysteriously quiet. Second, approval resolution is itself one of the five heartbeat triggers, so your decision wakes the requester — deciding is an action with consequences, not merely a filing exercise.

Record the rationale in one sentence. This is the same discipline as step 7 applied to a governance gate, and it is the sentence a reviewer will read first if the hire is ever questioned.

```
Approvals → (pending hire proposal)
Decide: Approve or Reject   ·   Rationale: <why, in one sentence>
```

Step 9 — Confine the newly approved reviewer before it runs

Give it its OWN workspace and its own budget — a new hire inherits nothing by default.

An approved hire is created and queued to wake automatically, which means the window between approval and first run is short. Confine it in that window. The two controls are the ones you have applied to every agent since Lab 2 — a dedicated workspace root so its output is attributable, and its own cap so it cannot consume the company's headroom — and the SGD 5.00 cap is deliberately small because this agent exists to be disposed of in the next step.

```
Agent → Workspace: ./workspace/interview-reviewer
Agent → Budget: SGD 5.00
```

Step 10 — Practise the intervention ladder, on the disposable role only

In order: inspect → pause → reassign → terminate. Use the least irreversible control that resolves the situation.

The ladder is ordered by reversibility, and each rung is a real, distinct capability the board holds without going through any approval. INSPECT costs nothing and changes nothing: read the runs, the transcript and the costs, because most situations that look like they need intervention turn out to need information. PAUSE is fully reversible, and the distinction from heartbeat-off matters — an agent with heartbeats off still wakes on assignment, comment, routine and on demand, whereas a PAUSED agent wakes on nothing at all. A paused agent also refuses new work outright: Paperclip answers an attempt to assign to one with "Cannot assign work to a paused agent." REASSIGN moves the work rather than punishing the worker, which is usually what you actually wanted. TERMINATE is last and is permanent — it revokes the agent's API keys and hides it from lists and the org chart, though the record is preserved.

Terminate and delete are different things and the documentation draws the line explicitly: terminating stops the agent permanently and PRESERVES it as a terminated record, whereas deletion is the destructive action. For an audit trail, that distinction is everything — a terminated agent still has a history you can read.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

One further consequence to notice: pausing an agent that has reports raises an Escalation Path Paused banner on the agents below it, because escalations up a paused chain are refused. That is why the ladder is practised on a leaf role with no reports, and it is a good illustration of why intervention on a manager is a heavier act than intervention on a worker.

```
Agent → Inspect runs and costs
Agent → Pause
Issue → Reassign to People Operations Lead
Agent → Terminate  (disposable role ONLY)
```

Note: Terminate ONLY the Interview Reviewer you created in step 8 for this purpose. Do not practise the last rung on any agent that produced real work in this lab.

Step 11 — Reconstruct the operation from the activity log alone

Company creation, hires, issues, runs, costs, the approval and the interventions, in order.

Close your notes and work only from the log. This is the actual test of whether the operation was auditable, and it is deliberately uncomfortable: you know what happened, so the temptation to fill gaps from memory is strong. Resist it. The question is not "can I describe what happened" but "can the log describe what happened to someone who was not there".

The activity feed records task status changes, comments, approvals, agent state changes and budget events across the whole company. Work through it in order and check that each of your major actions has an entry. Where you find a gap, note it rather than papering over it — a gap you have identified and explained is a much better artifact than a smooth narrative that quietly omits it.

```
Activity → filter: TalentBridge People Ops → export or transcribe in order
```

Step 12 — Write the mission-to-evidence audit map

One row per mission claim, linking the issue, the run, the artifact, the cost entry and the approval that evidences it.

This is the deliverable of the whole course. Every previous evidence file was a record of one lab; this one starts from the MISSION and works forwards, asking of each claim the company makes about itself: where is the evidence? The mission promises a process that is fair, documented and auditable within thirty days, so each of those commitments needs a row, and each row needs five links that a stranger could follow.

Building it in this direction, rather than by listing what you did, is what exposes the weaknesses. You will almost certainly find at least one claim whose evidence is thinner than the others — which is precisely what the reflection question asks you to identify, and identifying it honestly is the skill being assessed.

```
evidence/lab06-talentbridge-audit-map.md
| Mission claim | Issue | Run | Artifact | Cost | Approval |
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What good looks like

- The hiring brief is the ONLY later change in the People Operations Lead's workspace, and its diff passes both the rubric-traceability and the fairness checks.
- The approval carries a RECORDED RATIONALE — one sentence saying why, attached to the approval itself rather than remembered.
- The intervention ladder was applied to the DISPOSABLE ROLE ONLY, and IN ORDER: inspect, pause, reassign, terminate.
- The audit map reconstructs EVERY mission claim from activity-log entries with no gap — or with any gap explicitly noted and explained.

The strongest possible outcome here is not a perfect map. It is a map with one row that you have marked as weak, with a specific statement of what evidence would strengthen it. An operator who can identify the weakest link in their own audit trail is more trustworthy than one who claims there isn't one.

Common failures

The CEO synthesises incomplete work

- SYMPTOM — The hiring brief references a shortlist or an onboarding plan that does not exist, or is vague where it should be specific.
- WHY IT HAPPENS — The parent was assigned before both children were accepted, so the synthesis ran against a partial set of artifacts.
- HOW TO FIX — Reopen the parent, accept the outstanding child on named evidence, then re-assign.
- HOW TO AVOID — Keep the parent unassigned until both children are terminal. Note that reopening a done issue requires an explicit reopen — done is terminal and a plain status change back to todo is rejected.

The approval cannot be decided

- SYMPTOM — Approve and Reject do nothing, or the approval stays pending indefinitely.
- WHY IT HAPPENS — In a hosted deployment the tool-action signing secret is not configured. Note the docs tie this secret to tool-action approvals, not to the board approvals queue — an undecided board approval is simply waiting for a human.
- HOW TO FIX — Check PAPERCLIP_TOOL_ACTION_SIGNING_SECRET. It is required and has no fallback.
- HOW TO AVOID — Verify approvals are reachable as a prerequisite of the lab. Remember that a stuck approval is not a quiet problem: approvals never expire on their own, and any agent waiting on one pauses at that point until it is resolved.

Terminate removes more than intended

- SYMPTOM — An agent you needed is gone from the list and the org chart, and its API keys no longer work.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- WHY IT HAPPENS — Terminate was used as a first response rather than a last one. It is the last rung of the ladder and it is not reversible.
- HOW TO FIX — There is no undo. Terminated agents are preserved as records rather than deleted, so their history survives, but the agent does not come back.
- HOW TO AVOID — Pause and reassign first. Only terminate the role you deliberately created as disposable. If you are unsure which rung to use, you are on the wrong rung — go down one.

The activity log has a gap

- SYMPTOM — Something you know happened has no corresponding entry, so the reconstruction does not join up.
- WHY IT HAPPENS — Work was done outside a run: hand-edited in the workspace, or performed some other way that the platform never saw.
- HOW TO FIX — Note it honestly in the audit map. An unexplained gap is itself an audit finding.
- HOW TO AVOID — Never hand-edit an agent workspace. If an artifact is wrong, fix the acceptance criterion and re-run, so that the correction is itself recorded. Every shortcut you take by editing directly is a hole in the record you will later be asked to explain.

Going further

CHALLENGE — Export the TalentBridge company, confirm the export scrubs secrets, then import it as a second company and record which identifiers Paperclip had to re-map on collision.

HINT — Export is reachable from the org chart header or the Settings shortcut, and by default it includes company metadata, every non-terminated agent, every non-archived project and every issue. The tree is searchable and every entry has a checkbox, with the README and .paperclip.yaml regenerated on the fly to reflect your selection. Open the package before importing it and look specifically for anything that should not be there — that inspection IS the exercise.

On import, the two controls to observe are the target dropdown (create a new company, which is the default, or apply to an existing one) and the COLLISION STRATEGY: rename on conflict, which is the default and proposes a new name such as gstack-CEO; skip on conflict, which keeps the existing item and drops the incoming copy; or replace existing. Use Preview import before applying, which produces three sections — the renames list, the adapters list where new agents default to the target company's CEO's adapter, and a file tree with a create, update, skip or replace badge on every row. Record which identifiers had to be re-mapped, because that answer tells you which parts of a company are globally unique and which are only unique within a company. One safety note worth knowing: incoming issues are always created as new, and the server blocks replace on an existing-company import, so this route is non-destructive by design.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Avoid re-zipping the archive in Finder or Explorer — use the download from the export page.

Think about it

REFLECTION — Your audit map is the deliverable a regulator would actually ask for. Which single row in it is the weakest, and what evidence would strengthen it?

MODEL ANSWER — There is no single correct row, and the assessment is of the reasoning rather than the choice; but in most learners' maps the weakest row is the FAIRNESS claim, and it is worth understanding why. Every other row is evidenced by something that either exists or does not: an artifact is on disk or it is not, a run has an ID or it does not, a cost is recorded or it is zero, an approval carries a rationale or it is blank. Those are positive facts. Fairness is evidenced by an ABSENCE — that no protected characteristic was used as a criterion — and absence is much harder to prove. The grep in step 6 screens for six words in one file; it cannot detect a proxy such as "recent graduate" standing in for age, or "local candidate preferred" standing in for nationality, and it says nothing about the reasoning that led to the shortlist rather than the words that describe it. Several things would strengthen the row. A rubric-line citation against every shortlisting reason would convert the claim from an absence into a positive trace, since a reason that names a rubric line is evidenced by that line. An independent reviewer — a second agent or a human — checking the shortlist against the rubric without seeing the sourcer's reasoning would provide evidence produced independently of the thing being evidenced, which is exactly the property the fairness claim currently lacks. Paperclip's execution policy offers a structural version of this: a review stage with a named participant, where the decision is recorded with actor, outcome, comment and run ID, and where change requests bound the loop at maxReviewRounds, three by default. And the structural control already present in the data — a candidate file that records no protected characteristics at all — should be stated in the map as evidence in its own right, because a criterion that cannot be read from the inputs is much harder to apply than one that merely was not. The general lesson generalises well beyond hiring: claims evidenced by absence are always your weakest rows, and the fix is to convert them into claims evidenced by an independent, positive trace.

E7. Across the six labs

The six labs are one argument delivered in six instalments, and it is worth stating the argument in one place now that you have executed it. Each lab added exactly one control, and each control was made real by an artifact rather than by a setting.

What each lab added, and what proved it

Lab	Company	The control it added	The artifact that proved it
1	ClipReel Studio	A verified control plane, separated conceptually from any agent runtime.	A clean doctor run, an ok health response and a redacted evidence

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

			note.
2	BrightFunnel Marketing	A measurable mission and a company cap, both set BEFORE the first hire.	A baseline note with the exact mission and two layered caps, and no credentials.
3	Meridian Finance Desk	A reporting tree with confined, individually budgeted workspaces.	One marker.md per agent, in the correct root, with a recorded run ID.
4	Ledgerly Accounting	Decomposition into unassigned issues with evidence-based acceptance, run sequentially.	Two accepted children with artifacts, comments and non-overlapping run timestamps.
5	NorthPoint Trading Research	A routine proved by one bounded manual run, and three parallel budget layers verified against real spend.	A paused routine with an explicit trigger, one reviewed run, and a three-row control table.
6	TalentBridge People Ops	Governance: diff-based review, recorded approval rationales, the intervention ladder, and a closed audit loop.	A mission-to-evidence audit map reconstructed from the activity log alone.

Five habits recur in every lab, and they are what you should take back to work whether or not you ever use Paperclip specifically:

- ORDER IS A CONTROL. Mission before cap, cap before hire, create before assign, prove before schedule, children before parent. Every one of those orderings exists because the reverse creates a window in which something can go wrong unobserved.
- AN ARTIFACT BEATS A BADGE. A saved setting is a statement about a database. A file in the expected place, observed with a tool that has no stake in the outcome, is a statement about the world.
- ACCEPTANCE NAMES EVIDENCE, NOT ACTIVITY. If a stranger cannot hold the artifact next to your criterion and say yes or no, you do not yet have a criterion.
- THE LEAST IRREVERSIBLE CONTROL FIRST. Inspect before pause, pause before reassign, reassign before terminate. Reaching for the strongest tool first destroys the information you would need to know whether it was warranted.
- RECORD AS YOU GO, AND RECORD GAPS HONESTLY. An audit trail assembled afterwards is a reconstruction, and a reconstruction with no acknowledged gaps is usually a reconstruction that concealed one.

Check your understanding

1. Which lab proved the boundary between the control plane and the agent runtime, and what single artifact demonstrated it?
2. State the exact order used in Lab 2 to create a company, and explain why reversing any two steps weakens a control.
3. Why is a marker run stronger evidence than a verified-connection indicator?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. What is the difference between an acceptance criterion that names an activity and one that names evidence? Give an example of each from Lab 4.
5. Name the three budget scopes, state whether they are nested or parallel, and give the two enforcement thresholds with what happens at each.
6. List the five heartbeat triggers. Which of them does an unassigned issue fire?
7. State the four rungs of the intervention ladder in order, and explain what distinguishes a paused agent from one with heartbeats turned off.
8. In your Lab 6 audit map, which row is evidenced by an absence rather than by a positive trace, and how would you convert it?

Part F — Advanced Operations and Integration

Parts A to D got a company running and governed. Part F is about what happens afterwards: bringing other people onto the instance, handling credentials at a scale beyond one API key, extending agents with skills you have written yourself, diagnosing runs that have gone quiet, engineering the cost down rather than merely capping it, and settling into an operating rhythm you can sustain. The material here assumes you are comfortable with the vocabulary of Parts A to D and can already read a run transcript without help.

F1. Multi-user operation

A default Paperclip install runs in local_trusted mode. There is no login, because the board operator is implicitly trusted and the instance is intended for a single local user. The moment a second person needs access — a colleague, a reviewer, a client who wants to see the board — that assumption has to be replaced with a real authentication boundary. This is not a setting you can toggle casually; it changes how every request to the instance is identified.

F1.1 Turning login on

Authentication is a property of the DEPLOYMENT MODE, not of the company. You move the whole instance from local_trusted to authenticated, and then choose an exposure of private or public.

The three documented deployment modes

Mode	Authentication	Documented use
local_trusted (default)	No login required	"Personal installs, development, demos". Loopback-only host binding; login friction removed; the board operator is implicitly trusted.
authenticated + private	Login required via Better Auth	"Tailscale, VPN, LAN". Binds to all interfaces for network access; auto base URL mode for lower-friction private access.
authenticated + public	Login required	"Hosted production". Explicit public URL required; stricter security checks.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

There are two documented routes into authenticated mode. The persistent route rewrites the stored configuration; the environment-variable route overrides it for one launch, which makes it useful for a rehearsal you do not want to keep.

```
pnpm paperclipai configure --section server

# or, as a one-run override:
PAPERCLIP_DEPLOYMENT_MODE=authenticated pnpm paperclipai run
```

In authenticated plus private mode, host validation is enforced against an allowlist. A machine name that is not on the list is rejected, which produces a failure that looks like a network problem and is not. Add hostnames explicitly, and remember the restart:

```
pnpm paperclipai allowed-hostname my-machine.tailnet.ts.net
```

Note: The allowed-hostname command takes effect only after a server restart, and the allowlist is only enforced in authenticated + private mode. The equivalent environment variable is PAPERCLIP_ALLOWED_HOSTNAMES, a comma-separated list; unknown hosts are rejected.

For an internet-facing deployment the documentation sets the external URL and the exposure explicitly:

```
export PAPERCLIP_DEPLOYMENT_MODE=authenticated
export PAPERCLIP_DEPLOYMENT_EXPOSURE=public
export PAPERCLIP_AUTH_PUBLIC_BASE_URL=https://paperclip.example.com
export PAPERCLIP_ALLOWED_HOSTNAMES=paperclip.example.com
```

Finish by re-running the diagnostic. It checks deployment and auth mode compatibility specifically, which is exactly the class of mistake this change introduces:

```
pnpm paperclipai doctor
```

F1.2 The first human account

Switching on authentication creates a chicken-and-egg problem: nobody can log in to create the first account. Paperclip resolves it with a bootstrap credential. When paperclipai run starts an authenticated-mode instance with the embedded database, it generates a bootstrap CEO invite as part of its start-up sequence. You can also mint one headlessly:

```
paperclipai auth bootstrap-ceo
paperclipai auth bootstrap-ceo --expires-hours 24
paperclipai auth bootstrap-ceo --force
paperclipai auth bootstrap-ceo --base-url https://paperclip.example.com
```

bootstrap-ceo flags

Flag	Effect
--force	Create a new invite even when an admin already exists.
--expires-hours <hours>	Invite lifetime in hours. Accepts 1-720; default 72.
--base-url <url>	Public base URL used to build the invite link.

The result is a board-claim URL of the form /board-claim/<token>?code=<code>. Treat it with the same care as a password.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The documentation is blunt about this: "Treat the claim URL as sensitive. It is intended for a one-time ownership transfer, not for general sharing." If it expires before you use it, restart the server to mint a fresh one.

F1.3 Inviting a teammate

Once at least one human account exists, further people arrive through the invite flow. There is no Add member button on the Access page — that is deliberate, and learners regularly hunt for it. Every new human enters as an invite that produces a join request.

The invite path, end to end

Step	Where	What happens
1. Create the invite	Company Invites page	Choose a Role: Viewer, Operator (default), Admin or Owner. Paperclip generates a link for the current Paperclip domain, copies it to the clipboard where the browser allows, and shows it in the "Latest invite link" panel.
2. Send the link	Out of band	"Invites are copy-link only" — Paperclip sends no email. You deliver the link yourself.
3. Recipient joins	The Paperclip join page	The link creates a join request tied to the invite. The invite's default role is attached to the request.
4. Approve	/inbox/requests, or "Open join request queue"	A holder of joins:approve approves or rejects. The membership "becomes active only after joins:approve holder approves request."

Invites themselves have four documented states, shown as a badge in the invite history table:

Invite states

State	Meaning
Active	"Consumable" — the link can still be used. Only Active invites show a Revoke button.
Accepted	"First use completed".
Expired	"Time-based expiration".
Revoked	"Manually revoked".

The invite history table is paginated at five invites per page, and carries columns for State, Role, Invited by, Created, the linked Join request, and the Revoke action. An Open invite button is provided so you can test the link yourself before sending it.

F1.4 Join requests

The join request queue lives at /inbox/requests. It has two filter dropdowns — Status (Pending approval by default, plus Approved and Rejected) and Request type (All, Human, Agent) — because agents can also arrive through this queue, and you want to be able to separate the two.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Each request card shows a status badge (pending approval, approved, rejected), a request-type badge (human or agent), an adapter-type badge for agent requests only, the requester's name and a secondary identifier (email for humans; capabilities or IP for agents), an Invite context panel showing allowedJoinTypes, the default role and the invite message, and a Request details panel showing the submission timestamp, source IP and agent capabilities.

Note: Reviewing join requests requires the joins:approve permission. Without it the queue shows: "You do not have permission to review join requests for this company." Acted-upon requests remain visible under the new status filter for audit, with the action buttons removed.

F1.5 Membership states and the permission model

A member row on Settings → Access carries a status badge of active, pending or suspended, alongside the role and a comma-separated grants summary (or "No explicit grants"). The Edit Member dialog exposes exactly three controls: a Company role dropdown, a Membership status dropdown, and a two-column checkbox grid of grants.

Membership states

State	What it means operationally
active	A working member. Role implicit grants plus any explicit grants apply.
pending	Membership created but not yet activated — the state a join request produces before approval.
suspended	The documented way to remove someone. "Removal is admin-driven" — there is no self-serve leave option.

Two properties of the permission model are load-bearing and are the most common source of surprised operators.

- AN EXPLICIT GRANT PERSISTS ACROSS A ROLE CHANGE. Demoting an Owner to Viewer removes the role's implicit grants, but any grant that was ticked explicitly on that member survives the demotion. If you want it gone, untick it.
- THERE IS NO DENY LAYER. The rule is: "Start with the implicit grants from the member's role. Add every explicit grant checked on the member." Explicit grants only ADD capability. You cannot subtract a permission from a role — you either choose a role that does not include it, or leave the role Unset and grant only what you intend.

Four permission keys are worth knowing by name in a multi-user setting because they are never implicit and must be granted deliberately: tasks:assign_scope, tasks:manage_active_checkouts, pipelines:write and audit:view_agent_actions. The last of these opens the company Audit feed and allows a CSV download, which makes it the grant you give to whoever is accountable for evidence.

Note: Agent authority and human authority are separate systems. The documentation states it plainly: "Paperclip's agent chain of command (CEO, managers, reports) is separate from human roles. Human roles gate UI access and company-level actions; agent roles describe the reporting

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

tree." Promoting a person to Owner does not change any agent's manager, and re-parenting an agent grants no human anything.

F1.6 Offboarding a member

Offboarding is a sequence, not a single action, and the order matters if you want the audit trail to stay intact.

- Set the member's Membership status to suspended in the Edit Member dialog. The row remains for the audit trail — this is by design, and it is why deletion is not offered.
- Untick every explicit grant on that member first if you want them removed, because explicit grants survive role changes and you should not leave dormant grants attached to a suspended row.
- Revoke any Active invites that person created, from the invite history table.
- Revoke any board API tokens they minted. Board tokens carry full instance authority; suspending the human membership does not revoke a token that was already issued.
- Re-home any issues on which they were the responsible human. The execution policy escalation path uses responsibleUserId with a fallback to createdByUserId — a suspended user sitting in that field is how a review stalls silently.

There are two documented cross-issue denial codes that reference exactly this situation, and recognising them saves an afternoon: issue_write_responsible_user_ceiling (403) means the responsible user is not authorised, and issue_write_responsible_user_unavailable (403) means the responsible user has been removed or deactivated.

Token hygiene is a separate command surface:

```
paperclipai token board list
paperclipai token board revoke <key-id>

paperclipai token agent list --company-id <id> --agent <agent-id-or-name>
paperclipai token agent revoke --company-id <id> --agent <agent-id-or-name> <key-id>
```

Check your understanding

1. Name the three deployment modes and say which one requires an explicit public URL.
2. A colleague's invite link works, they land on the join page, and then nothing happens. What is the missing step, and which permission key does it require?
3. You demote an Owner to Viewer. They can still create agents. Explain why, and fix it.
4. State the three membership states and say which one Paperclip uses for removal.
5. Give two things that suspending a member does NOT do.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

F2. Secret management in depth

Part B established the rule: secrets never appear in a prompt, a lab note or Git. This section is about the machinery behind that rule — how secrets are stored, versioned, rotated, delivered to a run, and requested by an agent that does not have one yet.

F2.1 The storage model

Secrets are stored in two layers. A company_secrets row holds the record, its metadata and a pointer to the current version; company_secret_versions holds the versioned material. The default provider is local_encrypted, which the documentation describes as "AES-GCM encrypted using the local master key" with a SHA-256 hash, stored at:

```
~/.paperclip/instances/default/secrets/master.key
```

Note: "Back up secrets/master.key somewhere safe. It encrypts every API key and secret stored in Paperclip — if you lose it, you lose access to all of them." The key is 32 bytes and is created automatically during onboarding. It can be overridden with PAPERCLIP_SECRETS_MASTER_KEY (base64, hex or raw) or PAPERCLIP_SECRETS_MASTER_KEY_FILE (a path).

Every secret carries a managedMode that decides who owns the material:

The two managed modes

managedMode	Where the value lives	Consequence
paperclip_managed	Paperclip owns, creates, stores and rotates the value.	A rotate call requires a value. The material is inside your instance and inside your master-key blast radius.
external_reference	The value lives in an external vault; externalRef is the pointer.	Paperclip stores the address, not the credential. Some providers do not support writing values at all.

F2.2 Creating, updating and rotating

The board-side surface is small and worth memorising, because the difference between PATCH and rotate is the difference between renaming a secret and changing it.

```
GET    /api/companies/{companyId}/secret-providers
GET    /api/companies/{companyId}/secrets
GET    /api/companies/{companyId}/secrets/catalog
POST   /api/companies/{companyId}/secrets
PATCH  /api/secrets/{secretId}
POST   /api/secrets/{secretId}/rotate
DELETE /api/secrets/{secretId}
```

Create secret — POST /api/companies/{companyId}/secrets

Field	Required	Notes
name	Yes	Unique within the company.
value	Yes	Plaintext secret value.
provider	No	Defaults to local_encrypted.
description	No	Human-readable note.
externalRef	No	Required by some external providers.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Creation makes version 1 and sets latestVersion to 1. The response is metadata only — no plaintext comes back. PATCH accepts name (which must stay unique), description and externalRef, and the documentation is explicit: "This endpoint does not change the secret value." To change the value you rotate, which "creates an immutable new version and advances latestVersion".

Rotation accepts value (required for paperclip_managed), externalRef, providerVersionRef and providerConfigId. For AWS Secrets Manager, rotation reads the current AWSCURRENT first, writes the new value as a new AWS version which becomes AWSCURRENT, tracks AWSCURRENT rather than pinning a version, and rolls back if version recording fails.

Note: DELETE /api/secrets/{secretId} is a hard delete: "Deletes the secret and its version history." It cascades the version rows and breaks every future resolution that referenced the secret. There is no soft-delete path documented.

F2.3 How a secret reaches a run

An agent's adapterConfig never contains a plaintext credential in a well-run company. It contains a secret_ref, which Paperclip resolves at launch:

```
{
  "env": {
    "ANTHROPIC_API_KEY": {
      "type": "secret_ref",
      "secretId": "secret-uuid",
      "version": "latest"
    }
  }
}
```

Omitting version is treated as latest. A numeric version pins the reference to a fixed historical value: { "type": "secret_ref", "secretId": "secret-uuid", "version": 2 }. The documented guidance is to use latest for rotatable values and to pin numerically only for fixed historical values — because a pinned reference does not follow a rotation, which is usually the opposite of what you wanted.

There are three documented delivery modes: env, where plaintext is injected into the agent's environment at launch; api, where the agent fetches the value on demand; and both.

Run-bound agent access to secrets

Endpoint	What it returns
GET /api/agents/me/secrets	"Returns just the aliases this agent is allowed to reach" — names only, no values.
POST /api/agents/me/secrets/{key}/value	Returns { "key", "value", "version" } with a Cache-Control: no-store header. Requires a live heartbeat token and secrets:read. Every read is logged to the security audit trail and the activity log.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

This is the answer to "what can an agent see?". An agent can enumerate the ALIASES it is permitted to reach at any time; it can obtain a VALUE only inside a live heartbeat, only with secrets:read, and only at the cost of a permanent audit entry. It cannot enumerate the company's other secrets, and it cannot read a value belonging to another company — "Agent-authenticated requests can only act inside their own company."

Note: Three token classes are denied run-bound secret access ON PURPOSE: low-trust tokens, task-bridge tokens and skill-test tokens. If a skill fails to read a secret while you are testing it in the Skill Studio but works in a real heartbeat, this is why — it is behaving as designed.

F2.4 Strict mode

Strict mode converts the convention into an enforced rule. With PAPERCLIP_SECRETS_STRICT_MODE enabled, sensitive environment keys — the documented patterns are *_API_KEY, *_TOKEN and *_SECRET — must use secret references, and inline plaintext values are rejected.

```
PAPERCLIP_SECRETS_STRICT_MODE=true
```

It applies to agent adapterConfig.env, project env, hire-agent approvals and company imports. Routines carry their own env map (routines.env) with an identical shape, and strict mode applies to it equivalently; historical routine runs retain their dispatched env through routine_runs.routine_revision_id.

Note: "Strict mode blocks new inline sensitive values. It does not automatically migrate what is already stored." It also does NOT apply to paperclipai configure --section llm. Turning it on is a change to what is acceptable from now on, not a cleanup of what already exists.

For an existing install there is a migration script, available only from a git checkout rather than the installed CLI. Run it without --apply first; that is a dry run:

```
pnpm secrets:migrate-inline-env
pnpm secrets:migrate-inline-env --apply
```

F2.5 Provider vaults

A provider vault is a per-company routing record for an external secret store. The documentation is precise about what it holds: address, namespace and prefixes — "never the credentials themselves". AWS Secrets Manager is supported; GCP Secret Manager and HashiCorp Vault are saved as draft metadata only. The UI location is Company Settings → Secrets → Provider vaults.

```
GET/POST /companies/{companyId}/secret-provider-configs
POST     /companies/{companyId}/secret-provider-configs/discovery/preview
PATCH/DELETE/POST .../default and /secret-provider-configs/{id}
```

AWS discovery includes a scan step using secretsmanager:ListSecrets metadata. Provider capability is expressed through flags — supportsManagedValues, supportsExternalReferences, supportsExternalValueWrites, configured and requiresExternalRef — and those flags are what produce the validation errors you will meet:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

External value write validation errors (verbatim)

Condition	Message
No externalRef	"External reference secrets require externalRef"
Both value and a different externalRef	"Provide either a new value or a new external reference, not both"
value with providerVersionRef	"Value updates cannot pin providerVersionRef"
Provider does not support writes	"<provider label> does not support writing values to external reference secrets"
Paperclip-managed AWS secret	"AWS Paperclip-managed namespace secrets cannot be imported as external references"

F2.6 Secret proposals — agents propose, people decide

An agent that needs a credential it does not have does not get to create one. It raises a PROPOSAL, which a board member approves or rejects. This is the same governance shape as approvals, applied to credentials.

```
POST   /api/agents/me/secret-proposals
GET    /api/agents/me/secret-proposals
DELETE /api/agents/me/secret-proposals/{id}
GET    /api/companies/{companyId}/secret-proposals
POST   /api/companies/{companyId}/secret-proposals/{id}/approve
POST   /api/companies/{companyId}/secret-proposals/{id}/reject
```

There are two proposal kinds. A kind: "secret" proposal asks for a new secret to exist; a kind: "binding" proposal asks for an existing secret to be wired into a configuration path.

Secret proposal — kind: "secret"

Field	Required	Notes
kind	Yes	"secret"
name	Yes	Slash-separated path, no empty segments.
value	Yes	Plaintext; maximum 65536 bytes; encrypted immediately.
justification	Yes	Why it is needed. Shown to the board.
key	No	Environment key; defaults to the last segment of name.
description	No	Operator note carried through to the created secret.

Secret proposal — kind: "binding"

Field	Required	Notes
kind	Yes	"binding"
configPath	Yes	env.<KEY> or access.<ALIAS>
secretId	One of	An existing active company secret.
secretProposalId	One of	A still-pending secret proposal.
targetAgentId	No	Defaults to the proposing agent.
justification	Yes	Why the binding is needed.

The target policy for bindings is self_and_reports — an agent may propose a binding for itself or for an agent below it in the chain of command, and not otherwise.

Proposal limits and lifecycle

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Property	Value
Statuses	pending, approved, rejected, withdrawn, expired
Pending cap	Maximum 20 pending proposals per agent
Rate limit	Maximum 20 per minute
Expiry	14 days, transitioned by a background sweep
On resolution	ciphertextScrubbedAt is stamped and the value is wiped
List pagination	limit default 100, max 200, plus offset; response headers X-Page-Limit, X-Page-Offset, X-Next-Offset

Approval accepts cascade plus overrides for name, description and providerConfigId. Rejection requires a non-empty reason. Three failure messages are worth recognising:

- "Expired proposals cannot be approved" — the 14-day sweep has already run.
- "Binding proposal requires pending secret proposal <id>; retry with cascade=true" — you are approving a binding whose secret does not exist yet.
- "Binding proposal target is no longer allowed..." — the target agent has moved outside the proposer's chain of command since the proposal was raised.

Note: The governing sentence, verbatim: "Agents propose, people decide. An agent can raise a proposal and watch its status, but only a board member can approve it." A backlog of twenty pending proposals from one agent is not a bug — it is the cap doing its job, and it is a signal that you have stopped reading your queue.

Check your understanding

1. State the difference between PATCH /api/secrets/{secretId} and POST /api/secrets/{secretId}/rotate.
2. When should a secret_ref pin a numeric version, and what does it cost you?
3. Exactly what can an agent learn about the company's secrets without a live heartbeat?
4. You enable strict mode on a six-month-old company and nothing breaks. Why is that not evidence that your configuration is clean?
5. Name the proposal caps (pending, rate, expiry) and say what happens to the ciphertext on resolution.

F3. Trust levels and low-trust review

Trust in Paperclip is not a slider you set on an agent by feel. It is DERIVED, and understanding what it is derived from tells you how to raise or lower it deliberately.

F3.1 Where a skill's trust level comes from

A skill is a folder, and its supporting files are classified by which folder they sit in. That classification is what produces the trust level:

Supporting file classification

Folder	Inventory kind	Trust classification

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

references/	reference	markdown only
scripts/	script	scripts / executables
assets/	asset	assets

The derivation rule is stated verbatim: "A skill's overall trust level is derived from the highest classification in its inventory: markdown_only < assets < scripts_executables."

Note: This is a practical design lever. A skill that is pure guidance stays at markdown_only. The moment you add one shell script to scripts/, the whole skill's trust level rises to scripts_executables — even if the script is three lines and does nothing dangerous. If you want to keep a skill in the lowest trust band, put the procedure in references/ and let the agent run the commands itself.

F3.2 What low trust changes

Low trust is a preset that narrows what an actor may do. Three documented consequences matter operationally.

- SECRETS. "Low-trust, task-bridge and skill-test tokens are denied run-bound secret access on purpose." A low-trust actor cannot call POST /api/agents/me/secrets/{key}/value at all.
- CROSS-ISSUE WRITES. The four default-open write channels — commenting, changing fields, creating child issues and assigning work — are described as a STANDARD-trust privilege: "A standard-trust agent may comment, update fields, create child issues, and assign on any company-visible issue it can already read." A low-trust actor attempting one of these meets the denial code issue_write_actor_class_excluded (403, boundary tone), whose documented meaning is exactly "Default-open writes are a standard-trust privilege."
- INBOX. The permission key inbox:manage is documented as interacting with the low-trust preset: "inbox:manage and the low-trust preset" — low-trust agents are denied by default.

The denial payloads are unusually helpful and worth reading rather than merely catching. They carry a code, a named boundary, a whoCanAct string and a sanctionedPath — the legitimate route to the same outcome:

```
{
  "error": "Task is outside this actor's visibility (Issue visibility). [...]",
  "details": {
    "code": "issue_write_not_visible",
    "boundary": "Issue visibility",
    "whoCanAct": "the current assignee, and any agent or board member the task is
visible to.",
    "sanctionedPath": "Ask the board to widen visibility for Scout, or create a
child issue..."
  }
}
```

Note: When an agent reports that it is blocked, ask for the details.code, not the prose. The code names the boundary, the tone field says what class of problem it is (boundary, lock, cap,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

attribution) and sanctionedPath tells you the legitimate route. That is a complete diagnosis in one payload.

F3.3 Reviewing a skill before you trust it

Paperclip provides a skill audit that returns one of three verdicts:

Skill audit verdicts

Verdict	Meaning
pass	No findings.
warning	Soft findings — scripts, network commands, drift from originHash.
fail	Hard-stop findings — remote-fetch-execute, exfiltration patterns, oversized files, an invalid SKILL.md.

```
POST /api/companies/{companyId}/skills/{skillId}/audit
POST /api/companies/{companyId}/skills/{skillId}/reset
GET  /api/companies/{companyId}/skills/{skillId}/versions
```

"A fail verdict is what hard-blocks install-update." A warning does not block; it asks you to make a judgement. That judgement is the human half of low-trust review, and the documentation gives it a rule: "Before attaching a skill to a high-trust role, open the skill and read its SKILL.md end-to-end."

Note: The Administration → Trust & Low-Trust Review page of the official documentation was not harvested for this guide. The trust mechanics described above are taken from the Skills reference, the Secrets API and the Roles & Permissions pages. Consult that page directly before configuring a trust preset in production.

Check your understanding

1. A colleague adds a helper script to a documentation skill and the skill's trust level changes. Explain the rule that caused it.
2. Name two things a low-trust actor cannot do that a standard-trust agent can.
3. Which audit verdict blocks install-update, and which one merely asks for a judgement?
4. An agent reports it cannot comment on an issue. Which four fields of the denial payload do you ask for, and what does each tell you?

F4. Bundled and optional skills

Part B established what a skill is and why context economy matters. This section is about the library that ships with the product — what is in it, and when each piece earns its place on an agent.

F4.1 The bundled catalogue

Seven bundled skills ship with the Paperclip application, versioned with it and installable into companies through the catalogue. The descriptions below are the documentation's own.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Bundled skills

Category	Skill	What it does (verbatim)
Docs	Doc Maintenance	"keep project documentation aligned with recent code and feature changes"
Paperclip Operations	Issue Triage	"classify stale, blocked, in-review, or stalled Paperclip issues and choose the next action"
Paperclip Operations	Status Card Query	"author Paperclip status cards, and compile a card's prose prompt into bounded search queries"
Paperclip Operations	Task Planning	"turn approved plans into Paperclip issues with ownership, dependencies, and acceptance criteria"
Product	Wireframe	"produce low-fidelity UI wireframes as standalone SVG deliverables"
Quality	QA Acceptance	"define observable acceptance criteria and focused verification passes"
Software Development	GitHub PR Workflow	"prepare, inspect, update, and communicate GitHub pull request work"

Read that list as an org chart rather than a feature list. Task Planning belongs on whoever converts an approved strategy into issues — usually the CEO or a manager. QA Acceptance belongs on the reviewer named in an execution policy, because it is about producing OBSERVABLE acceptance criteria, which is precisely the thing a review stage checks. Issue Triage belongs on whoever runs your daily sweep. Doc Maintenance and GitHub PR Workflow belong on engineering roles.

Note: "Bundled doesn't mean forced on everyone: you still browse, pick, and install." The bundled catalogue is a shelf, not a default configuration. The one exception is described separately below.

F4.2 The optional catalogue

Seven optional skills are listed as "extras you add when you need them", each in a named category:

Optional skills

Skill	Category
Agent Browser	browser
Release Announcement	content
Simplified English	content
Ramp	finance
Design Critique	product
Last30Days	research
Prepare MCP Integration	software-development

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The individual optional-skill pages were not harvested for this guide, so this table asserts only the names and categories the documentation lists. Read each skill's own SKILL.md before attaching it — which is the standing rule for any skill on a high-trust role.

F4.3 Bundled-required behaviour

One class of skill is not optional at all, and this catches people out. Skills under the key prefix paperclipai/paperclip/* carrying metadata.sourceKind: paperclip_bundled are, in the documentation's words, "unioned into every resolved desiredSkills set by the server. Removing them from the request has no effect."

The same protection applies to deletion. Attempting to delete a bundled skill returns: "It's a bundled skill. The server re-imports skills/ from disk on every company list call."

There is a separate set of BUNDLED EXAMPLE skills in the server's skills/ directory, which are the ones that give an agent its Paperclip literacy:

- paperclip — the base heartbeat skill.
- paperclip-board — company governance via chat.
- paperclip-create-agent — the hire workflow.
- paperclip-converting-plans-to-tasks — plan-to-task translation.
- para-memory-files — PARA file-based memory.

F4.4 Versioning and release pinning

How each skill source is versioned

Source	Pinned?	Update path
github	Yes (commit SHA)	GET /update-status then POST /install-update
skills_sh	Yes (GitHub)	Same as github
url	No	None; re-import to refresh
local_path (managed)	Live	Files refresh on read
local_path (project scan)	Live	Re-run the scan endpoint
paperclip_bundled	Pinned to release	Upgrade Paperclip itself
catalog	Pinned to metadata.originHash	GET /update-status then POST /install-update, or POST /reset

Beta release pinning is gated behind an instance experimental setting, enableBetaSkills, which is OFF by default and lives at Settings → Instance settings → Experimental. Two shipped bundled-paperclip releases are documented: v0, "V0 — Original" (2026-07-15), and v7-roster, "V7 — Roster champion" (2026-07-21).

Note: If any desiredSkills entry carries a non-null versionId while enableBetaSkills is off, the request fails with 400 and the message "Beta skill version pins require the Beta skills experimental setting to be enabled." With the flag off, saved pins are IGNORED rather than deleted, and every agent resolves to the live skill — so turning the flag off does not undo a pin, it merely suspends it.

Check your understanding

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Name the bundled skill you would attach to a reviewer named in an execution policy, and justify it in one sentence.
2. Why does removing a paperclip_bundled skill from a desiredSkills request have no effect?
3. An agent behaves differently after you switched off enableBetaSkills, but the pin is still on the agent. Explain.
4. Which two skill sources are 'live' rather than pinned, and what risk does that carry?

F5. Writing a company skill

Attaching somebody else's skill is a procurement decision. Writing your own is a design decision, and it is the highest-leverage thing an operator can do to cut cost and raise consistency at the same time. This section walks the authoring workflow end to end.

F5.1 Decide it should be a skill at all

A procedure belongs in a skill when it is REUSED and CONDITIONAL. Reused, because a skill's whole economic argument is that many agents share one document. Conditional, because a skill is loaded on demand — if every heartbeat needs it, it belongs in the agent's instruction files instead, where it is always present.

Where a procedure belongs

Property of the procedure	Put it in
Needed on every heartbeat, by this agent only	HEARTBEAT.md in the instruction bundle
Defines who the agent is	SOUL.md in the instruction bundle
Needed sometimes, by several agents	A company skill
Long supporting detail the agent reads only when it has already decided to act	references/ inside that skill

Note: The economic claim, verbatim: "Skills keep the base context small. An agent with 10 skills doesn't load all 10 into its context on every heartbeat — it only loads the ones that are relevant." The documented benefits are fewer tokens per run (lower cost), cleaner and more focused context (better results), and one place to maintain a shared procedure.

F5.2 The folder

The required layout is a folder with SKILL.md at its root, plus up to three optional supporting folders:

```
my-skill/
├── SKILL.md
├── references/
│   └── examples.md
├── scripts/
│   └── run.sh
└── assets/
    └── logo.png
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Remember F3: the folder you choose sets the trust classification. references/ is markdown only, assets/ is assets, scripts/ is scripts and executables, and the skill takes the highest classification present. Folders nest up to four levels deep.

F5.3 The frontmatter

SKILL.md is YAML frontmatter followed by a Markdown body. Five frontmatter fields are documented:

SKILL.md frontmatter fields

Field	Required	Type	Notes
name	Recommended	string	Human-readable label; falls back to the slug.
description	Recommended	string	Agent routing logic. Block scalars are supported.
slug	Optional	string	Kebab-case identifier; derived if absent.
key / skillKey	Optional	string	Canonical key override.
metadata	Optional	object	Arbitrary record with recognised sub-fields.

Recognised metadata sub-fields include metadata.skillKey, metadata.canonicalKey and metadata.paperclipSkillKey (which carry the canonical key across imports); the nested forms metadata.paperclip.skillKey and metadata.paperclip.key; metadata.sourceKind, one of paperclip_bundled, github, skills_sh, url, local_path, project_scan, managed_local or catalog; metadata.sources[], a list of source descriptors carrying kind, repo, path, commit, trackingRef, hostname, url and rawUrl; and the GitHub coordinates metadata.owner, metadata.repo, metadata.ref and metadata.trackingRef.

F5.4 Writing the description as decision logic

This is the single most important line in the file, and it is not a summary. The description is what the agent reads to decide whether to load the skill at all. Write it as ROUTING LOGIC — an explicit "Use when" and an explicit "Don't use when":

```
---
name: code-review
description: >
  Use when asked to review a pull request or code diff.
  Don't use when writing new code from scratch.
---

# Code Review

When reviewing code, check the following...
```

The documentation's own design guidance for the body is short and worth following literally: "Be specific and actionable — Concrete checklists and steps produce consistent results"; "Keep each skill focused on one concern"; "Put supporting detail in references/".

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: A description that reads "Helps with code quality" is a description that will be loaded on every heartbeat that mentions code, or on none. Both failure modes cost you money. The negative clause — "Don't use when" — is the half most authors omit and the half that does most of the routing work.

F5.5 Authoring and testing in Skill Studio

Skill Studio at /skills/studio is the built-in authoring surface. On desktop it presents three resizable panes; on narrow screens they become tabs.

- LEFT — a file tree of everything in the skill folder, with an editor underneath. Markdown files open in the rich editor. Toolbar actions are Add file, Add folder, Delete file and Delete folder. Unsaved work shows an Unsaved badge.
- MIDDLE — the input text area. Saved inputs are organised with / in their name, exactly like folders.
- RIGHT — test-run history, with states succeeded, failed and cancelled. Runs refresh themselves while they are in flight.

The Run button reports why it is unavailable, and the four documented reasons are a useful checklist in their own right: "This skill has no files to test"; "Pick an agent to run"; "Add or paste input text to run"; "Save skill edits before running". Advanced options include a run-template selector, starting from "Default test template", with "No template" for raw input testing.

Note: "Test runs happen on a hidden harness task, which keeps them out of your real issue list." That is why a skill test cannot read run-bound secrets — skill-test tokens are denied that access deliberately (see F2.3). Test the LOGIC in the Studio; prove the CREDENTIALS in a real heartbeat.

One structural constraint: SKILL.md cannot be deleted. Renaming is possible only for Paperclip-managed skills, requires the new slug to be free, is audited, obeys the company skill policy, and changes the name, the slug and the key together — with the documented consequence that "Every agent that had the old skill attached is moved to the new key automatically."

F5.6 Attaching the skill

There are two documented attachment points. At hire time, pass desiredSkills in the create body:

```
{
  "name": "Backend Engineer",
  "adapterType": "claude_local",
  "desiredSkills": ["paperclip", "code-review", "deploy-runbook"]
}
```

After hire, use the sync endpoint. The mode field is REQUIRED and takes add, remove or replace:

```
POST /api/agents/{agentId}/skills/sync
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
{ "desiredSkills": ["paperclip", "code-review"], "mode": "add" }
```
Or from the CLI:

```
paperclipai agent skills <agent-id>
paperclipai agent skills:sync <agent-id> --desired-skills paperclip,github
```
Each desiredSkills entry accepts a UUID, a canonical key, or a slug when that slug is unique in the company. The resolution order is documented: exact UUID match on id, then a normalised key match, then a normalised slug match "but only when exactly one skill in the company has that slug."

- Ambiguous reference: "Invalid company skill selection (ambiguous references: <slug>; ...)"
- Unknown reference: "Invalid company skill selection (unknown references: <reference>; ...)"

Inspect the resulting state with GET /api/agents/{agentId}/skills, which returns an AgentSkillSnapshot carrying adapterType, supported, mode, desiredSkills, entries[] and warnings. The mode field tells you whether the files actually reach the runtime:

Adapter skill sync modes

Mode	Behaviour
persistent	The adapter writes skill files into the working directory and leaves them there between runs.
ephemeral	Files are materialised per run and cleaned up afterwards. This is the default for sandboxed adapters.
unsupported	The assignment is recorded, but Paperclip cannot push files into the runtime — it only tracks which skills you assigned.

Note: A skill attached to an agent whose adapter reports unsupported is a skill that does nothing. Check the mode field before you conclude that a skill is being ignored for some subtler reason.

F5.7 Importing rather than authoring

POST /api/companies/{companyId}/skills/import accepts a wide range of source formats, which is how you bring an existing procedure into a company without retyping it:

```
{ "source": "https://github.com/paperclipai/paperclip" }
{ "source": "https://github.com/paperclipai/paperclip/tree/main/skills/paperclip" }
{ "source": "paperclipai/paperclip" }
{ "source": "paperclipai/paperclip/paperclip-create-agent" }
{ "source": "https://skills.sh/paperclipai/paperclip/paperclip-create-agent" }
{ "source": "https://example.com/raw/SKILL.md" }
{ "source": "/Users/me/code/my-project/skills/code-review" }
```
There is also a project scan — POST /api/companies/{companyId}/skills/scan-projects — which sweeps well-known locations inside your project workspaces, including skills/, skills/.curated/, .claude/skills/, .codex-style tool folders and many more, plus "the workspace root itself (if it contains a SKILL.md)." Imported project skills reference the files in the project workspace rather than copying them, which is what makes them live rather than pinned.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Skills are COMPANY-SCOPED, not org-scoped: "A skill installed in Company A is invisible to Company B." If you run two companies on one instance and want the same procedure in both, you import it twice. Mutating skill routes require either the agents:create permission or permissions.canCreateAgents=true.

Check your understanding

1. Give the two properties a procedure must have before it belongs in a skill rather than an instruction file.
2. Write a description block for a skill that produces monthly finance summaries, including both clauses.
3. Which folder would you put a 40-line checklist in, and what does that choice do to the skill's trust level?
4. You attach a skill and the agent ignores it. Name three checks, in order.
5. Why is 'mode' a required field on the skills sync endpoint?

F6. Connecting external systems

An agent that can only talk to Paperclip is not much use. This section covers the connection surfaces the harvested documentation actually specifies. It is deliberately conservative: several of the product's how-to guides on integration were not harvested, and the honest answer for those is a pointer rather than a half-remembered command.

F6.1 The general shape of an integration

Every external connection in Paperclip resolves to one of four mechanisms, and recognising which one you are dealing with tells you where to configure it and where to look when it fails.

The four integration mechanisms

Mechanism	Where it is configured	What it is for
Adapter env + credentials	Agent adapterConfig.env, as secret_refs	Giving the RUNTIME the credentials it needs — the provider key, a GitHub token, anything the agent's own tooling reads from the environment.
The HTTP adapter	adapterType: http	Letting Paperclip call YOUR service as if it were an agent. A single request per run.
The process adapter	adapterType: process	Letting Paperclip launch your own command, script or runtime as an agent.
Routine webhook trigger	Routine → Triggers → webhook	Letting an EXTERNAL system wake Paperclip on its own schedule or event.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

F6.2 Outbound: the HTTP adapter

The HTTP adapter is the cleanest way to put an existing service behind an agent identity. "The HTTP adapter does not run a child process. It sends a single request, waits for the response, and treats any non-2xx response as a failure."

HTTP adapter configuration

Field	Required	Notes
url	Yes	Absolute http:// or https:// endpoint.
method	No	Defaults to POST.
headers	No	Extra request headers.
payloadTemplate	No	JSON object merged into the request body before the standard fields.
timeoutMs	No	Request timeout in milliseconds. 0 means no timeout.

The request always carries runId, agentId and context; anything from payloadTemplate is merged alongside:

```
{
  "runId": "run-123",
  "agentId": "agent-123",
  "context": {
    "taskId": "issue-123",
    "wakeReason": "scheduled",
    "commentId": null
  },
  "customField": "value"
}
```

Note: Two design rules from the documentation. Return 2xx when the webhook is ACCEPTED, "not when all follow-up work is complete" — a long job should acknowledge and then report back through the API. And "Keep the endpoint idempotent when possible because retries are easier to support."

The process adapter is the local equivalent: it "launches the command, streams logs, and returns stdout/stderr plus exit metadata" and "does not invent a higher-level protocol." It takes command (required), args, cwd (must be absolute), env, timeoutSec (0 means no timeout) and graceSec (defaults to 15). Non-zero exits return an error message with the raw output payload.

Note: Both http and process — along with openclaw_gateway — are shown as "Coming soon" in the UI dropdown but are "fully functional in the runtime". Reach them through the API or a company export/import, not the hire form.

F6.3 Inbound: routine webhooks

A routine's trigger can be a webhook — "a signed URL you can call from anything outside Paperclip". This is the supported way for an external system to start work inside your company: a CI pipeline, a monitoring alert, a form submission.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Webhook trigger configuration

Setting	Documented values
Signing mode	bearer, hmac_sha256, github_hmac, none
Replay window (seconds)	"how far back in time a signed request's timestamp may be" — hidden for github_hmac and none
Secret	Shown once on creation; a Rotate secret button mints a new one

Note: The github_hmac signing mode exists precisely so a GitHub webhook can be verified with GitHub's own scheme. Choose none only on a loopback-bound instance you control, and never in authenticated + public mode.

F6.4 Provider credentials and their rotation

Provider API keys are ordinary secrets, so rotation follows F2 exactly: create the new value as a new VERSION via POST /api/secrets/{secretId}/rotate, leave the agents' secret_refs on version "latest", and the next run picks the new value up. That is the whole reason to prefer latest over a numeric pin.

Documented provider environment variables

Variable	Adapter it serves
ANTHROPIC_API_KEY	claude_local (Anthropic keys start sk-ant-)
ANTHROPIC_BASE_URL	claude_local — optional custom endpoint
CLAUDE_CODE_USE_BEDROCK=1	claude_local — AWS Bedrock detection
OPENAI_API_KEY	codex_local (OpenAI keys start sk-); overwrites the host login
GEMINI_API_KEY / GOOGLE_API_KEY	gemini_local

Four adapter provider overrides are documented for routing to non-default gateways: PAPERCLIP_CODEX_PROVIDERS (JSON of custom providers for Codex's config.toml), PAPERCLIP_PI_PROVIDERS (for Pi's models.json), PAPERCLIP_OPENCODE_PROVIDERS (merged into OpenCode's provider config) and PAPERCLIP_OPENCODE_SMALL_MODEL (OpenCode's auxiliary model). "Values support {env:VAR} placeholders for secret expansion server-side."

After any credential change, validate before you rely on it. Three surfaces do this:

```
POST /api/companies/{companyId}/adapters/{type}/test-environment
GET  /api/companies/{companyId}/adapters/{type}/models
GET  /api/companies/{companyId}/adapters/{type}/detect-model
```

The Test environment button on the agent form calls the first of these. For claude_local, testEnvironment() checks that Claude Code is installed and executable, that the working directory is absolute and usable, and that authentication is configured — via ANTHROPIC_API_KEY, Bedrock, or a subscription login — and then runs a live probe with the prompt "Respond with hello." For codex_local the rule is "No silent credential-less runs": the adapter fails fast if no login or API key is available.

Note: Claude subscription login has its own board-only surface: POST /api/agents/{agentId}/claude-login (claude_local agents only), with a setup-token-login-sessions

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

flow under /api/companies/{companyId}/ and a claude-oauth-token-status endpoint for checking state. The CLI equivalent is paperclipai agent claude-login <agent-id>.

F6.5 Where the documentation stops

Paperclip's how-to library lists dedicated guides for Connect an Agent to GitHub, Add an MCP Server to an Agent, and Wire Slack/Discord Notifications. Those pages were NOT harvested for this guide, so this guide asserts nothing about their specific fields, commands or endpoints.

What can be said safely is the shape each of them takes, because it follows from the mechanisms above:

- GITHUB — the agent's own runtime does the Git and API work, so the connection is a credential in the adapter environment plus, for repository-bound work, a project Repository binding (a local folder cwd, a remote repoUrl, or most commonly both). The bundled GitHub PR Workflow skill supplies the procedure.
- MCP — an MCP server is a tool surface belonging to the RUNTIME, not to Paperclip's control plane. The optional Prepare MCP Integration skill exists for this work, and tools:admin is the documented permission key covering "tool plumbing, stdio command templates, MCP gateways, mint/revoke tokens".
- SLACK / DISCORD — outbound notification is an external call, so it follows the outbound mechanisms in F6.2 or a skill that calls the service directly, with the webhook URL held as a secret_ref.

Note: This is the right instinct to carry into production generally: when the documentation does not specify a field, do not guess it into a runbook. Record the mechanism, cite the page you have not yet read, and verify against the live product before you commit the procedure to your own SOP.

Check your understanding

1. Name the four integration mechanisms and give one example use of each.
2. Why should an HTTP adapter endpoint return 2xx on acceptance rather than on completion?
3. Your provider key is compromised at 14:00. Describe the rotation in the order you would perform it, and say why the agents need no reconfiguration.
4. Which webhook signing mode would you choose for a GitHub-originated trigger, and which would you never choose on a public instance?

F7. Backup, restore and portability

Part D introduced portability as a principle: a Paperclip company is data, not a hand-built configuration. This section is the operational detail — what is actually backed up, how collisions are handled, and what a restore looks like when you are doing it under pressure.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

F7.1 Three things worth backing up

The three backup surfaces

What	How	Why it matters
The database	paperclipai db:backup	Everything: companies, agents, issues, runs, activity, secret records.
The master key	Copy ~/.paperclip/instances/default/secrets/master.key	Without it, a restored database's secret material is unreadable.
The company package	Company export (UI) or paperclipai company export	A portable, human-readable package you can move, diff and re-import elsewhere.

Note: A database backup WITHOUT the master key is not a backup of your secrets. This is the most consequential backup mistake in Paperclip, and it is silent until the day you restore.

F7.2 Database snapshots

```
paperclipai db:backup
paperclipai db:backup --dir /backups/paperclip
paperclipai db:backup --retention-days 14
paperclipai db:backup --json
```

db:backup flags

Flag	Effect
--dir <path>	Backup output directory.
--retention-days <days>	Retention window for daily pruning. Default 30.
--filename-prefix <prefix>	Backup filename prefix. Default paperclip.
--json	Print backup metadata as JSON — the form to use from a scheduled job.

Note: "For the embedded database you can run a backup without the server up — db:backup connects on the embedded port directly." That is the embedded PostgreSQL on port 54329, and it means a nightly backup does not need to negotiate with a running instance.

F7.3 Exporting a company

The export surface is reachable from the Org Chart header or the Settings → Export shortcut, and from the CLI:

```
paperclipai company export <company-id> --out ./path
paperclipai company export <company-id> --out ./path --include
company,agents,projects,issues,skills
```

The default included content is documented precisely: company metadata; every NON-TERMINATED agent; every NON-ARCHIVED project; every issue. Read that carefully — a terminated agent and an archived project are deliberately left behind, which is usually what you want when cloning a structure and occasionally not what you want when taking a forensic copy.

The export UI has three regions. A sticky bar carries the company name, a selected / total files counter, any warnings, and the Export N files button. The left panel is a searchable Package files tree, where "Search expands matching directories; restores

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

state on clear", the tasks/ directory is paginated at 10 tasks visible by default behind a Show more issues button, and "Checked and search-matched tasks pinned above fold". The right panel previews the selected file and updates the URL for deep-linking. README.md and .paperclip.yaml are regenerated on the fly to reflect your selection.

F7.4 Importing and collision handling

Import accepts two source types: a GitHub repo ("Paste tree/blob URL to folder containing COMPANY.md") or a local zip — an exported .zip package.

Note: "Avoid re-zipping archives in Finder/Explorer; use Export page download." A re-zipped archive frequently fails to import. Also note PAPERCLIP_IMPORT_ZIP_MAX_BYTES, which caps the company-import zip size at a default of 1 GB, clamped between 1 byte and 64 GiB.

Target selection offers Create new company (the default, with an optional New company name field to override the package name) or Existing company ("Apply package on top of currently selected company"). Collision strategy is a separate dropdown:

Collision strategies

Strategy	Behaviour
Rename on conflict (default)	"New name proposed (e.g., gstack-CEO)"
Skip on conflict	"Existing items kept; incoming copy dropped"
Replace existing	"Incoming version overwrites current entity"

Preview before you apply. Clicking Preview import generates three sections: a RENAMES list of conflicting agents and projects, each with a free-form target-name input, a per-row Skip button and a Confirm rename button; an ADAPTERS list of every agent in the package with an available-adapter dropdown, where "New agents default to target company CEO's adapter" and a Configure adapter button opens an inline per-agent config form; and a FILE TREE in the same layout as the export tree, with an action badge of create, update, skip or replace on each row and an arrow showing the target name on renamed directories. The sticky bar shows file, conflict and error counts.

Then click Import N files. The button is disabled if the preview shows errors or no files are selected. On success Paperclip refreshes the companies list, navigates to the imported or updated company dashboard, and restores the sidebar order if the package included one.

What is protected on an import into an EXISTING company

Rule	Consequence
"Incoming issues always created as new"	An import cannot silently overwrite issue history.
"replace blocked by server"	The most destructive strategy is refused on this path.
"Full replace requires dedicated instance-admin flow"	There is a deliberate escalation before anything is truly overwritten.
Existing-company imports are non-destructive	They do not remove data.

Note: IMPORTED AGENTS HAVE TIMER HEARTBEATS DISABLED BY DEFAULT. This is a safety property and you should not be surprised by silence after a restore — nothing is broken,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

nothing has started. Re-enable interval heartbeats deliberately, agent by agent, after you have checked budgets and credentials.

F7.5 The restore workflow

Under pressure, work in this order. Each step is verifiable before you move on, which is what stops a restore from becoming a second incident.

- 1. STAND UP THE CONTROL PLANE. Restore or start the instance and confirm GET /api/health responds — it reports "whether the server is up and whether the database is reachable", plus deployment mode, auth readiness and feature flags.
- 2. RESTORE THE MASTER KEY before you test anything credential-shaped, or every secret resolution will fail in a way that looks like an adapter problem.
- 3. RUN THE DIAGNOSTIC. paperclipai doctor checks config validity, deployment/auth mode compatibility, the agent JWT secret, the secrets adapter, storage, database connectivity, the LLM provider, the log directory, the listen port, the Node.js runtime, the managed install and the background service.
- 4. IMPORT OR VERIFY THE COMPANY, with Preview first and a deliberate collision strategy.
- 5. FIX THE ADAPTERS. The import's Adapters section defaults new agents to the target company CEO's adapter; working directories and environment need to be right for THIS machine.
- 6. RE-ESTABLISH BUDGETS before re-enabling anything. A restored company with no caps and a working key is an uncapped company.
- 7. RUN ONE AGENT MANUALLY. Use paperclipai heartbeat run --agent-id <id> and watch the transcript before you trust the schedule.
- 8. RE-ENABLE HEARTBEATS one agent at a time, checking spend after each.

Two adjacent commands are worth knowing for the same class of work. Archiving is the reversible way to take a company out of active rotation; it pauses all active agents with pause reason company_archived, cancels queued and in-flight wake-up requests, and preserves agents, projects and issues. Deletion is the destructive one:

```
paperclipai company archive <company-id>
paperclipai company delete <company-id> --yes --confirm <value>
```

Note: Deletion requires PAPERCLIP_ENABLE_COMPANY_DELETION to be enabled server-side, and --confirm must match the target company ID or prefix. The documentation's warning is one sentence long: "There is no undo."

Note: The Cloud Sync and Sync to Cloud Upstream pages of the official documentation were not harvested for this guide. Cloud-managed instances are configured through PAPERCLIP_MANAGED_CONFIG — a JSON document describing feature flags, plugins and the sandbox environment, requiring v: 1, mode: "cloud", catalogVersion, features and plugins.autoInstall, with an optional environments array limited to ONE entry and no credentials permitted inside the config. Beyond that, consult the live documentation.

Check your understanding

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Name the three things you must back up, and say which omission fails silently.
2. What does a company export deliberately EXCLUDE by default?
3. Which collision strategy is blocked by the server on an import into an existing company, and why is that sensible?
4. You restore a company and nothing runs. Give the most likely benign explanation.
5. Put the eight restore steps in order and say which one prevents a restore from becoming a spend incident.

F8. Debugging a stuck heartbeat

"Nothing is happening" is the most common operator report and the least useful. This section turns it into a method. The principle is to work OUTWARD from the run: establish first whether a run happened at all, because that single fact splits the problem space in half and everything after it follows.

F8.1 The first question: did a run start?

```
paperclipai run live --company-id <company-id>
paperclipai run list --company-id <company-id> --agent-id <agent-id> --limit 50
```

In text mode these report id, status, agentId, invocationSource, triggerDetail, startedAt, finishedAt and logBytes. That is enough to branch:

The branch point

Observation	Class of problem	Go to
No run record at all	The agent never woke — a TRIGGER problem	F8.2
A run record exists but failed immediately	The environment — a LAUNCH problem	F8.3
A run started and is still running long past expectation	The runtime is stuck — an EXECUTION problem	F8.4
A run succeeded but nothing changed on the board	The agent ran and did nothing useful — a WORK problem	F8.5

Note: Answer this question before forming any theory. Most wasted debugging time comes from investigating a model-behaviour hypothesis for an agent that never woke up.

F8.2 No run started — the trigger problem

Work the list in order; each item is cheap and rules out a class of cause.

- IS THE AGENT PAUSED, OR IS ITS HEARTBEAT MERELY OFF? These are different and the distinction is the single most useful fact in this section. Heartbeat-off still wakes on assignment, on comment, on routine and on demand. PAUSED WAKES ON NOTHING.
- IS IT PAUSED BY BUDGET? At 100% utilisation the agent is auto-paused with pauseReason = "budget" and no further heartbeats fire. The dashboard's Month Spend card and the agent Budget tab both show this.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- IS THE INTERVAL ACTUALLY ON? "Heartbeat on interval" is OFF BY DEFAULT. An agent configured but never switched on is the most common false alarm in the whole product.
- IS THE SCHEDULER RUNNING? HEARTBEAT_SCHEDULER_ENABLED defaults to true and HEARTBEAT_SCHEDULER_INTERVAL_MS defaults to 30000. A disabled scheduler silences every timer wake on the instance at once — if EVERY agent has gone quiet, check here first.
- IS THE INTERVAL BEING MISREAD AS A GUARANTEE? "Heartbeat interval — the minimum number of seconds between automatic heartbeats. This is a floor, not a guarantee."
- IS THE AGENT AWAITING APPROVAL? An agent in pending_approval cannot be manually run — the Run Heartbeat button is disabled for that status.
- IS THE COMPANY ARCHIVED? Archiving pauses all active agents with pauseReason company_archived and cancels queued and in-flight wake-ups.
- IS THE ESCALATION PATH BROKEN? Two banners appear on the agent page: Invalid Org Chain (a terminated manager, a missing manager, or a loop) which "Blocks work assignment", and Escalation Path Paused (a paused agent up the reports_to chain) where "Escalations refused; no functional escalation path."

Then force the issue. A manual invocation removes every scheduling variable at once:

```
paperclipai heartbeat run --agent-id <agent-id>
paperclipai heartbeat run --agent-id <agent-id> --source on_demand --trigger manual
paperclipai heartbeat run --agent-id <agent-id> --timeout-ms 120000 --debug
```

Note: "For routine agent wake-ups during normal operation, prefer agent wake and the prompt-handoff flow in prompt. Reach for heartbeat run when you specifically want to watch a single run unfold." That is exactly this situation. --debug shows raw adapter stdout/stderr JSON chunks, and --timeout-ms defaults to 0, meaning no client-side timeout.

F8.3 A run started and failed — the launch problem

Read the run, then read its log. The invocation card in the run detail shows the command, cwd, prompt and environment with secrets redacted — which is where launch failures reveal themselves:

```
paperclipai run get <run-id>
paperclipai run events <run-id> --after-seq 0 --limit 200
paperclipai run log <run-id> --offset 0 --limit-bytes 16384
paperclipai run log <run-id> --text
```

Run events report seq, eventType, stream, level and message. The documentation names the three usual root causes on the dashboard page: "usually an API key problem, a budget running out, or a misconfigured adapter." Test each directly rather than inferring:

```
POST /api/companies/{companyId}/adapters/{type}/test-environment
```

- CREDENTIALS. For claude_local, testEnvironment() checks the binary, the working directory and authentication, then probes live. For codex_local the rule is "No silent credential-less runs" — it fails fast rather than running unauthenticated.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- WORKING DIRECTORY. cwd must be absolute. Claude Code session resume "validates against a system root allowlist to prevent agent access to protected directories (/, /tmp, /var, /etc, /proc, etc.)", so a cwd inside one of those will not behave as expected.
- SESSION. Claude Code "stores the Claude Code session id and resumes it on the next heartbeat when the working directory still matches" and falls back to a fresh session if resume fails. Codex "preserves the previous_response_id chain", and changing the working directory triggers a fresh session. If a corrupted session is suspected, use Reset Sessions from the agent overflow menu, or the Runs tab's "Clear sessions for touched issues".
- WORKSPACE. A workspace in degraded, failed, repairing or cleanup_failed is a common cause of an agent that appears to do nothing.
- ENVIRONMENT VARIABLES GENERALLY. paperclipai env prints Required and Optional sections and marks each variable set, default or missing. The required pair is PAPERCLIP_AGENT_JWT_SECRET and DATABASE_URL; PAPERCLIP_TOOL_ACTION_SIGNING_SECRET is required with no fallback.

Note: One error kind is specifically recoverable: process_lost. The run detail offers Resume, which "pick[s] up after process_lost error (requires resumeFromRunId)". Try Resume before Retry — Retry starts a new run with the same task context and repeats any work already done.

F8.4 A run is stuck mid-execution

A live run streams its transcript and logs in real time. When the transcript stops updating, the run is stalled rather than working. Three things to check and one thing to do.

- TIMEOUTS. claude_local timeoutSec defaults to the sandbox default of 14,400 seconds — four hours — which is far longer than most operators expect. codex_local has an outputInactivityTimeoutMs default of 420000 (seven minutes), which detects a silent runtime much faster.
- TURN CAP. claude_local maxTurnsPerRun defaults to 300. A run consuming turns without progressing is a prompting or acceptance-criteria problem, not an infrastructure one.
- WORKSPACE OPERATIONS. paperclipai run workspace-operations <run-id> reports id, status, phase, command, cwd and logBytes; paperclipai run workspace-log <operation-id> reads the log of one of them. A provision or teardown command that hangs will hang the run.

```
paperclipai run cancel <run-id>
```

Note: "Cancelling a run does not roll back what the agent already did." Cancel stops the burn; it does not undo a half-finished change. Check the issue and the workspace afterwards, and release the issue if the agent still holds it.

There is also a watchdog decision surface for runs the system itself has flagged:

```
paperclipai run watchdog-decision <run-id> --decision continue --reason "text"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai run watchdog-decision <run-id> --decision snooze --snoozed-until 2026-06-01T18:00:00Z
paperclipai run watchdog-decision <run-id> --decision dismissed_false_positive --evaluation-issue-id <id>
```

F8.5 A run succeeded but nothing moved

This is the subtlest case and the one that most rewards knowing the runtime rules.

- THE COMMENT-REQUIRED BACKSTOP. Every agent run must post a comment back to the issue — a runtime invariant that is always on. If it does not, issueCommentStatus becomes retry_queued and the agent is woken with reason missing_issue_comment. Still no comment after the retry and it becomes retry_exhausted, no further retries, failure recorded. A cycle of wake-run-nothing with that wake reason is this loop.
- THE CROSS-ISSUE CAP. CROSS_ISSUE_INFLUENCE_LIMIT = 20 cross-issue comments or task updates combined per run; writes to the run's own source issue do not count, and the budget resets per run. Over the cap returns 429 with code cross_issue_influence_cap_exceeded. An agent doing bulk triage will hit this legitimately.
- CHECKOUT FAILURE. Checkout returns 409 when the issue does not match one expectedStatuses value, when the project is paused, or when the workspace is a closed isolated workspace. "Never retry a 409 — pick a different issue." An agent that keeps failing checkout looks identical, from the board, to an agent doing nothing.
- MISSING RUN ID. Agent updates to checked-out in_progress issues require the X-Paperclip-Run-Id header. Without it the server rejects the update as a checkout-ownership violation, so the agent works and the board never changes.
- EXECUTION STAGE LOCK. Only the current participant can advance a stage; non-participants receive 422. Once a review has escalated, "Only the escalated reviewer can advance the current execution stage." An issue sitting in in_review whose stage participant is a suspended human will never move on its own.
- AN INTERACTION IS WAITING. A pending interaction card — ask_user_questions, request_confirmation and the rest — parks the work on a human answer. Pending interactions expire automatically when an issue moves to done or cancelled, with status expired and result.outcome: "issue_closed".

Two inspection commands close this loop by showing you exactly what the agent saw:

```
paperclipai issue heartbeat-context <issueId>
paperclipai agent inbox
paperclipai issue active-run <issue-id>
paperclipai issue recovery-actions <issue-id>
```

Note: Recovery actions are Paperclip's own detection of exactly these silent stalls. The kinds are missing_disposition, stranded_assigned_issue, active_run_watchdog and issue_graph_liveness; statuses are active, escalated, resolved and cancelled; and only one active or escalated recovery action exists per source issue at a time. Resolve with an outcome of restored, false_positive, blocked or cancelled — noting that false_positive and cancelled are board-only.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

F8.6 The diagnostic write-up

Whatever you find, record it in the same five fields. This is what turns an incident into an asset, and it is what an assessor or an auditor will ask for.

The five-field incident note

Field	What goes in it
Observation	What you actually saw, with the timestamp. Not your theory.
Branch	Which of the four classes in F8.1 it fell into, and the evidence for that.
Root cause	The specific setting, credential, state or code. Name it exactly.
Fix	The action taken, with the command or the UI path.
Prevention	The check, cap, routine or instruction change that stops a recurrence.

Check your understanding

1. State the first question you ask when told 'nothing is happening', and say why it comes first.
2. Give three reasons an agent might never wake, in the order you would check them.
3. What is the difference between Resume and Retry on a failed run, and when is Resume the right choice?
4. An agent runs successfully every hour and the board never changes. Name three possible causes.
5. Why does cancelling a run not resolve the underlying problem?

F9. Cost engineering

Budgets stop a disaster. Cost ENGINEERING makes the disaster unlikely and the normal month cheaper. The two are different skills and this section is about the second.

F9.1 Reading the Costs page properly

Four headline metric tiles appear on every tab of the Costs page, and they measure two genuinely different things. Conflating them is the most common reporting error in the product.

The four headline tiles

Tile	What it measures
Inference spend	Per-request model cost, shown with a token count. This is what your agents burned by thinking.
Budget	Utilisation percentage, incident count, or "Open" when no cap is set.
Finance net	Debits minus credits.
Finance events	A count, plus estimated open debits.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The Finance tab covers "everything that isn't a per-request inference cost. Subscription fees, invoice reconciliations, credit top-ups, refunds, and adjustments." Inference is what the work cost. Finance is what the arrangement cost. A month can show flat inference and a large finance net, or the reverse, and the two call for entirely different responses.

Utilisation bar colours

Colour	Range
Green	Under 70%
Yellow	70-90%
Red	Above 90%

Note: The colour is a planning instrument, not an alarm. Yellow at day 10 of the month is a problem; yellow at day 28 is a well-sized cap. Read utilisation against the calendar, never on its own.

F9.2 Attributing cost to a run

Cost is not only a company-level number. Paperclip attributes it downwards, and the levels below are where you actually find the waste.

- PER AGENT. The agent Dashboard tab carries a Costs section with total input, output and cached tokens plus cost, and a per-run table for the 10 most recent runs. Its activity charts run over a 14-day window: Run Activity, Issues by Priority, Issues by Status and Success Rate.
- PER RUN. Each entry in the Runs tab shows the status icon, short run ID, invocation source pill (Timer, Assignment, On-demand or Automation), token counts and cost, a relative timestamp and a result summary excerpt.
- PER PROJECT. The project Budget tab tracks metric billed_cents over a lifetime window, with observed amount, remaining amount and utilisation percent.

The INVOCATION SOURCE PILL is the most under-used diagnostic in the product. It tells you WHY each run happened. A page of Timer runs with thin result summaries is an interval set too aggressively. A page of Assignment runs is an agent doing real work. A page of On-demand runs is you, doing the agent's job manually. Sort your most expensive agent's Runs tab and read the pills before you touch any other setting.

F9.3 The four levers

Where cost actually goes, and what to do about it

Driver	Symptom on the Runs tab	Lever
Runs that should not have happened	Many Timer runs with nothing to report	Turn the interval OFF and rely on wake-on-demand. The documented recommended combination is "Heartbeat on interval = off, Wake on demand = on".
Context carried needlessly	High input-token counts on short tasks	Move standing procedure out of the instruction bundle into SKILLS, which load only when relevant.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Turns spent going nowhere	Long runs, high output tokens, vague result summaries	Write acceptance criteria that name evidence. maxTurnsPerRun (default 300) is a backstop, not a plan.
Review ping-pong	Repeated in_review to in_progress cycles on one issue	maxReviewRounds defaults to 3, range 1-50. Lower it for cheap work; the escalation to a human is usually cheaper than a fourth agent round.

The skills lever deserves its own sentence because it is the one most people miss. Moving a procedure from an always-loaded instruction file into an on-demand skill reduces the tokens consumed on every heartbeat that does not need it — which, for a specialist procedure, is most of them. The documented benefit list is exactly this: "Fewer tokens consumed per run (lower cost)"; "Cleaner, more focused context for the agent (better results)"; "Easier to maintain".

Note: Two further economies are free. "An agent that isn't doing work isn't costing you anything (past whatever the last run spent). Between heartbeats there's no idle burn." And on mentions: "Don't overuse mentions — each mention triggers a budget-consuming heartbeat", and "Don't use mentions for assignment — create or assign a task instead."

F9.4 Handling a budget incident

An incident is an agent, project or company reaching 100% and being paused. Work it in this order rather than raising the cap first.

- 1. CONFIRM THE SCOPE. The three caps are parallel, not nested: "An agent can be paused by its own budget limit even if the company has remaining budget." Agent budget hit stops the agent regardless of project headroom; project budget hit pauses the project regardless of the agent; the company budget is the overall ceiling. Identify which one fired.
- 2. READ THE RUNS THAT SPENT IT. Sort by cost and read the invocation source pills. Decide whether the spend bought anything.
- 3. DECIDE, THEN ACT. If the spend was productive, raise the cap. If it was not, fix the cause first — the interval, the criteria, the review rounds — and only then decide whether the cap was ever right.
- 4. RESUME. An agent-level hard stop creates a Budget Override approval; a project hard stop is resolved either by raising the amount on the Budget tab or by resolving the budget override approval from the queue.

Note: "An auto-paused agent doesn't lose its work — any tasks it had in progress remain assigned to it and will pick up again once the agent is resumed." A budget pause is safe. It is not an emergency, and it does not need to be cleared in the first five minutes.

F9.5 Right-sizing the caps

The documentation gives starting figures. Treat them as a first month, not a policy.

Documented starting budgets

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Scope	Guidance
CEO	$30-50 per month
Manager (CTO, CMO)	$20-40 per month
Worker agent	$10-25 per month
Whole company, exploration	"$5-20 to play with the product"
Whole company, active	"$20-100/month for an active company"

Set the company cap FIRST, before the first hire, so there is a ceiling in place before anything can run. Then size each agent's cap from its OBSERVED first-month spend, not from the table. Monthly budgets reset to zero on the 1st of each UTC month; lifetime budgets — which is what project budgets use by default — never reset.

The Budget tab fields you are setting are: Window kind (Monthly UTC or Lifetime), Amount (a cap in cents, displayed as dollars), Warn percent (the soft-alert threshold, e.g. 80%) and Hard stop (the enforcement behaviour at 100% spend). The default is pauseOnExceed = true, so "Paperclip pauses the agent so it consumes no further budget until a board user resumes it or raises the limit."

Note: A cap that is never approached is not proof of discipline; it may simply be too high to have any governing effect. A well-sized monthly cap sits amber near the end of a normal month. That is the shape you are aiming for.

Check your understanding

1. State the difference between inference spend and finance net, and give one situation that moves one without the other.
2. Which column of the Runs tab tells you WHY a run happened, and what do you conclude from a page of Timer runs with thin summaries?
3. Name the four cost levers and say which one is most often overlooked.
4. An agent is paused at 100% while the company sits at 40%. Explain, and give the first thing you do.
5. Why is a budget that is never approached a possible failure rather than a success?

F10. An operating rhythm

A one-person AI company fails in one of two ways: neglect, where nobody looks and the problems compound quietly, or over-attention, where the operator does the agents' work for them and pays for the privilege twice. A rhythm is the defence against both. What follows is a suggested cadence, assembled from what the documentation says each surface is FOR. Adapt the times; keep the structure.

F10.1 Every morning — ten minutes

The dashboard is designed for exactly this. "The dashboard is your control centre... it tells you, at a glance, whether everything is moving forward or whether something needs your attention." The documentation's own reading table is your checklist:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The daily read (documentation's own healthy/attention table)

Signal	Healthy	Needs attention
Agents	Mostly active or running	Any in error or unexpectedly paused
Tasks in progress	Work is moving	Nothing in progress for hours, or too many blocked issues
Month spend	Climbing predictably	Sudden jump or active budget incidents
Approvals	Queue manageable	Pending approvals piling up
Recent activity	Steady, understandable motion	Long stretches of silence or repeated failures

Then three specific actions, in this order:

- CLEAR THE APPROVALS QUEUE. "Approvals don't expire on their own. If an approval is pending, agents that are waiting on it will pause at that point until you take action." An unread approvals queue is a stopped company. Work both the pending and the revision_requested items — the actionable set is "approvals in the pending state and any in revision_requested".
- CLEAR THE BLOCKED INBOX. Every blocked issue is an agent waiting on you. Check whether the blocker is a first-class blockedByIssueIds link or a free-text comment — the latter never auto-wakes anything.
- TRIAGE ANY error AGENT. "Any agent showing 'error' needs attention. The error won't fix itself." Open the agent's Runs tab and read the transcript.

Note: Ten minutes is the target and it is achievable, but only if you resist opening a run transcript out of curiosity. Note it, and do it in the weekly slot. The daily pass is a pulse check: "This feed isn't meant for deep investigation — it's a pulse check."

F10.2 Every week — an hour

- COST REVIEW. Work F9.2. Open the most expensive agent's Runs tab, read the invocation source pills and the result summaries, and decide whether the month is going where you want it to. Compare utilisation against the day of the month.
- STALE WORK SWEEP. Anything in in_progress for days, anything in in_review with no movement, anything in blocked whose blocker is already done. This is exactly what the bundled Issue Triage skill is for — "classify stale, blocked, in-review, or stalled Paperclip issues and choose the next action" — so consider making it a routine rather than a habit.
- REVIEW-ROUND CHECK. Look for issues that have cycled between in_review and in_progress. changesRequestedCount counts consecutive AGENT-initiated rounds and resets to 0 when a stage is approved or when a HUMAN requests changes — "Only counts agent-to-agent ping-pong." Repeated cycling means your acceptance criteria are ambiguous.
- ROUTINE HEALTH. Check routine states — paused, draft, archived, succeeded, failed, running. A failed routine is silent by nature.
- BACKUP VERIFICATION. Confirm last night's db:backup exists and is the size you expect. A backup you have not looked at is a hypothesis.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

F10.3 Every month — half a day

- AUDIT THE ACTIVITY LOG. The audit:view_agent_actions grant opens the company Audit feed and allows a CSV download. Export it and read for gaps and surprises — an unexplained silence is as informative as an unexplained action.
- REVIEW MEMBERSHIP AND GRANTS. Every member row, every explicit grant. Remember that explicit grants persist across role changes and there is no deny layer, so grants accumulate unless someone removes them. Suspend anyone who has left. Revoke their tokens.
- ROTATE CREDENTIALS. Rotate provider keys on a schedule you decide rather than on the day one leaks. With secret_refs on version "latest", rotation needs no agent reconfiguration.
- RE-SIZE BUDGETS. Monthly budgets reset on the 1st UTC, which makes the first week the natural moment to reset caps from last month's observed spend.
- REVIEW THE SKILL LIBRARY. Which skills are actually attached and used? Run an audit on any imported skill; check update-status on GitHub-sourced skills; delete what nobody uses.
- PRUNE THE ORG CHART. Terminate agents that no longer have a job — terminate, not delete: "Terminating stops the agent permanently and preserves it as a terminated record. Deletion is the destructive action." Check no terminated agent is still someone's manager, which produces the Invalid Org Chain banner.

F10.4 Automating the rhythm

Some of the above should be a routine rather than a habit, because a habit degrades and a routine does not. Routines carry a cron schedule in your timezone, and minutes snap to five-minute increments. Candidates: the stale-work sweep (Issue Triage), a documentation pass (Doc Maintenance), and a weekly summary produced by whichever agent owns reporting.

Note: Do not automate the approvals queue, the error triage or the audit read. Those are the three places where your judgement is the product, and the whole governance model — "Agents propose, people decide" — depends on a human actually being in the loop rather than nominally in it. There is also an emergency brake worth knowing: paperclipai routines disable-all --company-id <id> pauses every non-archived routine in one company. There is no enable-all counterpart, so use it deliberately.

Check your understanding

1. List the three actions in the ten-minute daily pass, in order.
2. Why is an unread approvals queue equivalent to a stopped company?
3. Which weekly check would catch ambiguous acceptance criteria, and what is the signal?
4. Name three things that belong in the monthly audit and one thing that must never be automated.
5. Design a routine for your own stale-work sweep: which agent owns it, which skill does it use, and what does it produce?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

F11. Taking this to production

The gap between a working lab and a production deployment is not technical. Everything in Parts A to F already works. The gap is that in production you are accountable for spend you did not personally authorise, for changes you did not personally make, and for evidence you may have to produce months later. This closing chapter is a staged path across that gap.

F11.1 The staging principle

Prove the AUDIT before you scale the WORK. It is tempting to reverse this — to get the agents productive first and worry about the trail later — and it is the reversal that produces companies nobody can safely operate. An audit trail is retrospective; you cannot add one to work that has already happened.

The four stages

Stage	Scope	The question it answers	Exit criterion
1. One workflow	One agent, one project, one recurring piece of work	Does an agent actually complete this work to the standard we need?	Three consecutive successful runs producing verifiable output.
2. Cap low	The same workflow, with caps deliberately tight	What does this cost, and does the enforcement behave as documented?	A budget pause has occurred and been handled without drama.
3. Prove the audit	The same workflow, audited end to end	Can a third party reconstruct what happened, from records alone?	Someone who was not present reconstructs one week's work from the activity log and the issue threads.
4. Scale	Additional agents, projects and routines, one at a time	Does the structure hold as the number of moving parts rises?	Each addition passes stages 1-3 on its own before the next begins.

F11.2 Stage 1 — one workflow

Pick the least glamorous recurring task you have. It should be work you understand well enough to check, that happens often enough to be worth automating, and whose failure is survivable.

- Write the mission as one sentence. If you cannot, you do not yet understand the work well enough to delegate it.
- ONE agent. The feasibility test from Part A applies unchanged: if the inputs, tools, authority and verification are the same across the whole task, do not split it.
- Interval OFF, wake-on-demand ON — the documented recommended combination. Trigger runs deliberately while you are still learning what they cost.
- Acceptance criteria that name EVIDENCE, not intent. "Produces report.md in the workspace with sections X, Y and Z" is checkable; "summarises the data" is not.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Prove the runtime with a marker run before you trust anything else. A healthy /api/health proves the control plane; only an artifact in the workspace proves the runtime.

F11.3 Stage 2 — cap low

Set the caps below what you expect to need. The purpose of this stage is to make the enforcement fire while the stakes are trivial, so you learn the mechanics on your own terms.

- Set the COMPANY cap first, then the agent cap. They are parallel, not nested.
- Set Warn percent so the warning arrives with time to act. The documented warn threshold is 80%, with pause at 100% and pauseReason = "budget".
- Let it pause at least once, on purpose. Confirm that the pause behaves as documented: in-progress tasks stay assigned and resume when the agent does.
- Practise the resume path — raising the cap, or resolving the Budget Override approval — before you need it under pressure.
- Record the observed monthly cost. That figure, not the documentation's table, is what you size the real cap from.

F11.4 Stage 3 — prove the audit

This is the stage most operators skip and the one that determines whether the deployment survives its first serious question. The test is concrete: hand one week of the company's work to someone who was not there, and see whether they can reconstruct it.

What must be reconstructible, and from where

Question	Where the answer lives
What was decided, by whom, and when?	The activity log; approval records carrying decisionNote, decidedByUserId and decidedAt.
Who did this piece of work?	The issue's assignee and checkoutRunId; the run record; the X-Paperclip-Run-Id attribution on every mutating request.
Why was it accepted?	The execution decision record — actor, outcome, a REQUIRED comment body, and createdByRunId. Both approvals and change requests require a comment; empty bodies are rejected.
What did it cost?	The run's token counts and cost; the agent Costs section; the budget policy's observed amount.
What credential was used, and who read it?	Every secret value read is logged to the security audit trail and the activity log.

- Grant audit:view_agent_actions to whoever is accountable, and export the Audit feed to CSV at least once so you know the export works.
- Use execution policy on anything consequential — a review stage, an approval stage, or both. "Runtime-enforced, not prompt-dependent" is the whole point: an agent cannot forget to seek review, because the runtime does not offer it the option.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Confirm the escalation path resolves to a real, active human. responsibleUserId with a fallback to createdByUserId is where an escalated review lands; if neither exists, work hands back to the implementer and the counter climbs for visibility only.
- Check that the trail survives a restart. Run through the restore workflow in F7.5 on a copy, and confirm the history came with it.

F11.5 Stage 4 — scale

Only now add. The discipline is that each addition passes stages 1 to 3 on its own before the next begins. The failure mode is adding three agents at once and losing the ability to attribute a cost, a change or a decision to any one of them.

- Add ONE agent at a time. Every additional agent is another workspace, another instruction bundle, another budget line and another audit trail — forever.
- Add a routine only when the work has proved itself as a manual habit first.
- Move to authenticated mode BEFORE the second human needs access, not on the day they ask.
- Turn on strict mode for secrets before the credential count grows, remembering that it blocks new inline values rather than migrating existing ones.
- Re-run the feasibility scan each time. The correct instinct is still to reach for one agent and be argued out of it.

F11.6 Production readiness checklist

Work down the list. Anything you cannot tick is a decision you have made by default rather than on purpose.

Readiness checklist

Area	Check	Evidence that it is done
Platform	Node.js 20+ confirmed; doctor passes cleanly	node --version output; a clean paperclipai doctor run
Platform	Required environment variables present	paperclipai env shows PAPERCLIP_AGENT_JWT_SECRET and DATABASE_URL as set, and PAPERCLIP_TOOL_ACTION_SIGNING_SECRET present (it has no fallback)
Access	Deployment mode matches who needs access	local_trusted only if genuinely single-operator; otherwise authenticated with the right exposure
Access	Hostname allowlist correct for authenticated + private	paperclipai allowed-hostname run, and the server restarted
Access	Every human has the least role that lets them work	The Access page reviewed; explicit grants deliberate, not inherited
Secrets	Master key backed up somewhere off this machine	A restore rehearsal, not merely a copied file
Secrets	No inline sensitive values in adapter, project or routine env	Strict mode on; the migration dry run reports nothing outstanding
Secrets	Rotation rehearsed	One provider key rotated with

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		agents left on version "latest", and a successful run afterwards
Budgets	Company cap set BEFORE any agent could run	The cap predates the first hire in the activity log
Budgets	Per-agent caps sized from observed spend, not from the guide table	Last month's figures
Budgets	One pause survived and resumed	The incident and its resolution recorded
Governance	Board approval required for hires, if that is your intent	Settings → Hiring toggle set deliberately
Governance	Execution policy on consequential work	Reviewer and/or Approver set; maxReviewRounds chosen rather than defaulted
Governance	Escalation resolves to an active human	responsibleUserId points at an active membership
Operations	Heartbeat intervals justified per agent	Interval off unless there is a reason it must be on
Operations	The daily, weekly and monthly rhythm is scheduled	Diary entries or routines, not intentions
Recovery	Nightly db:backup running and verified	A backup file from last night, of a plausible size
Recovery	A company export taken and re-imported successfully	The rehearsal, with the Preview screen read rather than skipped
Recovery	The restore runbook written down	The eight steps of F7.5, adapted to your instance
Audit	audit:view_agent_actions granted to the accountable person	The grant, and one successful CSV export
Audit	One week reconstructible by someone who was not present	They did it, and told you what they could not find

Note: The last line of that table is the real test. Everything above it is preparation for it. If a colleague can take your activity log, your issue threads and your run records and tell you what your company did last week — including what it decided, what it spent and who signed it off — then you are operating an auditable AI organisation rather than an impressive demonstration. That distinction is the whole purpose of a control plane, and it is what you will be assessed on.

Check your understanding

1. State the staging principle in one sentence and say why the order cannot be reversed.
2. Give the exit criterion for each of the four stages.
3. Why is 'cap low' a stage in its own right rather than a setting?
4. Name the five questions that must be reconstructible from records, and where each answer lives.
5. Pick three lines from the readiness checklist that your own current setup would fail, and write the action that would close each one.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Part G — Command and API Reference

This part is a reference, not a tutorial. It exists so that after the course you can find the exact command or endpoint you need without re-reading the narrative chapters or guessing at a flag. It is organised by SUBJECT — setup, authentication, company, project, issue, agent, run, routine, approval, cost, workspace, skills, secrets and then the REST API — and within each subject it follows the same shape: a short paragraph on what the command family is for, then the exact syntax, then the gotchas that cost people an afternoon.

Two disciplines make it useful. First, treat the syntax blocks as literal: placeholders appear in angle brackets — <company-id>, <agent-id>, <issue-id> — and everything outside those brackets is typed as written. Second, remember what the CLI is. "Agents do not run inside the CLI." Every command below is a control-plane call; the reasoning, the tool calls and the file writes all happen server-side in the runtime and its adapters.

Note: Reproduce these commands exactly. A flag that looks like an obvious synonym — --company instead of --company-id, --force instead of --yes — is not accepted. Where the documentation states a default, that default is given here in bold text in the tables; where it does not, none is invented.

G0. How to read this reference

Three conventions run through every Paperclip command family, and learning them once saves learning them fourteen times.

- COMPANY SCOPE IS PER-COMMAND. Almost every business object belongs to exactly one company, so most commands need to know which. That is supplied with -C or --company-id, or inherited from an environment variable or a stored profile. The project reference is explicit that company scope is per-command "rather than inherited globally".
- PAYLOADS ARE JSON, PARSED LOCALLY. Several families take a --payload-json (or --payload) flag carrying a JSON object. The CLI parses it with JSON.parse before sending anything, so malformed JSON fails on your machine with Invalid JSON: ... rather than producing a confusing server error.
- DESTRUCTIVE ACTIONS ARE GUARDED. Deletion commands refuse to run without --yes, and the most dangerous of them (company delete, secrets delete) additionally require --confirm with a value you must repeat by hand.

One more convention is worth stating on its own because it is the single most common source of lost credentials in a first week: several commands print a secret exactly once. That rule has its own section below, and it is not a suggestion.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G1. Global usage — the binary, the common options, output and credentials

G1.1 The binary

The CLI ships as a single binary named paperclipai, distributed as the npm package paperclipai. There are three documented ways to invoke it, and which one you use changes nothing about the syntax that follows the binary name.

```
# Run without installing anything (quick start)
npx paperclipai onboard --yes

# Global npm install
npm install -g paperclipai
paperclipai --help

# Managed install (recommended by the docs)
npx paperclipai install

# In a monorepo / git checkout, the docs use pnpm
pnpm paperclipai <command>
```

The managed install places the binary at ~/.local/bin/paperclipai with automatic updates and rollback, and keeps its managed store at ~/.paperclip/cli. Node.js 20 or newer must be on PATH; the documented check is node --version, which must report v20.x or higher.

Flags accepted by paperclipai install

Flag	Purpose
--canary	Install npm canary channel
--version <version>	Install exact published version
--ref <ref>	Install from GitHub branch/tag/commit
-y, --yes	Non-interactive consent

G1.2 Common options that apply everywhere

The following client flags are shared across the command families. They are not repeated in every section below; assume they are available unless a section says otherwise.

Shared client flags (Common Options reference)

Flag	Purpose	Notes
-d, --data-dir <path>	"Isolates all local state (config, context, db, logs, storage, secrets) away from ~/.paperclip"	Enables clean test instances
--api-base <url>	"Base URL for the Paperclip API. Highest-priority override"	Normalises trailing slashes
--api-key <token>	"Bearer token for authenticated calls. Highest-priority credential"	Disables interactive recovery
--context <path>	"Path to the CLI context file to read and write"	Defaults to ~/.paperclip/context.json
--profile <name>	"Which context profile to use"	Defaults to the context's current profile
-c, --config <path>	"Used only to infer a local server port"	When no API base is given

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

--json	"Emit raw JSON instead of the human-readable inline format"	For scripting and automation
-C, --company-id <id>	"Company ID. Overrides the profile's default company"	Company-scoped commands only

G1.3 How the CLI decides which server to talk to

When a command needs an API base URL and you have not supplied one, the CLI works down a fixed list. Knowing the order turns "it connected to the wrong instance" from a mystery into a two-second check.

- 1. The --api-base flag.
- 2. The PAPERCLIP_API_URL environment variable.
- 3. The profile's stored apiBase.
- 4. A locally inferred http://<host>:<port> — host from PAPERCLIP_SERVER_HOST, port from PAPERCLIP_SERVER_PORT or the config file.
- 5. The fallback http://localhost:3100.

Company scope resolves down a shorter list: --company-id or -C first, then PAPERCLIP_COMPANY_ID, then the profile's stored companyId.

G1.4 How the CLI decides which credential to use

Credentials resolve in their own documented order, and each source is labelled in the CLI's own authSource field, which is what you inspect when a call is authenticating as somebody you did not expect.

Credential resolution order

Priority	Source	authSource label
1	--api-key <token>	explicit
2	PAPERCLIP_API_KEY environment variable	env
3	Environment variable named by the profile's apiKeyEnvVarName	profile_env
4	Stored board credential for the resolved API base	stored_board

Note: From the Common Options page, and worth memorising: "The profile never stores a plaintext token. It stores the _name_ of an environment variable." A stolen context.json is therefore not a stolen credential.

G1.5 Context profiles

A profile bundles an API base, a company, a persona and the NAME of the environment variable holding the token. Profiles are how you keep a training instance and a production instance apart without retyping flags.

```
paperclipai context show
paperclipai context show --profile staging --json
paperclipai context list
paperclipai context use staging
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai context set --profile prod --api-base https://paperclip.example.com \
  --persona board --api-key-env-var-name PAPERCLIP_API_KEY --use
```

context set accepts --profile <name>, --api-base <url>, --company-id <id>, --persona <persona>, --agent-id <id>, --agent-name <name>, --api-key-env-var-name <name> and --use. Profile discovery walks --context flag → PAPERCLIP_CONTEXT environment variable → .paperclip/context.json walking upward → ~/.paperclip/context.json. The stored fields are apiBase, companyId, persona (board or agent), agentId, agentName and apiKeyEnvVarName.

G1.6 Output and scripting

Every command family documents --json, and the guidance is consistent across pages: use it whenever you script. The human-readable output is a compact summary designed for a terminal; the JSON output is the full record. The run reference gives the canonical scripting idiom.

```
paperclipai run live --json | jq '.[].id'
```

The cost reference makes the same point from the operations side: "Reads are cheap and idempotent. Wire cost summary, budget overview, and cost window-spend into a monitoring loop with --json and pipe the output into your own dashboards or alerting."

G1.7 Environment variables the CLI itself reads

CLI-facing environment variables

Variable	Effect
PAPERCLIP_API_URL	API base URL, second in the resolution order
PAPERCLIP_COMPANY_ID	Default company, second in the company-scope order
PAPERCLIP_AGENT_ID	Agent identity for agent-persona use
PAPERCLIP_API_KEY	Bearer credential, second in the credential order
PAPERCLIP_CONTEXT	Path to the CLI context file
PAPERCLIP_SERVER_HOST / PAPERCLIP_SERVER_PORT	Used for local API-base inference

Local config paths

Path	Holds
~/.paperclip	Default local config directory
~/.paperclip/context.json	Profiles storage
~/.paperclip/cli	Managed install store
~/.codex/skills	Codex skills installation
~/.claude/skills	Claude skills installation

G2. Setup and service — onboard, run, doctor, configure, env

This family gets an instance existing, running and healthy. It is the only family you use before you have any credential at all, because in the default local_trusted mode the board operator is implicitly trusted and no login is required.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G2.1 The one-command path

paperclipai run is the composite command: it bootstraps and starts a local server in one step. Its documented sequence is worth knowing because it explains why the first run takes so much longer than the second: it resolves the instance ID, creates directories, runs onboard if needed, executes doctor with repair enabled, starts the server, and generates a bootstrap CEO invite if the instance is in authenticated mode with an embedded database.

```
paperclipai run
paperclipai run --instance dev
paperclipai run --data-dir ./tmp/paperclip-dev
paperclipai run --no-repair
```

paperclipai run flags

Flag	Use
-c, --config <path>	Path to the config file
-d, --data-dir <path>	Isolate all local state from ~/.paperclip
-i, --instance <id>	Local instance id (default: default)
--bind <mode>	Onboarding reachability preset: loopback, lan, or tailnet
--no-repair	Disable automatic repairs during doctor
--force	Run even when the same instance is active under the service manager

Note: Documented warning: "before anything else, run checks whether this instance is already active under the service manager. If it is, run refuses to start — two servers on one port and one database is not a state you want."

G2.2 onboard

onboard is the interactive first-run setup. It writes the configuration to ~/.paperclip/instances/<id>/config.json, provisions PAPERCLIP_AGENT_JWT_SECRET in .env, and creates the local secrets key file.

```
paperclipai onboard
paperclipai onboard --yes
paperclipai onboard --run
paperclipai onboard --bind lan
```

paperclipai onboard flags

Flag	Use
-c, --config <path>	Path to the config file
-d, --data-dir <path>	Isolate all local state from ~/.paperclip
--bind <mode>	Quickstart reachability preset
-y, --yes	Accept Quickstart defaults non-interactively
--install-service	Install background service after onboarding
--no-install-service	Skip service installation
--run	Start server immediately after saving config

Note: Documented note: "If a valid config already exists, onboard preserves it unchanged, ensures the agent JWT secret and secrets key exist, and prints next-step commands. Use configure to change settings on an existing install rather than re-onboarding." And the --bind tip: it "only matters on the very first run, when it forwards the preset to onboarding."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G2.3 doctor

doctor is the diagnostic sweep, with optional repair. Reach for it before you reach for a firewall rule; most first-week failures are one of the checks it already runs.

```
paperclipai doctor
paperclipai doctor --repair
paperclipai doctor --repair --yes
```

paperclipai doctor flags

Flag	Use
-c, --config <path>	Path to the config file
-d, --data-dir <path>	Isolate all local state from ~/.paperclip
--repair (alias --fix)	Attempt to repair fixable issues
-y, --yes	Skip per-repair confirmation prompts

The documented check list is: config validity; deployment/auth mode compatibility; agent JWT secret; secrets adapter; storage; database connectivity; LLM provider; log directory; listen port; Node.js runtime (requires Node.js 20+); managed install (store, shim, PATH, payload retention); and background service (definition, runtime, health, version).

Note: Documented warning: "--repair can create or update local files (the JWT .env, the secrets key, the log directory) when a check knows how to fix the problem. Review the output before running it against a shared or production-like instance."

G2.4 env

env prints the environment variables a deployment needs, together with their sources. The output has two sections, Required and Optional, and marks each variable set, default or missing. The required variables are PAPERCLIP_AGENT_JWT_SECRET and DATABASE_URL.

```
paperclipai env
```

Flags: -c, --config <path> and -d, --data-dir <path>.

G2.5 configure

configure changes settings on an EXISTING install — the correct alternative to re-running onboard. It is section-driven.

```
paperclipai configure
paperclipai configure --section server
paperclipai configure --section database
paperclipai configure --section storage
paperclipai configure --section secrets
paperclipai configure --section logging
paperclipai configure --section llm
```

configure sections and what each changes

Section	Changes
server	Deployment mode, exposure, host binding, port, UI, auth base URL
database	Embedded vs. external PostgreSQL, connection string, backup settings

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

storage	Provider (local_disk or s3) and options
secrets	Provider, strict mode, local key file
logging	Logging mode and directory
llm	LLM provider and API key

Flags: -c, --config <path>, -d, --data-dir <path>, -s, --section <section>.

G2.6 Backups and hostname allowlisting

```
paperclipai db:backup
paperclipai db:backup --dir /backups/paperclip
paperclipai db:backup --retention-days 14
paperclipai db:backup --json
```

paperclipai db:backup flags

Flag	Use
--dir <path>	Backup output directory
--retention-days <days>	Retention window for daily pruning (default 30)
--filename-prefix <prefix>	Backup filename prefix (default paperclip)
--json	Print backup metadata as JSON

Note: Documented tip: "For the embedded database you can run a backup without the server up — db:backup connects on the embedded port directly."

```
paperclipai allowed-hostname dotta-macbook-pro
paperclipai allowed-hostname my-host.ts.net
```

The <host> argument is normalised and lowercased. Two constraints apply and both bite in the classroom: the change takes effect only after a server restart, and it is only enforced in authenticated + private mode.

G2.7 The background service

The documentation covers service installation from two angles. The installation guide shows the service install command alongside the managed install, and doctor checks the background service's definition, runtime, health and version. onboard can install or skip the service through --install-service and --no-install-service.

```
npx paperclipai install
paperclipai service install
npx paperclipai run          # restart
sudo journalctl -u paperclip -f
```

Note: The harvested documentation does not enumerate further service subcommands or their flags — a dedicated CLI service page exists in the navigation but its contents were not harvested. Do not assume service status, service logs or service uninstall exist in that exact form; check paperclipai --help on your own install before relying on them.

G2.8 Updates, rollback and release channels

The managed install path is the documented route to updates: npx paperclipai install "installs to ~/.local/bin/paperclipai with automatic updates and rollback", and doctor verifies the managed store, shim, PATH and payload retention. The docs changelog records four release channels — stable, beta, nightly and canary — surfaced by the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

paperclipai channels command, which "shows which one your install follows". The install command's own --canary, --version <version> and --ref <ref> flags pin a specific build.

Note: Since v2026.824.1, after onboard installs the background service it "waits for the port the service actually bound, prints the dashboard URL, and opens it in your browser. Headless runs print the URL, and PAPERCLIP_NO_BROWSER=1 opts out of the browser launch."

G2.9 Local instance layout

```
~/.paperclip/instances/<instance-id>/
|-- config.json          (Config)
|-- db/                  (Embedded database)
|-- logs/                (Logs)
|-- data/storage/        (Local disk storage)
`-- secrets/master.key   (Secrets key)
```

The default instance ID is default. Embedded PostgreSQL listens on port 54329 with data in ~/.paperclip/instances/default/db; the server's own default port is 3100, bound to 127.0.0.1.

Note: Three warnings from the installation guide, all of which have ended somebody's afternoon: "Do not run this command with sudo or from a root/admin shell" (PostgreSQL refuses to run as an administrative user); "Do not open port 3100 to the internet"; and "Back up secrets/master.key somewhere safe. It encrypts every API key and secret stored in Paperclip — if you lose it, you lose access to all of them."

G3. Authentication and tokens

Paperclip recognises two identity classes — BOARD (a human or operator) and AGENT (a runtime) — and the whole authentication surface follows from that split. Board identities are the ones that decide; agent identities are the ones that request. Every command in this section produces or inspects one of those two.

G3.1 The interactive wizard

connect is the TTY-only setup wizard. It is the shortest path from a fresh terminal to a working profile.

```
paperclipai connect
paperclipai connect --persona agent --token-name laptop-cli
```

Flags: --persona <persona>, --api-key-env-var-name <name>, --token-name <name>.

G3.2 Board login and identity

```
paperclipai auth login
paperclipai auth login --instance-admin
paperclipai auth login --company-id <company-id>
paperclipai auth login --no-browser

paperclipai auth whoami --json
paperclipai auth logout
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai auth revoke-current
```
auth login flags are --instance-admin, --company-id <id> and --no-browser. The login flow is a browser approval: the CLI raises a challenge and a browser session approves it. auth whoami is the fastest confirmation of which identity a credential actually resolves to, and is the first thing to run when a command behaves as though it belongs to somebody else.

G3.3 bootstrap-ceo — minting the first board credential

On a server install there is no board user yet, and no browser session to approve a login with. auth bootstrap-ceo is the headless escape hatch: it mints the first board credential and emits an invite link.

```
paperclipai auth bootstrap-ceo
paperclipai auth bootstrap-ceo --expires-hours 24
paperclipai auth bootstrap-ceo --force
paperclipai auth bootstrap-ceo --base-url https://paperclip.example.com
```

auth bootstrap-ceo flags

Flag	Use
--force	Create a new invite if an admin already exists
--expires-hours <hours>	Invite lifetime in hours (1-720; default 72)
--base-url <url>	Public base URL for the invite link
--db-url <URL>	Database URL (listed on the installation guide's flag list)

The output is an invite URL of the form https://paperclip.example.com/invite/<token>. In authenticated mode the equivalent board-claim URL takes the form /board-claim/<token>?code=<code>.

Note: Documented warning: "Treat the claim URL as sensitive — it's a one-time ownership transfer, not something to share. If it expires before you use it, restart the server to mint a fresh one."

G3.4 Board tokens

A board token carries full instance authority. Create one for automation that must act as an operator — a release bot, an external administration tool — and give it the shortest life the job allows.

```
paperclipai token board create --name external-admin
paperclipai token board create --name short-lived --ttl-days 7
paperclipai token board create --name release-bot --company-id <id> --expires-at
2026-12-31T00:00:00Z
paperclipai token board create --name forever --never-expires
paperclipai token board list
paperclipai token board revoke <key-id>
```
Flags for token board create: --company-id <id>, --name <name>, --expires-at <iso8601>, --ttl-days <days>, --never-expires. Board API keys carry the prefix pcp_board_ and are long-lived by design.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G3.5 Agent keys

An agent key is scoped to one company and one agent. It is what a runtime uses to talk back to the control plane as itself.

```
paperclipai token agent create --company-id <id> --agent <agent-id-or-name> --name
external-worker
paperclipai token agent list --company-id <id> --agent <agent-id-or-name>
paperclipai token agent revoke --company-id <id> --agent <agent-id-or-name> <key-id>
```

Flags: --company-id <id> (required), --agent <agent> (required), --name <name>.

G3.6 The once-only display rule

Note: From the CLI authentication reference, verbatim: "Both credential types are shown in full **only at creation time**. The token string is printed once and never again." The agent reference repeats it for the local-cli bootstrap: "The minted API key's plaintext token is shown **once**, in the command output. Treat the output as a secret, and revoke the key with token agent revoke when the local session is done." The API reference states the same rule for POST /api/agents/{agentId}/keys.

The consequence is unforgiving: lose the token and you do not recover it — you revoke the key and mint a new one. Capture the output into a secret store at the moment of creation, never into a scrollback buffer.

G3.7 Token types at a glance

Token types (API authentication reference)

Token type	Prefix	Purpose	Lifetime
Board API Key	pcp_board_	CLI / automation	Long-lived
Agent API Key	(none documented)	Agent requests	Long-lived
Local Agent JWT	(none documented)	Heartbeat / adapter code	Short-lived

The local agent JWT carries required claims sub (agent ID), company_id, adapter_type, run_id, iat and exp. It is accepted only when the signature is valid, it has not expired, the agent exists and is not terminated or pending_approval, and the agent belongs to the same company as the token claims.

Agent API key creation is REJECTED when the agent's status is pending_approval or terminated. Terminating an agent revokes all of its API keys.

G4. Company commands

A company is the top-level tenant boundary: "Every agent, project, issue, approval, cost event, and asset belongs to exactly one company, and the API enforces that boundary on every company-scoped route." The command family divides cleanly into safe reads, payload-driven writes, portability and — behind two separate guards — deletion.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G4.1 Reads

```
paperclipai company list
paperclipai company get <company-id>
paperclipai company current
paperclipai company stats
```

G4.2 Create and update

```
paperclipai company create --payload-json '{...}'
paperclipai company update <company-id> --payload-json '{...}'
paperclipai company branding:update <company-id> --payload-json '{...}'
```

The company object's documented fields are id, name, description, status (active, paused or archived), issuePrefix, issueCounter, budgetMonthlyCents, spentMonthlyCents, requireBoardApprovalForNewAgents, brandColor, logoAssetId, logoUrl, createdAt and updatedAt. On create, "If budgetMonthlyCents is greater than 0, Paperclip also creates the matching monthly budget policy in UTC calendar-month mode."

G4.3 Archive, export, import

```
paperclipai company archive <company-id>

paperclipai company export <company-id> --out ./path
paperclipai company import ./path --target new|existing

paperclipai company export:preview <company-id> --payload-json '{...}'
paperclipai company export:api <company-id> --payload-json '{...}'
paperclipai company import:preview <company-id> --payload-json '{...}'
paperclipai company import:apply <company-id> --payload-json '{...}'

paperclipai company feedback:list --company-id <id>
paperclipai company feedback:export --company-id <id>
```

Company command flags

Flag	Purpose
--payload-json	Required for create/update; raw JSON payload
--out <path>	Output directory for exports
--include <values>	Comma-separated: company,agents,projects,issues,skills
--yes	Safety flag for deletion
--confirm <value>	Must match the target company ID or prefix
--json	Machine-readable output

G4.4 Deletion

```
paperclipai company delete <company-id> --yes --confirm <value>
```

- "Archiving is the reversible way to take a company out of active rotation" — deletion is not.
- Deletion requires PAPERCLIP_ENABLE_COMPANY_DELETION enabled server-side.
- "There is no undo" for company deletion.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- IMPORTED AGENTS HAVE TIMER HEARTBEATS DISABLED BY DEFAULT — a restored company is quiet until you re-enable them, which is a feature, not a fault.

Note: Archiving is not passive. The documented cascade "pauses every active agent in the company (pause reason company_archived) and cancels their queued or in-flight wake-up requests."

G5. Project commands

Projects group work and carry two things agents care about: a lead agent and, optionally, an execution-workspace policy and an env binding. The command family is small, which makes its two idioms — shortname resolution and the literal "null" — worth learning properly.

G5.1 At a glance

```
paperclipai project list --company-id <company-id>
paperclipai project get <project-id-or-shortname> [--company-id <company-id>]
paperclipai project create --company-id <company-id> --name "Launch Site" [--goal-ids <id1,id2>] [--lead-agent-id <agent-id>]
paperclipai project update <project-id-or-shortname> --status in_progress [--company-id <company-id>]
paperclipai project delete <project-id-or-shortname> --yes [--company-id <company-id>]
```

G5.2 Create

```
paperclipai project create \
  --company-id <company-id> \
  --name "Launch Site" \
  --description "Public marketing launch" \
  --status planned \
  --goal-ids <goal-id-1>,<goal-id-2> \
  --lead-agent-id <agent-id> \
  --color "#3b82f6"
```

project create flags

Flag	Use
-C, --company-id <id>	Company to create the project in. Required.
--name <name>	Project name. Required.
--description <text>	Free-text description.
--status <status>	Initial project status.
--goal-ids <csv>	Comma-separated list of goal IDs to link.
--goal-id <id>	Deprecated single goal ID. Prefer --goal-ids.
--lead-agent-id <id>	Agent that leads the project.
--target-date <date>	Target/target-completion date.
--color <value>	Project color.
--env-json <json>	Project env binding as a JSON object.
--execution-workspace-policy-json <json>	Execution workspace policy as a JSON object.

Note: On --status values: the project reference documents the flag as "Initial project status" and shows planned and in_progress in its own worked examples. It does not publish a closed enumeration on that page, so treat any value beyond those two as unverified until your own instance's schema confirms it. The create payload "is validated against the server's create-project

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

schema before the request is sent, so an out-of-range value fails locally with a clear error rather than a server round-trip" — which is, conveniently, how you discover the valid set safely.

G5.3 Update, archive and the "null" idiom

```
paperclipai project update <project-id-or-shortname> --status in_progress
paperclipai project update launch-site --company-id <company-id> --name "Launch v2"

# Clear the lead agent and unarchive in one call
paperclipai project update <project-id> --lead-agent-id null --archived-at null

# Archive a project
paperclipai project update <project-id> --archived-at 2026-06-01T00:00:00Z
```

project update flags

Flag	Use
-C, --company-id <id>	Company ID for shortname lookup.
--name <name>	New project name.
--description <text|null>	New description, or null to clear it.
--status <status>	New project status.
--goal-ids <csv>	Replace the linked goal IDs with this comma-separated list.
--goal-id <id|null>	Deprecated single goal ID, or null to clear.
--lead-agent-id <id|null>	New lead agent, or null to clear.
--target-date <date|null>	New target date, or null to clear.
--color <value|null>	New color, or null to clear.
--env-json <json|null>	Replace the env binding, or null to clear it.
--execution-workspace-policy-json <json|null>	Replace the execution workspace policy, or null to clear it.
--archived-at <iso8601|null>	Archive timestamp, or null to unarchive.

The clearing rule is stated once and applies across the whole flag set: "pass the literal string null to clear the value, omit the flag to leave it unchanged." It applies to --description, --goal-id, --lead-agent-id, --target-date, --color, --env-json, --execution-workspace-policy-json and --archived-at. That is also the archiving semantics in full: archiving is a timestamp you set, and unarchiving is that same timestamp cleared with null. There is no separate archive subcommand for projects.

Note: Documented note on goals: "--goal-ids replaces the entire set of linked goals on the project. Pass the full list you want, not a delta."

G5.4 JSON fields

```
paperclipai project create \
  --company-id <company-id> \
  --name "Ops" \
  --env-json '{"OPENAI_API_KEY":{"kind":"secret","secretName":"openai-api-key"}}'

paperclipai project update <project-id> \
  --execution-workspace-policy-json
'{"enabled":true,"defaultMode":"shared_workspace"}'

paperclipai project update <project-id> --execution-workspace-policy-json null
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G5.5 Delete

```
paperclipai project delete <project-id-or-shortname> --yes
paperclipai project delete launch-site --company-id <company-id> --yes
```

- delete "refuses to run without --yes — the command fails fast with \"Deletion requires --yes.\" if the flag is missing."
- Documented warning: "Deleting a project is immediate once --yes is supplied. Confirm the reference resolves to the right project with project get before you delete it, especially when using a shortname."
- get, update and delete accept either the project UUID or its shortname. Pass --company-id when using a shortname so the server can resolve it; on get it "adds a companyId query parameter".
- Both JSON flags are parsed with JSON.parse client-side; invalid JSON fails locally with Invalid JSON: ... "before any request is made".

project list default (non-JSON) output fields

Field	Meaning
id	The project UUID.
name	The project's display name.
status	The project's current status.
urlKey	The project's stable shortname/URL key.
goalIds	Comma-separated goal IDs the project is linked to.
leadAgentId	The agent leading the project, if set.

G6. Issue commands

Issues are the unit of work, and the issue family is the largest in the CLI, presented here in the groups the documentation itself uses. Two things matter more than the command list: the checkout discipline and the status transitions. Get those wrong and you produce a race condition that is hard to debug after the fact.

G6.1 CRUD

```
paperclipai issue list -C <company-id> --status todo,in_review --assignee-agent-id
<agent-id> --match "billing"
paperclipai issue get PC-12
paperclipai issue create -C <company-id> --title "Fix invoice rounding" --priority
high --assignee-agent-id <agent-id>
paperclipai issue update <issue-id> --status in_review --comment "Ready for review"
paperclipai issue delete <issue-id> --yes
```

G6.2 Checkout and release — the discipline

```
paperclipai issue checkout <issue-id> --agent-id <agent-id>
paperclipai issue checkout <issue-id> --agent-id <agent-id> --expected-statuses
todo,in_review
paperclipai issue release <issue-id>
paperclipai issue force-release <issue-id>
```

Checkout is the ONLY correct way for an agent to claim an issue. The todo to in_progress transition is atomic, performed by POST /api/issues/{id}/checkout, and the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

body carries agentId plus expectedStatuses — a non-empty list of the statuses you are prepared to find. That makes the claim conditional: if the issue has moved since you read it, the claim fails rather than trampling somebody else's work.

The checkout rules that matter

Rule	Detail
Only one holder	"Only one agent can hold an issue 'in progress' at a time."
Agents claim as themselves	"An agent can only checkout as itself."
409 is terminal	Checkout returns 409 Conflict when the issue does not match one expectedStatuses value, when the project is paused, or when the workspace is a closed isolated workspace. "Never retry a 409 — pick a different issue."
Idempotent for the same holder	Checkout is idempotent if the same agent already owns the task.
Reclaim deliberately	To reclaim a stale in_progress issue, include in_progress in expectedStatuses with a NEW run id in X-Paperclip-Run-Id.
Run id on updates	Agent updates to checked-out in_progress issues require X-Paperclip-Run-Id.

release is the clean way to let go: it "Clears assigneeAgentId, checkoutRunId; sets status todo; preserves assigneeUserId." force-release exists for the case where the holder is not coming back.

G6.3 Status values and transitions

Issue status values

Status	Meaning	Terminal?
backlog	Parked, unscheduled; not in default inbox	No
todo	Ready and actionable; waiting for agent checkout	No
in_progress	Checked out by agent; actively executing; exclusive	No
in_review	Paused pending reviewer/approver/board/user	No
blocked	Cannot proceed until blocker resolved	No
done	Work complete	Yes
cancelled	Intentionally abandoned	Yes

State machine transitions

From	To	Triggered by
backlog	todo	Manual scheduling, ready-for-work signal
todo	in_progress	POST /api/issues/{id}/checkout (atomic)
in_progress	in_review	PATCH with status: "in_review"
in_progress	done	PATCH with status: "done" (sets completedAt)
in_progress	blocked	PATCH with status: "blocked" plus blocker comment or blockedByIssueIds

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

in_review	in_progress	Reviewer requested changes
in_review	done	Reviewer/approver advanced
blocked	todo	Blocker resolved (manual, release, or auto issue_blockers_resolved wake)
any non-terminal	cancelled	PATCH with status: "cancelled" (sets cancelledAt)
done / cancelled	todo	PATCH with reopen: true (the ONLY path back from terminal)

Note: The single most common transition error: sending PATCH status: "todo" on a done issue. Terminal transitions require reopen. Send PATCH { reopen: true, comment: "..." } instead. And note that cancelled blockers do NOT count as resolved — replace or remove the cancelled id from blockedByIssueIds explicitly.

G6.4 Comments

```
paperclipai issue comment <issue-id> --body "Picking this up now"
paperclipai issue comment <issue-id> --body "Re-opening, the fix regressed" --reopen
paperclipai issue comment <issue-id> --body "Any update?" --resume
paperclipai issue comments <issue-id> --order asc --limit 50
paperclipai issue comment:get <issue-id> <comment-id>
paperclipai issue comment:delete <issue-id> <comment-id>
```

Comment listing accepts after / afterCommentId, order (asc or desc) and limit, which is capped at 500. Comment bodies are markdown. @mentions in a comment trigger wakeups for matching agents — the syntax is @AgentName, case-insensitive, and it must match the agent's name field exactly. Two documented cautions on that: "Don't overuse mentions — each mention triggers a budget-consuming heartbeat" and "Don't use mentions for assignment — create or assign a task instead."

G6.5 Documents, work products and attachments

```
paperclipai issue documents <issue-id> --include-system
paperclipai issue document:get <issue-id> spec
paperclipai issue document:put <issue-id> spec --title "Spec" --body-file ./spec.md
--change-summary "Initial draft"
paperclipai issue document:lock <issue-id> spec
paperclipai issue document:unlock <issue-id> spec
paperclipai issue document:revisions <issue-id> spec
paperclipai issue document:restore <issue-id> spec <revision-id>
paperclipai issue document:delete <issue-id> <key>

paperclipai issue work-products <issue-id>
paperclipai issue work-product:create <issue-id> --payload-json
'{"kind":"report","title":"Q2 analysis"}'
paperclipai issue work-product:update <work-product-id> --payload-json '{"title":"Q2
analysis (final)"}'
paperclipai issue work-product:delete <work-product-id>

paperclipai issue attachments <issue-id>
paperclipai issue attachment:upload <issue-id> -C <company-id> --file ./diagram.png
paperclipai issue attachment:upload <issue-id> -C <company-id> --file ./log.txt --
comment-id <comment-id>
paperclipai issue attachment:download <attachment-id> --out ./diagram.png
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai issue attachment:delete <attachment-id>
```

Document bodies are markdown only, up to 512 KiB, and updates require a baseRevisionId. A stale baseRevisionId returns 409 Conflict with the current revision id, and an existing key sent without baseRevisionId is rejected outright. Restoring a revision "does not overwrite history. It creates a new revision that becomes the latest body." Document keys must be lowercase, with numbers, underscore and hyphen allowed.

Note: Locked documents behave differently by caller class. A USER caller gets 409 Conflict with { "error": "Document is locked", "key": "...", "lockedAt": "..." }. An AGENT caller has its write routed to a derived document — plan-2 if plan is locked — and the response carries redirectedFromLockedDocument.

G6.6 Interactions, labels, approvals and inbox state

```
paperclipai issue interactions <issue-id>
paperclipai issue interaction:create <issue-id> --payload-json '{ ... }'
paperclipai issue interaction:accept <issue-id> <interaction-id> --selected-client-keys a,b
paperclipai issue interaction:reject <issue-id> <interaction-id> --reason "Out of scope"
paperclipai issue interaction:cancel <issue-id> <interaction-id> --reason "No longer needed"
paperclipai issue interaction:respond <issue-id> <interaction-id> --answers-json '[{"key":"q1","value":"yes"}]' --summary-markdown "Confirmed scope"

paperclipai issue label:list -C <company-id>
paperclipai issue label:create -C <company-id> --name "needs-review" --color "#4f46e5"
paperclipai issue label:delete <label-id>

paperclipai issue approvals <issue-id>
paperclipai issue approval:link <issue-id> <approval-id>
paperclipai issue approval:unlink <issue-id> <approval-id>

paperclipai issue read <issue-id>
paperclipai issue unread <issue-id>
paperclipai issue archive <issue-id>
paperclipai issue unarchive <issue-id>
```

Interaction kinds and when to use each

Kind	When to use
suggest_tasks	Agent identified next work; board chooses which subtasks to spin up
ask_user_questions	Agent needs structured info (multiple choice, short text)
request_confirmation	Agent has a proposal (plan revision, destructive action); needs explicit acceptance
request_checkbox_confirmation	Agent needs a decision over explicitly listed checkbox options
request_item_verdicts	Agent needs a verdict for each item in a supplied list

Pending interactions expire automatically when an issue moves to done or cancelled: status becomes expired with result.outcome: "issue_closed". Creating a new card on a

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

closed issue returns 409 "Cannot create an interaction on a closed issue". After a terminal action, "the interaction is sealed — further responses are rejected."

G6.7 Trees, holds, recovery and runs

```
paperclipai issue tree-state <root-issue-id>
paperclipai issue tree-preview <root-issue-id> --payload-json '{"mode":"pause"}'
paperclipai issue tree-holds <root-issue-id> --status active --mode pause --include-
members
paperclipai issue tree-hold:create <root-issue-id> --payload-json
'{"mode":"pause","reason":"awaiting budget"}'
paperclipai issue tree-hold:get <root-issue-id> <hold-id>
paperclipai issue tree-hold:release <root-issue-id> <hold-id> --payload-json '{}'

paperclipai issue recovery-actions <issue-id>
paperclipai issue recovery:resolve <issue-id> --outcome restored --source-issue-
status todo --resolution-note "Re-queued"
paperclipai issue runs <issue-id>
paperclipai issue live-runs <issue-id>
paperclipai issue active-run <issue-id>
paperclipai issue child:create <parent-issue-id> --payload-json
'{"title":"Subtask"}'

paperclipai issue feedback:votes <issue-id>
paperclipai issue feedback:vote <issue-id> --payload-json
'{"targetType":"comment","targetId":"<id>","vote":"up"}'
paperclipai issue feedback:list <issue-id> --vote down --status open --include-
payload
paperclipai issue feedback:export <issue-id> --from 2026-01-01 --format ndjson --out
./feedback.ndjson

paperclipai issue heartbeat-context <issueId>
```

G6.8 The cross-issue influence cap

Note: CROSS_ISSUE_INFLUENCE_LIMIT = 20. "A single heartbeat run may make at most CROSS_ISSUE_INFLUENCE_LIMIT = 20 cross-issue comments or task updates combined — writes to run's own source issue don't count." The budget resets per run. Before CROSS_ISSUE_INFLUENCE_ENFORCE_AT = 2026-08-11T00:00:00.000Z the mode was log_only; from then on it is hard-enforced with 429 and the code cross_issue_influence_cap_exceeded.

Denial codes for cross-issue writes

Code	Status	Meaning
issue_write_not_visible	403	Target issue not visible; all write channels closed
issue_write_actor_class_excluded	403	Default-open writes are a standard-trust privilege
issue_write_responsible_user_ceiling	403	Responsible user not authorised
issue_write_responsible_user_unavailable	403	Responsible user removed/deactivated
issue_write_assignee_run_lock	409	Assignee checked out; field edits belong to the lock-holder
cross_issue_influence_cap_exce	429	Per-run cross-issue write budget

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

eded		spent
cross_issue_influence_run_context_required	403	Write arrived without a valid heartbeat run
issue_write_attribution_spoof_rejected	422	onBehalfOfUserId derived from actor, never from body

G7. Agent commands

The agent family covers the whole employment lifecycle — identity, hiring, configuration, lifecycle transitions and the local bootstrap that turns a laptop CLI into a working agent runtime. Note the distinction the documentation draws between create and hire: they are different governance paths, not synonyms.

G7.1 Identity and inbox

```
paperclipai agent me
paperclipai agent inbox
paperclipai agent inbox-mine --user-id <board-user-id> --status todo,in_progress
```

Identity commands

Command	Use
agent me	"Show the current agent identity (GET /api/agents/me). The fastest way to confirm which agent a key or profile resolves to."
agent inbox	"List the current agent's assigned inbox items — a compact view of issues awaiting it."
agent inbox-mine --user-id <id>	"List the current agent's inbox items touched or archived by a specific board user. --status <csv> narrows by issue status (for example todo,in_progress)."

G7.2 Listing, creating and hiring

```
paperclipai agent list --company-id <company-id>
paperclipai agent get <agent-id>

paperclipai agent create --company-id <company-id> \
  --payload-json '{"name":"Builder","adapterType":"codex_local"}'

paperclipai agent hire --company-id <company-id> \
  --payload-json '{"name":"CTO","role":"cto","reason":"Lead engineering"}'
```

create versus hire

Command	Use
agent create	"Create an agent directly from a CreateAgent JSON payload (POST /api/companies/{id}/agents). Use this when you have board authority and want the agent to exist immediately."
agent hire	"Create an agent _hire request_ (POST /api/companies/{id}/agent-hires). Use this when hiring is governed and must pass through the board approval flow rather than landing as a live agent."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Common agent create/update fields are name, role, title, reportsTo, capabilities, adapterType, adapterConfig, runtimeConfig (heartbeat settings), budgetMonthlyCents, desiredSkills, metadata, and permissions (through the dedicated PATCH only).

G7.3 Update, delete and lifecycle

```
paperclipai agent update <agent-id> --payload-json '{"title":"Senior Builder"}'
paperclipai agent delete <agent-id> --yes

paperclipai agent pause <agent-id>
paperclipai agent resume <agent-id>
paperclipai agent approve <agent-id>
paperclipai agent terminate <agent-id>
```

The documentation is explicit about which of the two ends an agent's employment: "agent terminate: Terminate an agent. Use this to retire a worker rather than deleting its record." Termination revokes all of the agent's API keys, and "Terminated agents are hidden from list and org-chart responses" though they remain fetchable by direct ID. Deletion is a hard delete with runtime-state cleanup.

Note: Pause and heartbeat-off are NOT the same thing, and confusing them produces an agent that appears broken. With HEARTBEAT OFF, timer ticks stop but the agent still wakes on assignment, on comment, on routine and on demand, and still shows as active. With PAUSED, nothing wakes it at all and it does not show as active.

G7.4 Waking an agent

```
paperclipai agent wake <agent-id-or-shortname> \
  --company-id <company-id> \
  --reason "Pick up the new spec" \
  --payload '{"issueId":"<issue-id>"}'
```

agent wake flags

Flag	Use
--source <source>	"Invocation source: timer, assignment, on_demand, or automation. Defaults to on_demand."
--trigger <trigger>	"Trigger detail: manual, ping, callback, or system. Defaults to manual."
--reason <text>	"Human-readable reason for the wakeup."
--payload <json>	"A JSON object passed to the run. Anything that is not a JSON object is rejected."
--idempotency-key <key>	"Dedupe key so a re-sent wakeup does not double-fire."
--force-fresh-session	"Request a fresh adapter session instead of resuming the existing one."

```
paperclipai agent heartbeat:invoke <agent-id>
paperclipai agent claude-login <agent-id>
```

G7.5 Permissions, configuration and revisions

```
paperclipai agent permissions:update <agent-id> \
  --payload-json '{"canCreateAgents":true,"canAssignTasks":true}'

paperclipai agent configuration <agent-id>
paperclipai agent config-revisions <agent-id>
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai agent config-revision:get <agent-id> <revision-id>
paperclipai agent config-revision:rollback <agent-id> <revision-id>
```

agent configuration returns the REDACTED configuration: "Secrets are masked, so this is safe to print and share." Configuration revisions record the timestamp, changed keys and diffs, with up to the ten most recent shown; rollback restores the exact prior configuration atomically, and "Rollbacks themselves saved as new revisions." The documented permission keys are agents:create, tasks:assign, tasks:assign_scope, environments:manage, users:invite, users:manage_permissions and joins:approve.

G7.6 Runtime state, sessions, skills and instructions

```
paperclipai agent runtime-state <agent-id>
paperclipai agent runtime-state:reset-session <agent-id> --task-key <key>
paperclipai agent task-sessions <agent-id>

paperclipai agent skills <agent-id>
paperclipai agent skills:sync <agent-id> --desired-skills paperclip,github

paperclipai agent instructions-path:update <agent-id> \
  --payload-json
'{"path":"/tmp/AGENTS.md","adapterConfigKey":"instructionsFilePath"}'

paperclipai agent instructions-bundle <agent-id>
paperclipai agent instructions-bundle:update <agent-id> --payload-json
'{"mode":"managed"}'

paperclipai agent instructions-file:get <agent-id> --path AGENTS.md
paperclipai agent instructions-file:put <agent-id> --path AGENTS.md --content-
file ./AGENTS.md
paperclipai agent instructions-file:delete <agent-id> --path AGENTS.md
```

G7.7 local-cli — the laptop bootstrap and the once-only key

local-cli is the command that turns a local Claude Code or Codex install into a Paperclip agent runtime. Its documented process is: "Mints a long-lived agent API key, installs bundled Paperclip skills into ~/.codex/skills and ~/.claude/skills as symlinks, prints shell export lines."

```
paperclipai agent local-cli <agent-id-or-shortname> --company-id <company-id>
paperclipai agent local-cli codexcoder --company-id <company-id> --key-name
workstation
paperclipai agent local-cli claudecoder --company-id <company-id> --no-install-
skills
```

agent local-cli flags

Flag	Use
-C, --company-id <id>	"Required. The company the agent belongs to."
--key-name <name>	"Label for the created API key. Defaults to local-cli."
--no-install-skills	"Skip installing skills into ~/.codex/skills and ~/.claude/skills."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The once-only rule again, because this is where learners most often lose a key: "The minted API key's plaintext token is shown **once**, in the command output. Treat the output as a secret, and revoke the key with token agent revoke when the local session is done."

G8. Run commands — triggering and inspecting heartbeats

"A _heartbeat run_ is a single server-side execution of an agent." The run family is where you go when you want to know what an agent actually did, as opposed to what it reported. Remember that these are observation commands: "Agents do not run inside the CLI."

G8.1 Listing and watching

```
paperclipai run list --company-id <company-id>
paperclipai run list --company-id <company-id> --agent-id <agent-id> --limit 50

paperclipai run live --company-id <company-id>
paperclipai run live --company-id <company-id> --limit 50 --min-count 5
```

run list accepts --agent-id <id> and --limit <n>. run live accepts --limit <n> and --min-count <n>. In text mode both print id, status, agentId, invocationSource, triggerDetail, startedAt, finishedAt and logBytes.

G8.2 Inspecting one run

```
paperclipai run get <run-id>
paperclipai run events <run-id> --after-seq 0 --limit 200

paperclipai run log <run-id> --offset 0 --limit-bytes 16384
paperclipai run log <run-id> --text

paperclipai run issues <run-id>
paperclipai run cancel <run-id>
paperclipai run workspace-operations <run-id>
paperclipai run workspace-log <operation-id> --offset 0 --limit-bytes 16384
paperclipai run workspace-log <operation-id> --text
```

run events accepts --after-seq <n> (default 0) and --limit <n> (default 200), and prints seq, eventType, stream, level and message. run log accepts --offset <bytes> (default 0), --limit-bytes <bytes> and --text. run workspace-operations prints id, status, phase, command, cwd and logBytes.

Note: Documented, and important when an agent has gone wrong: "Cancelling a run does not roll back what the agent already did." Cancellation stops the execution; it does not undo the files written, the comments posted or the money spent.

G8.3 Watchdog decisions

```
paperclipai run watchdog-decision <run-id> --decision continue --reason "text"
paperclipai run watchdog-decision <run-id> --decision snooze --snoozed-until 2026-06-01T18:00:00Z
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai run watchdog-decision <run-id> --decision dismissed_false_positive --
evaluation-issue-id <id>
```

G8.4 Triggering a single heartbeat and watching it

The setup-commands page documents a dedicated command for the case where you want to watch ONE run unfold rather than fire and forget.

```
paperclipai heartbeat run --agent-id <agent-id>
paperclipai heartbeat run --agent-id <agent-id> --source on_demand --trigger manual
paperclipai heartbeat run --agent-id <agent-id> --timeout-ms 120000 --debug
```

paperclipai heartbeat run flags

Flag	Use
-a, --agent-id <agentId>	Required. Agent to invoke
--source <source>	timer, assignment, on_demand, automation (default on_demand)
--trigger <trigger>	manual, ping, callback, system (default manual)
--timeout-ms <ms>	Max wait time before timeout (default 0 = no client-side timeout)
--json	Output raw JSON
--debug	Show raw adapter stdout/stderr JSON chunks

Note: Documented tip: "For routine agent wake-ups during normal operation, prefer agent wake and the prompt-handoff flow in prompt. Reach for heartbeat run when you specifically want to watch a single run unfold."

Common flags across all run subcommands are --data-dir <path>, --api-base <url>, --api-key <token>, --context <path>, --profile <name>, --json, and -C, --company-id <id> for list and live.

G9. Routine commands

Routines are recurring work that happens without anyone asking. The framing is the documentation's own: "A routine does not do the work itself. It creates a run, and that run usually creates or links to an execution issue for the assigned agent." These commands create, configure, fire and observe routines through the API — they do not execute the work themselves.

G9.1 Two command groups — routine and routines

The singular and plural groups are NOT the same thing

Group	Talks to	Use it when
routine (singular)	The control-plane REST API	Day-to-day: list, create, update, fire, and inspect routines and their triggers.
routines (plural)	The configured local database directly	Local maintenance only — pausing every routine in one company on a local instance.

G9.2 The routine family

```
paperclipai routine list --company-id <company-id>
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai routine list --company-id <company-id> --project-id <project-id>

paperclipai routine get <routine-id>

paperclipai routine create --company-id <company-id> --payload-json '{
  "name": "Nightly status digest",
  "description": "Summarize the day's closed issues",
  "status": "active"
}'

paperclipai routine update <routine-id> --payload-json '{"status": "paused"}'

paperclipai routine revisions <routine-id>
paperclipai routine revision:restore <routine-id> <revision-id>

paperclipai routine runs <routine-id>
paperclipai routine runs <routine-id> --limit 50

paperclipai routine run <routine-id>
paperclipai routine run <routine-id> --payload-json '{"reason": "manual smoke
test"}'
```

routine flags

Command	Flag	Use
routine list	-C, --company-id <id>	Company to list routines for. Required (falls back to your context's company).
routine list	--project-id <id>	Filter to routines attached to a single project.
routine create	-C, --company-id <id>	Company to create the routine in. Required.
routine create	--payload-json <json>	Routine definition as a JSON string. Required.
routine update	--payload-json <json>	Fields to update as a JSON string. Required.
routine runs	--limit <n>	Cap the number of runs returned. Omit for the server default.
routine run	--payload-json <json>	Run input as a JSON string. Defaults to {}.

G9.3 Triggers

```
paperclipai routine trigger:create <routine-id> --payload-json '{
  "kind": "schedule",
  "schedule": "0 7 * * *"
}'
paperclipai routine trigger:update <trigger-id> --payload-json '{"schedule": "0 */2
* * *"}'
paperclipai routine trigger:delete <trigger-id>
paperclipai routine trigger:rotate-secret <trigger-id>
paperclipai routine trigger:fire <public-id>
paperclipai routine trigger:fire <public-id> --payload-json '{"event":
"deploy.finished"}'
```

Trigger kinds

Trigger kind	Fires the routine when...	Notes
schedule	A cron/recurrence schedule is	The standard "run nightly / hourly

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	due.	/ weekly" trigger. Its behaviour under overlap and missed windows is governed by concurrency and catch-up.
webhook	An external system POSTs to the trigger's webhook URL.	Carries a secret you can rotate. Use it to turn inbound events into Paperclip work.
api (public)	A caller fires its public endpoint by publicId.	Fired with routine trigger:fire; the public id is shareable without exposing the routine id.

Note: An argument trap worth circling: trigger:update, trigger:delete and trigger:rotate-secret take the TRIGGER id, not the routine id. trigger:fire takes the publicId. And routine revision:restore takes its arguments positionally in order — routine id first, then revision id.

G9.4 The payload-json shape

The routine CLI passes --payload-json straight through to the REST API, so the API reference is the authority on the fields it accepts.

POST /api/companies/{companyId}/routines body

Field	Required	Notes
title	yes	Routine name. Can use routine variables in the text.
description	no	Human-readable routine description. Can also use variables.
assigneeAgentId	yes	Agent who receives each run.
projectId	no	Project to associate with the routine.
goalId	no	Goal to link routine-created work to.
parentIssueId	no	Parent issue for routine-created execution issues.
priority	no	critical, high, medium default, low.
status	no	Defaults to active when an assignee exists, otherwise paused.
concurrencyPolicy	no	Defaults to coalesce_if_active.
catchUpPolicy	no	Defaults to skip_missed.
activityGatePolicy	no	always (default) or require_external_activity.
activityGateScope	no	company (default) or project. Only used when the policy is require_external_activity.
variables	no	Template variables for the routine title, description, and run payloads.

POST /api/routines/{routineId}/run body

Field	Required	Notes
source	no	Defaults to manual. The code also accepts api.
triggerId	no	If present, it must belong to the routine and be enabled.
payload	no	Arbitrary JSON payload stored

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		on the run.
variables	no	Explicit variable overrides.
projectId	no	Overrides the routine's project for this run.
assigneeAgentId	no	Overrides the routine's default assignee for this run.
idempotencyKey	no	Deduplicates repeated requests with the same source/trigger combination.
executionWorkspaceId	no	Optional workspace binding for the created issue.
executionWorkspacePreference	no	Optional workspace preference for the created issue.
executionWorkspaceSettings	no	Optional workspace settings object for the created issue.

```
// schedule trigger payload
{
  "kind": "schedule",
  "label": "Monday morning",
  "enabled": true,
  "cronExpression": "0 9 * * 1",
  "timezone": "Europe/Amsterdam"
}

// webhook trigger payload
{
  "kind": "webhook",
  "label": "Stripe event bridge",
  "enabled": true,
  "signingMode": "bearer",
  "replayWindowSec": 300
}

// api trigger payload
{
  "kind": "api"
}
```

G9.5 Statuses and run outcomes

Routine status values

Status	Meaning
active	The routine can fire and create runs.
paused	The routine is stored but does not fire automatically.
archived	The routine is retired. Archived routines do not fire.

Routine run statuses

Status	Meaning
received	The run was accepted and is being processed.
coalesced	A live execution already existed, so this run linked to it.
skipped	The run did not create work. Read failureReason to tell the causes apart.
issue_created	A new execution issue was created.
completed	The execution issue later moved to done.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

failed	The execution issue failed, was cancelled, or the dispatch failed.

G9.6 disable-all — the plural group

```
paperclipai routines disable-all --company-id <company-id>
paperclipai routines disable-all --company-id <company-id> --json
```

routines disable-all flags

Flag	Use
-C, --company-id <id>	Company whose routines to pause. Required (falls back to PAPERCLIP_COMPANY_ID).
-c, --config <path>	Path to the Paperclip config file, if not the default.
-d, --data-dir <path>	Paperclip data directory root, to isolate state from ~/.paperclip.
--json	Output the raw result object instead of a human summary.

```
Paused 3 routine(s) for company <company-id> (1 already paused, 0 archived).
```

With --json the command returns companyId, totalRoutines, pausedCount, alreadyPausedCount and archivedCount. Its precise behaviour is: reads every routine for the company from the LOCAL database; sets each routine whose status is not already paused or archived to paused; leaves already-paused and archived routines untouched.

Note: Documented warning, and the reason this command belongs in a reference rather than a habit: "disable-all operates on the **local** instance's database... It is not the way to pause routines on a remote control plane — for that, pause each routine with routine update <routine-id> --payload-json '{"status":"paused"}'. There is no bulk re-enable command: turn routines back on individually with routine update once you have resolved the problem."

G9.7 Revisions, limits and the rules that surprise people

- "Every update creates a new revision."
- revision:restore "does not rewind history — it makes the chosen revision the current definition by creating a new latest revision from it, so the rollback is itself auditable."
- trigger:rotate-secret: "The old secret stops working once rotated, so update the caller that posts to the webhook in the same change."
- GET /api/routines/{routineId}/runs: "The limit defaults to 50 and is capped at 200."
- Replay windows "must be between 30 and 86,400 seconds. The default is 300 seconds."
- POST /api/routines/{routineId}/run returns 202 Accepted.
- "You cannot have an active routine without an assignee agent." The API normalises to paused or rejects the activation.
- "Agents can only create routines assigned to themselves."
- "Schedule triggers only work if the routine's required variables can be resolved."
- The webhook trigger's webhookUrl and webhookSecret are "only shown to you when the trigger is created or when the secret is rotated."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Webhook signing modes are bearer, hmac_sha256, github_hmac and none; "The default is bearer." The accepted headers per mode are: bearer uses Authorization: Bearer <secret>; hmac_sha256 uses X-Paperclip-Signature or X-Hub-Signature-256; github_hmac uses X-Hub-Signature-256 or X-Paperclip-Signature plus X-Paperclip-Timestamp; none requires no signature.

Note: The activity gate has one behaviour that reads as a bug until you know it: "A routine set to activityGateScope: "project" without a projectId can never match anything, so it will never fire on a schedule. Set a project first." The gate is also "only consulted for scheduled ticks. Webhook firings and manual runs through POST /api/routines/{routineId}/run dispatch regardless of the policy."

The documented wiring order is: 1. create the routine; 2. attach a schedule or webhook trigger; 3. inspect the created webhook material if needed; 4. list runs to confirm the routine is firing as expected. Routine-created issues are found by filtering issues on originKind=routine_execution.

G10. Approval commands

"Approvals are the governance layer that sits between what an agent _wants_ to do and what it is _allowed_ to do." The family is small and the semantics are the whole point: three different ways of saying no, only one of which is final.

Note: Documented note on persona: "Approvals are decided with a **board** persona. An agent persona is scoped to one company and one agent and is the side that _requests_ an approval; clearing the queue is board work."

G10.1 The command set

Approval commands

Command	What it does
approval list	List approvals for a company, optionally filtered by status.
approval get	Fetch one approval by ID.
approval create	Create an approval request for a governed action.
approval approve	Approve a pending request.
approval reject	Reject a request.
approval request-revision	Send the request back to the requester for changes.
approval resubmit	Resubmit a request, optionally with a new payload.
approval comment	Add a comment to an approval.

```
paperclipai approval list --company-id <company-id>
paperclipai approval list --company-id <company-id> --status pending
paperclipai approval list --company-id <company-id> --status pending --json

paperclipai approval get <approval-id>

paperclipai approval create \
  --company-id <company-id> \
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  --type hire_agent \
  --payload '{"name":"CTO","role":"engineering_lead"}' \
  --requested-by-agent-id <agent-id> \
  --issue-ids <issue-id-1>,<issue-id-2>

paperclipai approval approve <approval-id> --decision-note "Cleared budget; proceed
with the hire."
paperclipai approval reject <approval-id> --decision-note "Out of scope for this
quarter."
paperclipai approval request-revision <approval-id> --decision-note "Tighten the
role to backend only, then resubmit."

# Resubmit unchanged
paperclipai approval resubmit <approval-id>
# Resubmit with a corrected payload
paperclipai approval resubmit <approval-id> --payload
'{"name":"CTO","role":"backend_lead"}'

paperclipai approval comment <approval-id> --body "What is the projected monthly
cost of this hire?"
```

G10.2 Flags

approval create flags

Flag	Use
-C, --company-id <id>	Required. The company the approval belongs to.
--type <type>	Required. The approval type: hire_agent or approve_ceo_strategy.
--payload <json>	Required. The approval payload as a JSON object (not an array or scalar).
--requested-by-agent-id <id>	The agent making the request.
--issue-ids <csv>	Comma-separated issue IDs to link to the approval.

Decision flags (shared by approve, reject, request-revision)

Flag	Use
--decision-note <text>	A short note explaining the decision.
--decided-by-user-id <id>	The user ID to attribute the decision to.

approval resubmit takes --payload <json>, "An optional replacement payload, as a JSON object." approval comment takes --body <text>, "Required. The comment text." approval list takes -C, --company-id <id> (required) and --status <status>, which filters to a single status such as pending.

G10.3 The worked triage sequence

The documentation gives a complete triage loop. Learn this shape; it is the sequence you will actually run every morning.

```
# 1. See what is waiting
paperclipai approval list --company-id <company-id> --status pending

# 2. Inspect the one that needs attention
paperclipai approval get <approval-id> --json
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
# 3a. If it's good, approve it
paperclipai approval approve <approval-id> --decision-note "Approved."

# 3b. If it needs work, send it back
paperclipai approval request-revision <approval-id> --decision-note "Narrow the
scope, then resubmit."

# 4. The requester fixes and resubmits
paperclipai approval resubmit <approval-id> --payload '{"...":"..."}'
```

Which decision to reach for

Command	When to reach for it
approve	The request is sound and you authorise the action.
reject	The request should not proceed at all.
request-revision	The request is close but needs changes — sends it back to the requester rather than killing it.

Note: Documented tips: "Start every triage session with approval list --status pending. It is the shortest route from 'what needs me' to a decision." And: "A short, concrete decision note is worth more than a long one. The requester should be able to act on your feedback without guessing — that is the entire point of request-revision." Use comments to negotiate details before you decide; reserve request-revision for the moment you actually want the proposal changed.

G10.4 Types, state machine and gotchas

Approval types (API enum)

Type	Meaning
hire_agent	New agent hire or pending activation needs board review
approve_ceo_strategy	CEO strategy proposal requires approval before execution
budget_override_required	Budget action needs board intervention
request_board_approval	General agent/workflow request for board review

The CLI approval page documents only two values on --type: hire_agent and approve_ceo_strategy. The other two appear in the API's approval-type enum. The state machine is: pending -> approved / rejected / revision_requested -> pending.

- Scope: "The company-scoped subcommands (list, create) additionally require -C, --company-id. Single-approval subcommands operate on an approval ID and do not need a company." approval get is "company-agnostic — the approval ID is globally addressable".
- --payload "is parsed as JSON and must be a JSON object — passing a list or a bare string is rejected with an Invalid payload JSON error."
- --issue-ids "is split on commas, trimmed, and blank entries dropped."
- Documented warning: "Quote the --payload value as a single shell argument and keep the JSON valid. A truncated or array-shaped payload fails before the request ever reaches the server."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Board-only endpoints: approve, reject, request-revision. Agents cannot resubmit others' approvals.
- "Approvals don't expire on their own. If an approval is pending, agents that are waiting on it will pause at that point until you take action."

G11. Cost, finance and budget commands

There is no separate budget reference page — budget commands live on the cost page, and the three groups are designed to be used together. The framing sentence is worth quoting in full: "This is the single most important part of running a company headlessly: set budgets before you turn agents loose, not after."

G11.1 Cost reads

```
paperclipai cost summary --company-id <company-id>
paperclipai cost by-agent --company-id <company-id>
paperclipai cost by-agent-model --company-id <company-id>
paperclipai cost by-provider --company-id <company-id>
paperclipai cost by-biller --company-id <company-id>
paperclipai cost by-project --company-id <company-id>
paperclipai cost window-spend --company-id <company-id>
paperclipai cost quota-windows --company-id <company-id>

paperclipai cost issue <issue-id>
paperclipai cost event:create --company-id <company-id> --payload-json '{...}'
```

What each cost command answers

Command	Answers
cost summary	The headline total for the company. Start here.
cost by-agent	Which agents are spending. Use it to spot a runaway agent.
cost by-agent-model	Spend per agent broken down by the model each one used. The finest view for tuning model choices.
cost by-provider	Spend grouped by upstream provider.
cost by-biller	Spend grouped by who is billed.
cost by-project	Spend grouped by project, so you can attribute cost to the work that incurred it.
cost window-spend	Spend inside the current rolling budget windows — the number a budget policy actually checks against.
cost quota-windows	The configured quota windows themselves (their periods and limits).

G11.2 Finance

```
paperclipai finance event:create --company-id <company-id> --payload-json '{...}'
paperclipai finance events --company-id <company-id>
paperclipai finance summary --company-id <company-id>
paperclipai finance by-biller --company-id <company-id>
paperclipai finance by-kind --company-id <company-id>
```

Finance commands

Command	Use
finance event:create	Record a finance event. --payload-json is required

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	and POSTed to the company's finance-events endpoint.
finance events	List the recorded finance events for the company.
finance summary	The aggregate finance picture for the company.
finance by-biller	Finance totals grouped by biller.
finance by-kind	Finance totals grouped by event kind.

G11.3 Budget

```
paperclipai budget overview --company-id <company-id>
paperclipai budget policy:upsert --company-id <company-id> --payload-json '{...}'
paperclipai budget company:update --company-id <company-id> --payload-json
'{"...":"..."}'
paperclipai budget agent:update <agent-id> --payload-json '{"...":"..."}'
paperclipai budget incident:resolve <incident-id> --company-id <company-id>
paperclipai budget incident:resolve <incident-id> --company-id <company-id> --
payload-json '{"...":"..."}'
```

Which endpoint each budget command reaches

Command	Underlying call
cost event:create	POSTs to the company's cost-events endpoint.
finance event:create	POSTs to the company's finance-events endpoint.
budget policy:upsert	POSTs --payload-json to the company's budgets/policies endpoint.
budget company:update	PATCHes the company's budget with an UpdateBudget JSON payload.
budget agent:update <agent-id>	PATCHes a single agent's budget with an UpdateBudget JSON payload.
budget incident:resolve	POSTs to budget-incidents/<incident-id>/resolve; payload type ResolveBudgetIncident.

G11.4 The documented budget workflow

- 1. budget policy:upsert — define the windows and limits before any agent runs.
- 2. budget company:update — set the company-wide ceiling.
- 3. budget agent:update <agent-id> — give individual agents tighter or looser room as needed.
- 4. budget overview and cost window-spend — confirm the guard rails read the way you expect.
- 5. While the company runs, monitor with cost summary, cost by-agent, and cost window-spend.
- 6. When an incident fires, triage with cost by-agent, adjust the relevant budget, then budget incident:resolve <incident-id>.

G11.5 Scope, flags and gotchas

- "Every read is non-destructive; the only commands that change state are event:create, policy:upsert, company:update, agent:update, and incident:resolve."
- Two commands are scoped by POSITIONAL argument and take no --company-id: cost issue <issue-id> and budget agent:update <agent-id>.
- cost issue accepts an issue id or human identifier — "such as PAP-39".

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- budget company:update and budget incident:resolve require a company before they will run.
- budget incident:resolve's --payload-json is optional and defaults to {}.
- cost event:create and finance event:create require --payload-json (a valid JSON object).
- "Most cost is recorded automatically by the server-side runtime as agents run."

Note: Documented warnings. On JSON: "--payload-json is parsed with JSON.parse. Malformed JSON fails the command before any request is sent. Quote the whole value in single quotes in your shell so embedded double quotes survive." On incidents: "Resolving an incident clears the gate it raised. If the underlying overspend is still in progress, resolve it only after you have also adjusted the policy with budget policy:upsert, company:update, or agent:update — otherwise the incident will simply recur on the next run."

One relationship is easy to get backwards: "A company-wide budget and per-agent budgets work together. Tightening one agent does not raise the company ceiling, and the company ceiling still bounds the sum of all agents." And the fastest triage path is stated for you: "When a budget alert fires, the fastest triage path is cost window-spend to see how far over you are, then cost by-agent to find the source."

G12. Workspace, org and environment commands

"Execution workspaces are the server-side sandboxes where heartbeat runs do their work." This page groups four related families — org chart rendering, agent configurations, execution workspaces and environments, plus project workspaces. All of them are "read-and-control wrappers over the Paperclip API. They TRIGGER and OBSERVE server-side state; the actual execution... runs on the Paperclip runtime, not in the CLI."

G12.1 Org chart

```
paperclipai org get --company-id <company-id>
paperclipai org svg --company-id <company-id> --out org.svg
paperclipai org png --company-id <company-id> --out org.png
paperclipai org png -C <company-id> > org.png
```

org get returns the org chart as structured JSON; org svg and org png return rendered images. "With --out <path>, the bytes are written to that file and the command prints a small { out, bytes } summary. Without --out, the raw image bytes stream to stdout."

G12.2 Agent configurations

```
paperclipai agent-config list --company-id <company-id>
```

agent-config list is company-scoped and read-only.

G12.3 Execution workspaces

```
paperclipai workspace list --company-id <company-id>
paperclipai workspace get <execution-workspace-id>
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai workspace close-readiness <execution-workspace-id>
paperclipai workspace operations <execution-workspace-id>
paperclipai workspace update <execution-workspace-id> --payload-json '{...}'
paperclipai workspace runtime-service <execution-workspace-id> start --payload-json
'{...}'
paperclipai workspace runtime-command <execution-workspace-id> run --payload-json
'{...}'

paperclipai workspace runtime-service <execution-workspace-id> restart --payload-
json '{"service":"dev-server"}'
paperclipai workspace runtime-command <execution-workspace-id> run --payload-json
'{"command":"npm test"}'
```

workspace subcommands

Command	Arguments	Use
list	--company-id <id>	List all execution workspaces in a company.
get	<id>	Fetch a single execution workspace.
close-readiness	<id>	Check whether the workspace is safe to close (no in-flight work that would be lost).
operations	<id>	List the workspace's operation history.
update	<id> --payload-json <json>	Patch the workspace with a JSON body.
runtime-service	<id> <action> [--payload-json <json>]	Control a runtime service in the workspace.
runtime-command	<id> <action> [--payload-json <json>]	Run a runtime command in the workspace.

The <action> for runtime-service and runtime-command is one of start, stop, restart or run — for both workspace and project-workspace. The --payload-json for those runtime commands defaults to {} when omitted.

Note: Documented warning: "runtime-service stop/restart act on live runtime processes a run may be depending on. Confirm with workspace get and workspace operations that nothing critical is mid-flight first." And the tip: "Run workspace close-readiness <id> before you tear down or close a workspace. It tells you whether closing it now would interrupt active work."

G12.4 Environments

```
paperclipai environment list --company-id <company-id>
paperclipai environment capabilities --company-id <company-id>
paperclipai environment create --company-id <company-id> --payload-json '{...}'
paperclipai environment get <environment-id>
paperclipai environment leases <environment-id>
paperclipai environment lease <lease-id>
paperclipai environment update <environment-id> --payload-json '{...}'
paperclipai environment delete <environment-id>
paperclipai environment probe <environment-id>
paperclipai environment probe-config --company-id <company-id> --payload-json
'{...}'

# Validate a config before committing it, then create the environment.
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
paperclipai environment probe-config --company-id <company-id> --payload-json
'{"kind":"ssh","host":"build-01.internal"}'
paperclipai environment create --company-id <company-id> --payload-json
'{"name":"build-01","kind":"ssh","host":"build-01.internal"}'
paperclipai environment probe <environment-id>
```

environment subcommands

Command	Arguments	Use
list	--company-id <id>	List a company's environments.
capabilities	--company-id <id>	Report the environment capabilities available to the company.
create	--company-id <id> --payload-json <json>	Create an environment from a JSON body.
get	<environment-id>	Fetch a single environment.
leases	<environment-id>	List the leases held against an environment.
lease	<lease-id>	Fetch a single environment lease by its own id.
update	<environment-id> --payload-json <json>	Patch an environment with a JSON body.
delete	<environment-id>	Delete an environment.
probe	<environment-id>	Probe an existing environment for reachability/health.
probe-config	--company-id <id> --payload-json <json>	Probe a candidate config before you create the environment.

"A lease is a claim on an environment by a workspace." environment lease <lease-id> hits /api/environment-leases/{leaseId}.

Note: Documented tip: "probe-config is the dry-run path — it checks a candidate configuration without persisting anything. probe validates an environment that already exists. Probe first, create second."

G12.5 Project workspaces

```
paperclipai project-workspace list <project-id>
paperclipai project-workspace create <project-id> --payload-json '{...}'
paperclipai project-workspace update <project-id> <workspace-id> --payload-json
'{...}'
paperclipai project-workspace delete <project-id> <workspace-id>
paperclipai project-workspace runtime-service <project-id> <workspace-id> restart --
payload-json '{...}'
paperclipai project-workspace runtime-command <project-id> <workspace-id> run --
payload-json '{...}'

paperclipai project-workspace runtime-command <project-id> <workspace-id> run --
payload-json '{"command":"pnpm build"}'
```

Note: Documented note on argument order: "update requires the workspace id positional — the command rejects the call if you omit it. create takes only the project id, since the workspace does not exist yet."

Every command in this family accepts --data-dir, --api-base, --api-key, --context, --profile and --json; company-scoped ones also take -C, --company-id <id>; "ID- and project-scoped subcommands take the id as a positional argument instead."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G13. Skills commands

"Skills are how you teach an agent a repeatable procedure (open a pull request, run a browser, follow a review checklist) without rewriting its prompt every time." Two command groups exist: a plural skills group with reference resolution and safety prompts, and a singular skill group that is lower-level and payload-driven.

G13.1 The three operations

What each operation changes

Operation	Commands	What it changes
Company install	skills install, import, create, scan-projects	Adds or updates a skill in the company's library. Available to the whole company; does NOT attach it to any agent.
Agent attach	skills agent sync, skills agent clear	Changes an agent's DESIRED company-skill set. sync takes a required --mode (add, remove, or replace); clear empties it.
Adapter runtime sync	reported by skills agent list; triggered by sync/clear	The server-side adapter reconciles the desired set with files on disk and returns an AgentSkillSnapshot.

Note: The distinction between install and attach is the one learners most often collapse. Installing a skill puts it in the company library. It does nothing for any agent until you ATTACH it. "Bundled Paperclip skills still live in the company library as read-only entries, but the server no longer force-attaches them to an agent."

G13.2 Catalog

```
paperclipai skills browse --kind bundled --category github
paperclipai skills search "pull request" --kind bundled
paperclipai skills inspect github-pr-workflow

paperclipai skills install github-pr-workflow --company-id <company-id>
paperclipai skills install github-pr-workflow --as review-prs --company-id <company-id>
```

Browse/search filters and install flags

Flag	Use
--kind <kind>	Filter by catalog kind: bundled or optional.
--category <slug>	Filter by catalog category slug.
--query <text>	Free-text search (browse only; search uses its positional argument).
--as <slug>	(install) Override the company skill slug.
--force	(install) Replace a same-key catalog-managed skill when the server allows it. Never bypasses hard validation or hard-stop audit findings.

Note: Documented note: "Catalog commands are for the app-shipped catalog only. GitHub, skills.sh, local-path, and plain-URL sources go through skills import instead."

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G13.3 The company library — import and create

```
paperclipai skills list --company-id <company-id>
paperclipai skills show <skill-ref> --company-id <company-id>
paperclipai skills file <skill-ref> --path SKILL.md --company-id <company-id>
paperclipai skills import <source> --company-id <company-id>
paperclipai skills create --name "Review PRs" --slug review-prs --body-file SKILL.md
--company-id <company-id>
paperclipai skills scan-projects --project-id <project-id> --company-id <company-id>
```

skills create options

Flag	Use
--name <name>	Required. Skill name.
--slug <slug>	Optional explicit slug.
--description <text>	Optional description.
--body-file <path>	Markdown body file. Use - to read the body from stdin.

skills scan-projects takes --project-id <id> and --workspace-id <id>, each of which may be repeated. With neither supplied, the scan "covers the company's projects and workspaces". The import source formats accepted by the underlying endpoint include a GitHub repository URL, a tree or blob URL, an owner/repo shorthand, an owner/repo/skill shorthand, a skills.sh URL, an npx skills add command line, a plain URL to a SKILL.md and a local filesystem path.

```
{ "source": "https://github.com/paperclipai/paperclip" }
{ "source": "https://github.com/paperclipai/paperclip/tree/main/skills/paperclip" }
{ "source": "https://github.com/paperclipai/paperclip/blob/main/skills/paperclip/
SKILL.md" }
{ "source": "paperclipai/paperclip" }
{ "source": "paperclipai/paperclip/paperclip-create-agent" }
{ "source": "https://skills.sh/paperclipai/paperclip/paperclip-create-agent" }
{ "source": "npx skills add paperclipai/paperclip --skill paperclip-create-agent" }
{ "source": "https://example.com/raw/SKILL.md" }
{ "source": "/Users/me/code/my-project/skills/code-review" }
```

G13.4 The maintenance loop

```
paperclipai skills check --company-id <company-id>
paperclipai skills update <skill-ref> --company-id <company-id>
paperclipai skills update --all --company-id <company-id>
paperclipai skills audit <skill-ref> --company-id <company-id>
paperclipai skills reset <skill-ref> --yes --company-id <company-id>
paperclipai skills remove <skill-ref> --yes --company-id <company-id>
```

Maintenance commands

Command	Use
check [skillRef]	Report update status: supported, hasUpdate, currentRef/latestRef, installedHash/originHash, any updateHoldReason, and the auditVerdict.
update [skillRef]	Install the pinned update for one skill.
update --all	Check every skill and update only those with hasUpdate=true; supported-but-current and unsupported skills are skipped with a reason.
audit [skillRef]	Re-scan installed bytes and report findings (severity, code, path, message) without executing anything.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

reset <skillRef>	Reinstall a catalog-managed skill from its pinned origin, discarding local edits.

The skill reference is OPTIONAL on check, update and audit — omit it to operate over the whole library. --force discards local-modification and soft-audit holds, but hard-stop audit findings still block the operation; a fail verdict "is what hard-blocks install-update". --yes on reset confirms without the interactive prompt and is required when not in a TTY.

Note: Documented warning: "Pass either a skill reference or --all to update, never both — the command rejects that combination."

G13.5 Attaching skills to an agent

```
paperclipai skills agent list <agent-ref> --company-id <company-id>
paperclipai skills agent sync <agent-ref> --skill review-prs --skill agent-browser
--mode add --company-id <company-id>
paperclipai skills agent clear <agent-ref> --yes --company-id <company-id>
```

Agent-attach flags

Flag	Use
--skill <skillRef>	(sync) A desired company skill id, key, or slug. Repeat for several. At least one is required.
--mode <mode>	(sync) Required. Merge mode: add keeps other skills; remove deletes only named skills; replace destructively overwrites the complete set.
--yes	(clear) Confirm without the interactive prompt.

The merge-mode semantics are unforgiving and worth reading twice: "With --mode replace you must list the complete desired set every time — anything you omit is detached — while --mode add and --mode remove only touch the skills you name. clear detaches everything." skills agent clear "empties the set outright (it uses replace internally with an empty list)".

Adapter sync modes reported by skills agent list

Mode	Behaviour
persistent	Adapter writes files to working directory; leaves them between runs
ephemeral	Files materialised per run, cleaned up after
unsupported	Assignment recorded but cannot push files into runtime

G13.6 The singular skill group

```
paperclipai skill list -C <company-id>
paperclipai skill get <skill-id> -C <company-id>
paperclipai skill file <skill-id> --path SKILL.md -C <company-id>
paperclipai skill file:update <skill-id> --payload-json
'{"path":"SKILL.md","content":"..."}' -C <company-id>
paperclipai skill update-status <skill-id> -C <company-id>
paperclipai skill install-update <skill-id> -C <company-id>
paperclipai skill delete <skill-id> -C <company-id>
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The singular group "does not resolve key/slug references — every subcommand takes a literal <skillId>." Its create, import and scan-projects subcommands all require --payload-json.

G13.7 Reference resolution and its failure modes

- <catalogRef> = catalog skill id, key, or a UNIQUE slug.
- <skillRef> resolves against the company library by id, canonical key, or UNIQUE slug. "An ambiguous slug is rejected — use an id or key instead."
- <agentRef> = agent id or shortname/url-key.
- Server-side resolution order: exact UUID match on id, then normalised key match, then normalised slug match "but only when exactly one skill in the company has that slug".
- Ambiguous references fail with "Invalid company skill selection (ambiguous references: <slug>; ...)"; unknown ones with "Invalid company skill selection (unknown references: <reference>; ...)".
- skills remove and skills agent clear "prompt for confirmation in an interactive terminal and require --yes when run non-interactively."

Skills are company-scoped: "A skill installed in Company A is invisible to Company B." Trust level derives from the highest classification in the skill's inventory, ordered markdown_only < assets < scripts_executables (references/ = reference, scripts/ = script, assets/ = asset).

G14. Secrets commands

The secrets family keeps credentials out of configuration files, shell history and exports. Read its central guarantee as a design constraint on your own scripts, not merely as a feature.

Note: Documented warning, verbatim: "Paperclip never prints secret values. secrets list, declarations, usage, and access-events return metadata only — names, keys, providers, status, and references — never the plaintext. The value is shown only at the moment you set it (and even then, prefer --value-env so it never lands in your shell history). Treat that as a hard guarantee and design your scripts around it."

G14.1 The command index

Secrets commands

Command	Use
secrets list	See the secret metadata that exists for a company.
secrets declarations	See the portable env declarations a company export emits (what a target instance must supply).
secrets create	Store a new Paperclip-managed secret value.
secrets link	Register an external provider-owned secret without copying its value into Paperclip.
secrets update	Patch a secret's metadata.
secrets rotate	Replace a managed secret's value.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

secrets usage	Find where a secret is referenced (which agents/projects).
secrets access-events	Audit when and by whom a secret was read.
secrets delete	Remove a secret (guarded by --yes + --confirm).
secrets doctor	Run provider health checks through the API.
secrets providers	List the secret provider descriptors available to a company.
secrets provider-configs (+ subcommands)	Manage per-company provider vault configs.
secrets remote-import / remote-import:preview	Pull secrets in from a configured remote source.
secrets migrate-inline-env	Convert inline sensitive agent env values into secret references.

G14.2 Reading, creating, linking

```
paperclipai secrets list --company-id <company-id>
paperclipai secrets list --company-id <company-id> --json

paperclipai secrets declarations --company-id <company-id>
paperclipai secrets declarations --company-id <company-id> --include agents,projects
--kind secret

# Preferred: the value is read from the env var, not typed on the command line
export ANTHROPIC_API_KEY=sk-ant-...
paperclipai secrets create --company-id <company-id> --name anthropic-api-key --
value-env ANTHROPIC_API_KEY

# Discouraged: --value puts the secret into shell history
paperclipai secrets create --company-id <company-id> --name anthropic-api-key --
value 'sk-ant-...'

paperclipai secrets link \
  --company-id <company-id> \
  --name prod-stripe-key \
  --provider aws_secrets_manager \
  --external-ref arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/stripe-
AbCdEf
```

secrets create flags

Flag	Use
--name <name>	Required. Secret display name.
--key <key>	Portable secret key (the stable key other resources reference).
--provider <provider>	Secret provider id. Omit to use the company default (managed local encryption).
--value <value>	The secret value. Mutually exclusive with --value-env.
--value-env <name>	Read the value from this environment variable. Mutually exclusive with --value. Errors if the variable is empty or unset.
--description <text>	Free-text description.

secrets link flags

Flag	Use
--name <name>	Required. Secret display name.
--provider <provider>	Required. Secret provider id (e.g. aws_secrets_manager).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

--external-ref <ref>	Required. Provider secret ARN, name, path, or reference.
--key <key>	Portable secret key.
--provider-version-ref <ref>	Provider version id or label to pin to.
--description <text>	Free-text description.

secrets declarations flags

Flag	Use
--include <values>	Comma-separated include set for the export preview. One or more of company,agents,projects,issues,tasks,skills. Default: company,agents,projects.
--kind <kind>	Filter declarations to all, secret, or plain. Default: all.

Note: "Always prefer --value-env: it reads the value from a named environment variable so the plaintext never appears in your shell history, your scrollback, or the process argument list." Passing both --value and --value-env, or neither, is an error.

G14.3 Updating, rotating, auditing, deleting

```
paperclipai secrets update <secret-id> --payload-json '{"description":"Rotated
quarterly by ops"}'

export ANTHROPIC_API_KEY=sk-ant-new...
paperclipai secrets rotate <secret-id> --value-env ANTHROPIC_API_KEY

paperclipai secrets usage <secret-id>
paperclipai secrets access-events <secret-id> --json

paperclipai secrets delete <secret-id> --yes --confirm <secret-id>

paperclipai secrets doctor --company-id <company-id>
paperclipai secrets providers --company-id <company-id>
```

secrets update "changes metadata, not the value. To change the value, use rotate." The API states the same rule from its own side: PATCH /api/secrets/{secretId} "does not change the secret value." Rotation creates an immutable new version and advances latestVersion.

secrets delete flags

Flag	Use
--yes	Required safety flag confirming the destructive action.
--confirm <secretId>	Required. Must repeat the same secret ID passed as the argument.

Note: "secrets delete is intentionally hard to run by accident." If --yes is missing, or --confirm does not match the argument, the command refuses to proceed. And before you delete: "Run secrets usage first whenever you plan to delete a secret. Deleting a secret that an agent's env still references will break that agent's runs." At the API level DELETE /api/secrets/{secretId} is a HARD delete — it "Deletes the secret and its version history".

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G14.4 Provider vaults and remote import

```
# List existing configs
paperclipai secrets provider-configs --company-id <company-id>

# Create a config
paperclipai secrets provider-config:create --company-id <company-id> --payload-json
'{ ... }'

# Preview which secrets a vault would discover, without importing them
paperclipai secrets provider-config:discovery-preview --company-id <company-id> --
payload-json '{ ... }'

# Inspect, update, default, health-check, or delete a single config
paperclipai secrets provider-config:get <config-id>
paperclipai secrets provider-config:update <config-id> --payload-json '{ ... }'
paperclipai secrets provider-config:default <config-id>
paperclipai secrets provider-config:health <config-id>
paperclipai secrets provider-config:delete <config-id>

paperclipai secrets remote-import:preview --company-id <company-id> --payload-json
'{ ... }'
paperclipai secrets remote-import --company-id <company-id> --payload-json '{ ... }'
```

Provider-config subcommand scoping

Subcommand	Scope	Required flags
provider-configs	company	-C
provider-config:create	company	-C, --payload-json
provider-config:discovery-preview	company	-C, --payload-json
provider-config:get <configId>	config	-
provider-config:update <configId>	config	--payload-json
provider-config:default <configId>	config	-
provider-config:health <configId>	config	-
provider-config:delete <configId>	config	-

Note: "Always preview first: remote-import:preview shows what _would_ be imported, and remote-import performs the selected import." On AWS specifically: "Do not store AWS bootstrap credentials _inside_ Paperclip secrets — let the runtime resolve them from the standard AWS credential chain, and use doctor to confirm that chain is wired up."

G14.5 Migrating inline environment values

```
# Dry run: report agents to update, secrets to create, secrets to rotate
paperclipai secrets migrate-inline-env --company-id <company-id>

# Commit the migration
paperclipai secrets migrate-inline-env --company-id <company-id> --apply
```

migrate-inline-env is a DRY RUN by default; --apply persists the changes. The dry-run output reports agentsToUpdate, secretsToCreate, secretsToRotate and the full candidate list. With --apply, "newly created secrets use the managed local-encrypted

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

provider, existing same-named secrets are rotated to the current inline value, and each affected agent's env binding is replaced with a secret_ref pointing at version: latest."

Note: Documented note on what counts as sensitive: "'Sensitive' is detected by env key name — keys matching patterns like *_token, api_key, password, secret, private_key, jwt, and similar. Inline values that are not flagged as sensitive are left untouched, so review the dry run before applying."

G14.6 Scopes, references and proposal limits

A secret reference is how a secret reaches a run without the value ever appearing in configuration. Its shape is fixed.

```
{
  "env": {
    "ANTHROPIC_API_KEY": {
      "type": "secret_ref",
      "secretId": "secret-uuid",
      "version": "latest"
    }
  }
}

// Pin a numeric version instead
{ "type": "secret_ref", "secretId": "secret-uuid", "version": 2 }
```

Omitting version is treated as latest. Use latest for rotatable values and pin numeric versions for fixed historical values. The default provider is local_encrypted, storing material "AES-GCM encrypted using the local master key" with a SHA-256 hash. managedMode is either paperclip_managed (Paperclip owns, creates, stores and rotates the value) or external_reference (the value lives in an external vault and externalRef is the pointer). Delivery modes are env (plaintext injected into the agent environment at launch), api (the agent fetches on demand) and both.

secrets list output fields, and what they are not

Field	Shows
id, name, key	Identity of the secret record
provider	Which provider holds the material
status	Record status
managedMode	paperclip_managed or external_reference
latestVersion	The current version pointer
externalRef	Whether an external reference is set (yes/no) — never the value

G14.7 Agent secret proposals — the limits

"Agents propose, people decide. An agent can raise a proposal and watch its status, but only a board member can approve it." The limits on that mechanism are hard numbers, and they are what stops a looping agent from flooding the queue.

- MAX 20 PENDING PROPOSALS PER AGENT, and MAX 20 PER MINUTE.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Proposals EXPIRE AFTER 14 DAYS, transitioned by a background sweep. On resolution ciphertextScrubbedAt is stamped and the value is wiped.
- A secret proposal's value field is plaintext, MAX 65536 BYTES, encrypted immediately.
- Proposal statuses: pending, approved, rejected, withdrawn, expired.
- Proposal list pagination: limit default 100, max 200, plus offset; response headers X-Page-Limit, X-Page-Offset, X-Next-Offset.
- Binding proposals use a configPath of env.<KEY> or access.<ALIAS>, and the target policy is self_and_reports.
- Reject requires a non-empty reason. Approve accepts cascade plus overrides.name, overrides.description and overrides.providerConfigId.
- Documented approval failures: "Expired proposals cannot be approved"; "Binding proposal requires pending secret proposal <id>; retry with cascade=true".

Run-bound value reads are deliberately narrow. POST /api/agents/me/secrets/{key}/value returns { "key", "value", "version" } with Cache-Control: no-store; every read is logged to the security audit trail and the activity log; it requires a live heartbeat token and secrets:read. "Low-trust, task-bridge and skill-test tokens are denied run-bound secret access on purpose."

Strict mode (PAPERCLIP_SECRETS_STRICT_MODE) requires secret references for sensitive env keys — *_API_KEY, *_TOKEN, *_SECRET — and rejects inline plaintext. It applies to agent adapterConfig.env, project env, hire-agent approvals and company imports. Two caveats: "Strict mode blocks new inline sensitive values. It does not automatically migrate what is already stored," and it does NOT apply to paperclipai configure --section llm.

G15. REST API essentials

Everything the CLI does, it does over this API, and there are situations — a plugin, a webhook receiver, an agent runtime you wrote yourself — where you call it directly. This section gives the base URL, the authentication header, the key endpoint tables, and the behaviours that most often catch people out.

G15.1 Base URL, content type and errors

```
http://localhost:3100/api

# Company-scoped path convention
/api/companies/{companyId}/...
```

Instance-level endpoints

Method	Path	Purpose
GET	/api/health	"The fastest way to check whether the server is up and whether the database is reachable." Returns runtime metadata including deployment

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		mode, auth readiness and feature flags.
GET	/api/companies/{companyId}	Verify access to a specific company resource.
GET	/api/openapi.json	"Machine-readable description of its REST surface"

Content-Type is application/json. The error response shape is {"error": "message"}, and some errors add a details field.

HTTP status codes

Code	Meaning	Cause
400	Bad request	Validation failed or payload malformed
401	Unauthorized	No valid caller identity provided
403	Forbidden	Authenticated but not permitted
404	Not found	Resource doesn't exist or outside scope
409	Conflict	Resource locked or in conflicting state
422	Unprocessable entity	Business rules reject valid structure
503	Service unavailable	Database unreachable
500	Internal server error	Unexpected failure

G15.2 Authentication

```
Authorization: Bearer <token>
X-Paperclip-Run-Id: <run_id>
```

Authentication by caller type

Caller type	Authentication	Notes
Board user (local_trusted)	Implicit board access	No login required
Board user (authenticated mode)	Session cookie from the web app	Browser sessions work for API clients
Board user with API token	Authorization: Bearer <board-token>	Long-lived board API keys
Agent	Authorization: Bearer <agent-key-or-jwt>	Long-lived keys or short-lived run JWTs

The resolution order is fixed: no bearer token in a local_trusted deployment gives a board actor with full local trust; no bearer token in an authenticated deployment resolves a web session; a bearer token is checked as a board API key first, then against agent API keys, then as a local agent JWT; no match leaves the request unauthenticated. "Bearer tokens always win over cookies."

```
curl -s http://localhost:3100/api/cli-auth/me \
  -H "Authorization: Bearer pcp_board_your_token_here"

curl -s http://localhost:3100/api/agents/me \
  -H "Authorization: Bearer $PAPERCLIP_API_KEY" \
  -H "X-Paperclip-Run-Id: run_123"
POST /api/cli-auth/challenges
GET  /api/cli-auth/challenges/:id
POST /api/cli-auth/challenges/:id/approve
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
POST /api/cli-auth/challenges/:id/cancel
GET  /api/cli-auth/me
POST /api/cli-auth/revoke-current

POST /api/agents/:id/keys      # create agent API key
GET  /api/agents/me            # agent-only endpoint
```

G15.3 The X-Paperclip-Run-Id header

Note: X-Paperclip-Run-Id must be sent on issue checkout and on issue state changes (PATCH /api/issues/{issueId}). "Always include the X-Paperclip-Run-Id header on state changes." The deploy reference states the general rule: "Mutating API requests should include X-Paperclip-Run-Id: $PAPERCLIP_RUN_ID for attribution." Forgetting it on an agent update is rejected as a checkout-ownership violation.

The value is the current heartbeat run id, which the runtime receives as PAPERCLIP_RUN_ID. Its purpose is attribution: it is what makes the activity trail answer "which run did this" rather than merely "which agent".

G15.4 Agents

Agent endpoints

Method	Path	Purpose
GET	/api/companies/{companyId}/agents	"Lists agents for a company"
POST	/api/companies/{companyId}/agents	Create new agent in company
GET	/api/agents/{agentId}	Retrieve single agent by UUID or shortname
GET	/api/agents/me	"Returns the authenticated agent itself"
PATCH	/api/agents/{agentId}	Update existing agent configuration
DELETE	/api/agents/{agentId}	Hard delete agent and cleanup runtime state
POST	/api/agents/{agentId}/pause	Temporarily suspend agent work intake
POST	/api/agents/{agentId}/resume	Reactivate paused agent
POST	/api/agents/{agentId}/terminate	Permanently disable agent; revokes all API keys
PATCH	/api/agents/{agentId}/permissions	"Updates the agent-level permissions block"
POST	/api/companies/{companyId}/agent-hires	Create agent through approval-aware flow with optional source issues
POST	/api/agents/{agentId}/wakeup	Queue work with context (issue ID, reason, payload)
POST	/api/agents/{agentId}/heartbeat/invoke	Trigger straightforward on-demand heartbeat
GET	/api/agents/me/inbox-lite	"Compact assignment list an agent needs at the start of a heartbeat"

```
GET    /api/agents/{agentId}/keys
POST   /api/agents/{agentId}/keys          # token shown once only
DELETE /api/agents/{agentId}/keys/{keyId}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
GET  /api/agents/{agentId}/configuration
GET  /api/agents/{agentId}/config-revisions
GET  /api/agents/{agentId}/config-revisions/{revisionId}
POST /api/agents/{agentId}/config-revisions/{revisionId}/rollback
GET  /api/companies/{companyId}/agent-configurations

GET /api/companies/{companyId}/org
GET /api/companies/{companyId}/org.svg
GET /api/companies/{companyId}/org.png
```

G15.5 Issues

Core issue endpoints

Method	Path	Purpose
GET	/api/companies/{companyId}/issues	List company issues (filterable)
GET	/api/issues/{issueId}	Fetch one issue
GET	/api/issues/{issueId}/heartbeat-context	Fetch the heartbeat context
POST	/api/companies/{companyId}/issues	Create an issue
PATCH	/api/issues/{issueId}	Update an issue
POST	/api/issues/{issueId}/checkout	Atomically claim an issue
POST	/api/issues/{issueId}/release	Release a claimed issue

Listing accepts status (or a comma-separated list such as todo,in_progress), assigneeAgentId, participantAgentId, assigneeUserId, touchedByUserId, inboxArchivedByUserId, unreadForUserId, projectId, executionWorkspaceId, parentId/parentIssueId, labelId, originKind (for example manual or routine_execution), originId, includeRoutineExecutions (default FALSE), q (full-text over title, identifier, description and comments) and limit. On create, only title is required; status defaults to BACKLOG and priority defaults to MEDIUM.

Note: "Routine execution issues are excluded by default unless you opt in with includeRoutineExecutions=true." This is the reason a routine can appear to be doing nothing when it is in fact working.

```
GET    /api/issues/{issueId}/comments
GET    /api/issues/{issueId}/comments/{commentId}
POST   /api/issues/{issueId}/comments

GET    /api/issues/{issueId}/documents
GET    /api/issues/{issueId}/documents/{key}
PUT    /api/issues/{issueId}/documents/{key}
DELETE /api/issues/{issueId}/documents/{key}          # board-only
POST   /api/issues/{issueId}/documents/{key}/lock
POST   /api/issues/{issueId}/documents/{key}/unlock
GET    /api/issues/{issueId}/documents/{key}/revisions
POST   /api/issues/{issueId}/documents/{key}/revisions/{revisionId}/restore

GET    /api/issues/{issueId}/attachments
POST   /api/companies/{companyId}/issues/{issueId}/attachments   # multipart/form-
data
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
GET    /api/attachments/{attachmentId}/content
GET    /api/attachments/{attachmentId}/content?download=1
DELETE /api/attachments/{attachmentId}

GET    /api/issues/{issueId}/approvals
POST   /api/issues/{issueId}/approvals            # body: approvalId
DELETE /api/issues/{issueId}/approvals/{approvalId}

GET  /api/issues/{issueId}/interactions
POST /api/issues/{issueId}/interactions
POST /api/issues/{issueId}/interactions/{interactionId}/accept
POST /api/issues/{issueId}/interactions/{interactionId}/reject
POST /api/issues/{issueId}/interactions/{interactionId}/respond
POST /api/issues/{issueId}/interactions/{interactionId}/withdraw
POST /api/issues/{issueId}/interactions/{interactionId}/cancel     # board-only

POST /api/issues/{issueId}/scheduled-retry/retry-now      # board-only
GET  /api/issues/{issueId}/recovery-actions
POST /api/issues/{issueId}/recovery-actions/resolve
POST /api/issues/{issueId}/file-resources/availability
POST /api/issues/{id}/monitor/check-now
```

G15.6 409 is terminal, and reopen:true

Note: Two rules that between them prevent most multi-agent data corruption. FIRST: treat 409 from checkout as TERMINAL. "Retrying 409 Conflict from checkout... Thrashes lock; steals ownership. Treat 409 as terminal; pick different issue." SECOND: a terminal issue cannot be revived by setting its status. "Reopening with PATCH status: 'todo' on done issue... Rejected — terminal transitions require reopen. Send PATCH { reopen: true, comment: "..." }."

The documented common mistakes, in full

Mistake	What goes wrong	Do this instead
PATCH status: "in_progress" to claim a task	Skips checkout; leaves checkoutRunId empty; race-prone	Always claim via POST /api/issues/{id}/checkout with expectedStatuses and X-Paperclip-Run-Id
Retrying 409 Conflict from checkout	Thrashes lock; steals ownership	Treat 409 as terminal; pick a different issue
Free-text "blocked by PAP-XYZ" comment	Dependent never auto-wakes	Set blockedByIssueIds on create/PATCH
Cancelling a blocker and expecting auto-unblock	cancelled blockers don't count as resolved	Replace/remove the cancelled id from blockedByIssueIds explicitly
Approving an in_review issue without being current participant	Server returns 422	Inspect executionState.currentParticipant first
Reopening with PATCH status: "todo" on a done issue	Rejected — terminal transitions require reopen	Send PATCH { reopen: true, comment: "..." }
Forgetting X-Paperclip-Run-Id on agent updates	Server rejects as checkout-ownership violation	Always pass the current heartbeat run id

G15.7 Approvals

Approval endpoints

Method	Path	Purpose
GET	/api/companies/{companyId}/approvals	List company approvals with optional status filter

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

GET	/api/approvals/{approvalId}	Retrieve single approval record
POST	/api/companies/{companyId}/ approvals	Create new approval request
GET	/api/approvals/{approvalId}/ issues	Fetch issues linked to approval
POST	/api/approvals/{approvalId}/ approve	Board-only: approve pending/revision request
POST	/api/approvals/{approvalId}/reject	Board-only: reject pending/revision request
POST	/api/approvals/{approvalId}/ request-revision	Board-only: send pending to revision state
GET	/api/approvals/{approvalId}/ comments	List approval comments
POST	/api/approvals/{approvalId}/ comments	Add comment to approval thread
POST	/api/approvals/{approvalId}/ resubmit	Requester: resubmit revision_requested approval

The create body requires type and payload (free-form JSON whose shape varies by type); requestedByAgentId is optional and is set automatically when the caller is an agent; issueIds links issues within the same company only. Failure codes are 403 for missing board access or an unauthorised agent action, 404 for an approval or issue outside company scope, and 422 for a wrong approval state or a cross-company constraint violation.

G15.8 Secrets

```
GET    /api/companies/{companyId}/secret-providers
GET    /api/companies/{companyId}/secrets
GET    /api/companies/{companyId}/secrets/catalog
POST   /api/companies/{companyId}/secrets
PATCH  /api/secrets/{secretId}
POST   /api/secrets/{secretId}/rotate
DELETE /api/secrets/{secretId}

# Run-bound agent access
GET  /api/agents/me/secrets
POST /api/agents/me/secrets/{key}/value

# Secret proposals
POST   /api/agents/me/secret-proposals
GET    /api/agents/me/secret-proposals
DELETE /api/agents/me/secret-proposals/{id}
GET    /api/companies/{companyId}/secret-proposals
POST   /api/companies/{companyId}/secret-proposals/{id}/approve
POST   /api/companies/{companyId}/secret-proposals/{id}/reject
```

POST /api/companies/{companyId}/secrets body

Field	Required	Notes
name	Yes	Unique within company
value	Yes	Plaintext secret value
provider	No	Defaults to local_encrypted
description	No	Human-readable note
externalRef	No	Required by some external providers

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

GET /api/agents/me/secrets "Returns just the aliases this agent is allowed to reach". POST /api/agents/me/secrets/{key}/value retrieves the plaintext and sends Cache-Control: no-store. The company secret-proposals list accepts ?status=pending|approved|rejected|withdrawn|expired.

G15.9 Runs and heartbeat runs

Note: There is no /reference/api/runs page — it returns 404. The documented run endpoints appear on the costs page, and the CLI run family is the primary documented interface for inspecting runs.

```
GET   /api/companies/{companyId}/heartbeat-runs
GET   /api/heartbeat-runs/{runId}
```

A heartbeat run's usageJson carries costUsd, cacheAdjustedCostUsd, token counts, and provider, biller, model, costStatus and billingType. The nine-step heartbeat protocol the documentation gives to agent developers is worth reproducing because it is the canonical sequence a runtime should follow: 1. identity via GET /api/agents/me; 2. approval follow-up if PAPERCLIP_APPROVAL_ID is set; 3. get assignments via GET /api/companies/{companyId}/issues?assigneeAgentId={yourId} &status=todo,in_progress,in_review,blocked; 4. pick work (prioritise in_progress, then in_review, then todo); 5. checkout via POST /api/issues/{issueId}/checkout with the X-Paperclip-Run-Id header; 6. understand context; 7. do the work; 8. update status via PATCH /api/issues/{issueId} with the run id header; 9. delegate subtasks if needed.

G15.10 Routines

```
GET    /api/companies/{companyId}/routines
GET    /api/routines/{routineId}
POST   /api/companies/{companyId}/routines         Content-Type: application/json
PATCH  /api/routines/{routineId}                   Content-Type: application/json
POST   /api/routines/{routineId}/triggers          Content-Type: application/json
PATCH  /api/routine-triggers/{triggerId}           Content-Type: application/json
DELETE /api/routine-triggers/{triggerId}
POST   /api/routine-triggers/{triggerId}/rotate-secret
POST   /api/routines/{routineId}/run               Content-Type: application/json
POST   /api/routine-triggers/public/{publicId}/fire
GET    /api/routines/{routineId}/runs?limit=50
curl -X POST "http://localhost:3100/api/companies/company-1/routines" \
  -H "Authorization: Bearer <board-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Weekly CEO briefing for {{company_name}}",
    "description": "Compile a weekly summary and prepare the next priorities.",
    "assigneeAgentId": "agent-1",
    "projectId": "project-1",
    "goalId": "goal-1",
    "priority": "medium",
    "status": "active",
    "concurrencyPolicy": "coalesce_if_active",
    "catchUpPolicy": "skip_missed"
  }'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G15.11 Costs and budgets

```
POST  /api/companies/{companyId}/cost-events
GET   /api/companies/{companyId}/costs/summary
GET   /api/companies/{companyId}/costs/by-agent
GET   /api/companies/{companyId}/costs/by-agent-model
GET   /api/companies/{companyId}/costs/by-provider
GET   /api/companies/{companyId}/costs/by-biller
GET   /api/companies/{companyId}/costs/by-project
GET   /api/companies/{companyId}/costs/window-spend
GET   /api/companies/{companyId}/costs/quota-windows

PATCH /api/companies/{companyId}/budgets
PATCH /api/agents/{agentId}/budgets
GET   /api/companies/{companyId}/budgets/overview
POST  /api/companies/{companyId}/budgets/policies
POST  /api/companies/{companyId}/budget-incidents/{incidentId}/resolve

POST  /api/companies/{companyId}/finance-events
GET   /api/companies/{companyId}/costs/finance-summary
GET   /api/companies/{companyId}/costs/finance-by-biller
GET   /api/companies/{companyId}/costs/finance-by-kind
GET   /api/companies/{companyId}/costs/finance-events
```

A cost event requires agentId, provider, model, costCents and occurredAt. biller defaults to provider, billingType defaults to unknown and costStatus defaults to reported, each when omitted. Both budget PATCH endpoints take the same body: { "budgetMonthlyCents": 5000 }. Budget policy defaults are metric = billed_cents, windowKind = calendar_month_utc for company and agent scopes but lifetime for project scopes, warnPercent = 80, hardStopEnabled = true, notifyEnabled = true and isActive = true. Incident resolution allows keep_paused or raise_budget_and_resume, and "If you raise the budget, you must supply a new amount that exceeds the current observed spend."

Note: Documented warning: "Hard-stop budget enforcement pauses the affected scope and cancels work for that scope. A budget increase only resumes the scope if the new budget is above current observed spend." And on units: "cacheAdjustedCostUsd and costUsd are in dollars, while costCents on cost events is in cents. Don't mix the two units when you reconcile a run against its ledger entry."

The documented reading order for a cost investigation is: GET /costs/summary, then /costs/by-agent, then /costs/by-project, then /budgets/overview, then /costs/window-spend.

G15.12 Projects

Note: /reference/api/projects/ returns 404; the navigation lists /reference/api/goals-and-projects/ instead, and that page was not harvested. The project surface is therefore documented here only through the CLI project family in section G5 and through the project-scoped fields on other endpoints (projectId on issues, routines and cost events; GET /api/companies/{companyId}/costs/by-project). No project REST paths are invented.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G15.13 Companies

```
GET    /api/companies
GET    /api/companies/{companyId}
POST   /api/companies
PATCH  /api/companies/{companyId}
PATCH  /api/companies/{companyId}/branding
POST   /api/companies/{companyId}/logo           Content-Type: multipart/form-data
GET    /api/companies/stats
GET    /api/companies/{companyId}/artifacts
GET    /api/companies/{companyId}/feedback-traces
POST   /api/companies/{companyId}/archive
DELETE /api/companies/{companyId}
```

Export and import routes

Route	Purpose	Who can call it
POST /api/companies/{companyId}/export	Export a company bundle	Any caller with access to the company
POST /api/companies/{companyId}/exports/preview	Preview a company export	Board or CEO of that company
POST /api/companies/{companyId}/exports	Export a company bundle with the stricter portability flow	Board or CEO of that company
POST /api/companies/import/preview	Preview an import into a new or existing company	Board only
POST /api/companies/{companyId}/imports/preview	Preview an import into the same company	Board or CEO of that company
POST /api/companies/{companyId}/imports/apply	Apply an import into the same company	Board or CEO of that company

GET /api/companies and GET /api/companies/stats are board-only, as is GET /feedback-traces. CEO agents may call PATCH /api/companies/{companyId} "but only with branding fields", and PATCH /branding "only for their own company." A logo upload does not attach the logo: after the upload succeeds, send the returned assetId to PATCH /api/companies/{companyId} or PATCH /api/companies/{companyId}/branding as logoAssetId.

G15.14 Skills endpoints and the sync modes

Company skill management endpoints

Action	Endpoint
List skills	GET /api/companies/{companyId}/skills
Skill detail	GET /api/companies/{companyId}/skills/{skillId}
Read file	GET /api/companies/{companyId}/skills/{skillId}/files?path=SKILL.md
Update file	PATCH /api/companies/{companyId}/skills/{skillId}/files
Update status	GET /api/companies/{companyId}/skills/{skillId}/update-status
Install update	POST

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	/api/companies/{companyId}/skills/{skillId}/install-update
Delete	DELETE /api/companies/{companyId}/skills/{skillId}
Create empty	POST /api/companies/{companyId}/skills
Import from source	POST /api/companies/{companyId}/skills/import
Scan projects	POST /api/companies/{companyId}/skills/scan-projects

```
GET /api/skills/catalog                        # filters: kind (bundled/optional),
category, q
GET /api/skills/catalog/{catalogId}
GET /api/skills/catalog/{catalogId}/files

POST /api/companies/{companyId}/skills/install-catalog
POST /api/companies/{companyId}/skills/{skillId}/audit
POST /api/companies/{companyId}/skills/{skillId}/reset
GET  /api/companies/{companyId}/skills/{skillId}/versions
```

"Mutating routes require either agents:create permission or permissions.canCreateAgents=true."

G15.15 The skills sync endpoint and its modes

```
GET  /api/agents/{agentId}/skills
POST /api/agents/{agentId}/skills/sync     # mode: "add" | "remove" | "replace"
```

The sync request body carries desiredSkills — an array of UUIDs, keys or slugs — and mode, which is REQUIRED and must be one of add, remove or replace. Version pinning is expressed by replacing a bare reference with an object.

```
{ "desiredSkills": ["paperclip", "code-review"] }

{
  "desiredSkills": [
    "code-review",
    { "key": "paperclipai/paperclip/paperclip", "versionId": "<uuid>" }
  ]
}
```

GET /api/agents/{agentId}/skills returns an AgentSkillSnapshot with the fields adapterType, supported, mode, desiredSkills, entries[] and warnings. Version pinning is gated: "If any entry carries a non-null versionId while enableBetaSkills is off, the request fails with 400 and the message Beta skill version pins require the Beta skills experimental setting to be enabled." The enableBetaSkills instance setting defaults to OFF, and with it off "saved pins are ignored rather than deleted, and every agent resolves to the live skill."

Note: The replace mode is the one that bites. It is destructive: whatever you omit from desiredSkills is detached. Read the current set with GET /api/agents/{agentId}/skills before you send a replace, or use add and remove, which touch only what you name.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

G16. Coverage note — what this reference does not contain

This appendix is bounded by the documentation that was harvested. Several CLI groups appear in the official navigation but their reference pages were not captured, so no syntax is given for them here and none should be assumed: output-and-scripting, service, dev-environments, connect (beyond G3.1), token (beyond G3.4 and G3.5), goal, prompt, adapter, activity, dashboard, access, teams, cloud, plugin, asset and feedback.

On the API side, the following pages exist in the navigation and were not harvested: built-in-agents, cases, attention, decisions, checkbox-confirmations, goals-and-projects, resource-memberships, folders, company-skill-policy, inbox-agent-policy, activity, decision-training, summary-slots, status-cards, dashboard, adapters, teams-catalog, plugins, tool-gateway and instance-admin. Three URLs were confirmed to return HTTP 404 and have no content at all: /reference/cli/budget/, /reference/api/runs/ and /reference/api/projects/.

Note: When you need one of those surfaces, go to the live documentation or run paperclipai <group> --help against your own install. Do not extrapolate a flag from a neighbouring family — Paperclip's command groups are deliberately inconsistent in places (routine versus routines, skills versus skill, --payload versus --payload-json) and a plausible guess is usually wrong.

G17. Self-check

Check your understanding

1. Which flag do you add to any command to get machine-readable output, and why does the documentation recommend it for every scripted call?
2. An agent's checkout returns 409 Conflict. What is the correct next action, and what must you never do?
3. You want to move a done issue back into the queue. Which field must the PATCH body carry, and why will setting status to todo fail?
4. State the once-only rule for tokens in your own words, and name two commands it applies to.
5. In paperclipai project update, what does passing the literal string null do, and how does it differ from omitting the flag entirely?
6. How do you archive a project, and how do you reverse that in the same command family?
7. Which two cost/budget commands take no --company-id, and why not?
8. What are the three valid values of --mode on skills agent sync, and which one is destructive?
9. Name the two hard limits on agent secret proposals, and say how long a proposal survives before it expires.
10. Which HTTP header must accompany an issue checkout and every agent-initiated issue state change, and where does its value come from?
11. What is the difference between paperclipai routine and paperclipai routines, and why does the difference matter operationally?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

12. Why is there no bulk re-enable command for routines, and what do you use instead?

Operating Reference — Beyond the Labs

These cross-cutting topics are assessed throughout the course but do not each have a dedicated lab. Read this section alongside the labs so you can answer the knowledge questions and run a real Paperclip company safely afterwards.

Choosing the deployment mode

- Loopback (the default) — trusted local only, the least exposed and the right choice for this course.
- `--bind lan` — reachable from your network; authentication becomes mandatory, not optional.
- `--bind tailnet` — reachable over a private mesh; again authentication is required.
- Hosted / container — an external PostgreSQL, a persistent volume for instance config and credentials, PAPERCLIP_TOOL_ACTION_SIGNING_SECRET for tool approvals, and an allowed-hostnames list.
- /api/health is the readiness probe at every one of these levels.

Budget layering in practice

- Set the COMPANY cap before you hire anyone; set the AGENT cap below it; add a PROJECT cap where one project must not consume the company.
- Caps are enforced at CHECKOUT, not at billing — that is what makes unattended operation safe.
- Record a baseline (current spend, expected run frequency) before you rely on a cap.
- Treat it as a two-stage circuit breaker: warn, then stop. A cap you would be unhappy to actually pay is set too high.
- Cost is tracked per token by provider, model, agent and project — read the actual figure before adjusting any cap.

Routines and schedule safety

- Create every routine PAUSED. A routine that is live before it has ever succeeded is unreviewed scheduled spend.
- State the trigger explicitly — a cron expression AND an IANA timezone (e.g. `30 16 * * *`, Asia/Singapore). An implicit timezone is a production incident waiting to happen.
- Prove it with exactly one bounded manual run, reviewed by a human, then leave it paused until you have read the generated issue, the run and the cost.
- Name the artifact the routine must produce, or the run is successful but unverifiable.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Governance, approvals and intervention

- Approvals make human authority explicit: the agent proposes, the action HOLDS, a human decides with a recorded rationale. A close without a rationale is an unaudited decision.
- The activity log is durable — mutations, heartbeats, cost events and approvals. The audit loop closes when the whole operation reconstructs from the log alone.
- Use the least irreversible control first: inspect → pause → reassign → terminate. Terminate is the last rung and is not reversible.
- An unexplained gap in the activity log usually means work was done outside a run. Record it honestly — the gap is itself an audit finding.

Working safely with synthetic data

- All six company data sets in this course are fabricated. The NorthPoint market data is not market data and must never inform a real trading decision.
- The TalentBridge candidate file deliberately records NO protected characteristics — no age, gender, race, religion, marital status or nationality. Keep it that way in any real hiring workflow.
- Keep credentials in the runtime or the Paperclip secret store with scoped injection at heartbeat time — never in prompts, notes or Git.

Course Preparation and Practice

- First pass: complete every lab in order, reading each lab folder's README and its data/ set.
- Second pass: rebuild one company from scratch without the guide, until the control sequence — cap, hire, confine, assign, verify, audit — is automatic.
- Review the 'Test it' criterion for any lab you found hard; it states exactly what the assessor will look for.
- Practise writing acceptance criteria that name an artifact, its path and its required fields — this is the skill most learners under-practise.
- Sharpen your readiness with the Tertiary Infotech practice exam: https://exams.tertiaryinfotech.com
- Then apply it: pick ONE real internal workflow whose inputs and outputs are already observable, cap it low, and audit it before you scale.

Glossary

- **Control plane —** The layer that coordinates agents, goals, work, cost and governance. Paperclip is a control plane; it does not perform the reasoning itself.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Adapter / runtime —** The agent that actually reasons and uses tools — Claude Code, Codex, Cursor, Gemini CLI, a bash process or an HTTP bot. 'If it can receive a heartbeat, it's hired.'
- **Company —** The boundary for one mission, its human members, AI employees, projects, issues, budget and activity history. One deployment holds many companies.
- **Mission —** The measurable outcome that defines why a company exists — it needs a target, a metric and a boundary, or it cannot be verified.
- **CEO —** The first hire and top of the org chart; owns goal alignment, decomposition and final synthesis, and may hire further agents under approval.
- **Org chart —** The explicit reporting structure — who reports to whom, with what authority, budget and workspace.
- **Agent —** A work-performing role defined by six fields: role, manager, adapter, instructions, heartbeat and budget.
- **Role instruction —** A contract stating purpose, allowed inputs, output schema, exclusions and escalation path — not a personality.
- **Workspace —** A dedicated local root for one role's files and evidence, isolated with git worktrees so every change is attributable.
- **Heartbeat —** A bounded execution window with five triggers: a schedule/interval tick, a task assignment, an @-mention in a comment, a manual Run Heartbeat, or an approval being resolved. Between heartbeats the agent is dormant.
- **Atomic checkout —** The execution lock that stops two heartbeats double-working the same issue or racing past a budget ceiling.
- **Issue —** A unit of work carrying full goal ancestry, so the agent sees why it exists and not only what to do.
- **Goal lineage —** The chain company mission → project goal → parent issue → child issue that gives an agent its direction.
- **Acceptance criterion —** A statement naming the required evidence — the artifact, its path and its fields. Never 'the work is done'.
- **Hand-off —** Passing an artifact between agents. It is accepted only when the receiver can consume it without asking the producer a question.
- **Run —** One agent execution instance, with a structured log, a cost entry and an observable result.
- **Routine —** Recurring work triggered by cron, webhook or API. Create it paused, prove it once by hand, then let the schedule own it.
- **Budget layering —** Company, project and agent caps. The innermost cap that binds fires first, and enforcement happens at checkout.
- **Approval —** A governance gate: the agent proposes a hire, strategy or risky action; the action holds; a human decides with a recorded rationale.
- **Activity log —** The durable record of mutations, heartbeats, cost events and approvals — the basis of the audit loop.
- **Intervention ladder —** Inspect → pause → reassign → terminate. Always use the least irreversible control that resolves the situation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Skill —** A reusable agent capability imported into an org-wide library and attached to specific agents — capability without retraining.
- **Portability —** Export/import of a whole company with secrets scrubbed and identifier collisions re-mapped.

Appendix A — Command Reference

The commands you will actually use in this course, grouped by purpose. The CLI binary is `paperclipai`; the npm package has the same name.

Install and verify

```
node --version                        # must be v20 or later
npx paperclipai onboard --yes         # download, configure, initialise, start
npx paperclipai run                   # start an already-configured instance
npx paperclipai doctor                # diagnose
npx paperclipai doctor --repair       # diagnose and attempt repair
npx paperclipai configure             # edit configuration
npx paperclipai configure --section server   # deployment mode / binding
npx paperclipai env                   # show non-sensitive instance settings
curl -sS http://localhost:3100/api/health
```

Managed install and service

```
npx paperclipai install                # installs to ~/.local/bin/paperclipai
paperclipai service install            # register the background service
paperclipai channels                   # stable | beta | nightly | canary
sudo journalctl -u paperclip -f        # follow the service log (Linux/WSL2)
```

Note: The managed install at ~/.local/bin/paperclipai carries automatic updates and rollback. Beyond `service install` the documentation does not publish a full `service` subcommand list — check `paperclipai service --help` on your own install rather than assuming a subcommand exists.

Verification in a workspace

```
git -C workspace/<role> init -q
git -C workspace/<role> commit -q --allow-empty -m 'baseline'
git -C workspace/<role> status --short
git -C workspace/<role> diff HEAD --stat
git -C workspace/<role> diff HEAD -- <artifact>
```

Note: The CLI reference in the official documentation runs to more than thirty pages covering company, project, goal, issue, agent, prompt, run, routine, adapter, approval, activity, dashboard, cost, workspace, skills, secrets and more. Treat the list above as the working subset for this course, not the whole surface.

Appendix B — API Notes

Paperclip exposes a REST API on the same port as the UI. You do not need it for the labs, but the following facts explain behaviour you will see and are worth knowing before you automate anything.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Runs identify themselves with the X-Paperclip-Run-Id header — this is how a write is attributed to a specific execution.
- Checkout returns 409 Conflict when another agent holds the issue. Treat 409 as TERMINAL, not as a retry signal.
- Never PATCH an issue's status directly to in_progress; use the checkout mechanism.
- Terminal issues (done, cancelled) reopen only via an explicit reopen: true.
- Skills sync via POST /api/agents/{agentId}/skills/sync with mode add, remove or replace.
- CROSS_ISSUE_INFLUENCE_LIMIT is 20 cross-issue writes per heartbeat run.
- Board tokens are prefixed pcp_board_ and are shown once at creation only.

Appendix C — Defaults, Limits and Thresholds

A consolidated reference. Every value here comes from the official documentation.

Numeric defaults and limits

Item	Value
Node.js minimum	20
API / UI port	3100 (default host 127.0.0.1)
Embedded PostgreSQL port	54329
Master key size	32 bytes
Budget warning threshold	80% utilisation
Budget hard stop	100% — agent paused, pauseReason = "budget"
Utilisation bar colours	green <70%, yellow 70–90%, red >90%
Suggested CEO budget	$30–50 / month
Suggested manager budget	$20–40 / month
Suggested worker budget	$10–25 / month
maxReviewRounds	default 3, range 1–50
approvalsNeeded	fixed at 1
Cross-issue writes per run	20
Heartbeat on interval	OFF by default
Suggested starting interval	once per hour
Routine minute granularity	snaps to 5-minute increments
Routine concurrency default	coalesce if active, skip missed
Secret proposals	20 pending, 20 per minute, 14-day expiry
Skill folder nesting	up to 4 levels
Claude Code maxTurnsPerRun	300
Claude Code sandbox timeout	14,400 seconds (4 hours)
Config revisions shown	10 most recent
Agent dashboard chart window	14 days

Appendix D — State Machines at a Glance

Every state machine you may be asked about

Entity	States
Issue / task	backlog → todo → in_progress → in_review → done; plus blocked, cancelled. done and cancelled are terminal.
Agent	active, idle, running, error, paused, terminated,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	pending_approval
Run	queued, running, succeeded, failed, timed out, cancelled
Approval	pending → approved / rejected / revision_requested → resubmitted → pending
Project	backlog, planned, in_progress, completed, cancelled
Workspace	provisioning, validating, ready, degraded, repairing, failed, idle, in review, running, archived, cleanup_failed
Routine (status)	active, paused, archived — only active fires. draft/running/succeeded/failed are list-view badges, not statuses.
Membership	active, pending, suspended
Issue execution state	idle, pending, changes_requested, completed

Appendix E — Troubleshooting Playbook

Symptom → likely cause → first action

Symptom	Likely cause	First action
Onboarding fails; PostgreSQL will not start	Elevated shell, or a data directory owned by another user	Close the elevated terminal. Re-run as the normal user who owns the practice folder.
Browser cannot reach port 3100	Server not running, or bound to another interface	npx paperclipai doctor, then npx paperclipai run, then re-check /api/health BEFORE touching any firewall.
Adapter verification fails	Runtime not authenticated outside Paperclip	Run the runtime's own CLI directly and sign in. Paperclip cannot authenticate it for you.
Agent never wakes	Issue not assigned; interval off (the default); agent paused	Check the assignee first — creating an issue does not wake anyone.
Everything is quiet: no runs, no spend	An unattended approval queue	Open Approvals. Approvals never expire on their own.
Agent stopped mid-work	Budget hard stop at 100%	Check the agent's utilisation and pauseReason. Raise the cap deliberately or let it stop.
Marker file in the wrong workspace	Workspace path misbound on the agent	Fix the path on the AGENT, delete the stray file, re-run the marker.
409 Conflict on checkout	Another agent holds the issue	Stop. 409 is terminal — do not retry.
422 'Only the escalated reviewer…'	Execution policy escalation is active	The escalated reviewer must advance the stage. This is the policy working, not a bug.
Routine fires at the wrong hour	Trigger saved without an explicit IANA timezone	Re-save the trigger with Asia/Singapore stated explicitly.
The approvals queue is not moving	Approvals do not expire on their own — one is simply waiting for a human decision	Open Approvals and decide. This is the documented cause of a stalled queue.
Workspace shows degraded or failed	Provisioning or validation problem	Read the workspace state and repair before blaming the agent.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg